

weft 1.0.4

A Modern Literate Programming Tool

Cuauhtemoc Pacheco
Derived from nuweb 1.64 by Preston Briggs¹
with contributions by Keith Harwood, Simon Wright,
Marc W. Mengel, John D. Ramsdell, and Gregor Goldbach

July 18, 2026

¹Original nuweb work supported by ARPA, through ONR grant N00014-91-J-1989.

Contents

1	Introduction	1
1.1	weft	1
1.2	Writing weft	3
1.2.1	Source File Extension: <code>.weft</code>	3
1.2.2	The Major Commands	4
1.2.3	The Minor Commands	7
1.3	Sectioning commands	8
1.4	Running weft	9
1.5	Markdown Output	10
1.6	Section Markers and Language Auto-Detection	11
1.6.1	Overriding the Language with <code>@l</code>	11
1.6.2	Mid-Line Inline Expansion	12
1.6.3	The <code>@L</code> Command: Defining New Languages	13
1.6.4	The Language Table	13
1.6.5	Syntax Highlighting in Markdown Output	14
1.6.6	Output Format of Section Markers	14
1.6.7	Debugging Tangled Code	14
1.7	Restrictions	18
1.8	Project Introspection	19
1.8.1	Map (<code>-m</code>)	19
1.8.2	Extract (<code>-e</code>)	19
1.8.3	Reverse Map (<code>-R</code>)	19
1.9	Acknowledgements	20
2	The Overall Structure	21
2.1	Files	21
2.1.1	The Main Files	23
2.1.2	Support Files	25
2.2	The Main Routine	25
2.2.1	Command-Line Arguments	26
2.2.2	Help Message	31
2.2.3	Short Flags	33
2.2.4	File Names	37
2.3	Pass One	42
2.3.1	Accumulating Definitions	46
2.3.2	Block Comments	48
2.3.3	Fixing the Cross References	54
2.3.4	Dealing with fragment parameters	54
2.4	Writing the Latex File	58
2.4.1	Formatting Definitions	62
2.4.2	Generating the Indices	81
2.5	Writing the Markdown File	88

2.5.1	Formatting Definitions in Markdown	92
2.5.2	Generating the Indices in Markdown	105
2.5.3	Identifier Cross-References in Markdown	110
2.6	Map and Extract Output	113
2.6.1	JSON Map Output	113
2.6.2	Fragment Extraction	116
2.6.3	Reverse Map Output	118
2.7	Writing the Output Files	123
3	The Support Routines	126
3.1	Source Files	126
3.1.1	Global Declarations	126
3.1.2	Local Declarations	127
3.1.3	Reading a File	127
3.1.4	Opening a File	131
3.2	Scraps	132
3.2.1	Collecting Page Numbers	158
3.3	Names	160
3.3.1	Language Auto-Detection	168
3.3.2	User-Defined Languages	170
3.3.3	Weave Format Declaration	172
3.3.4	Collecting File and Fragment Names	173
3.3.5	Comment Styles	176
3.4	Searching for Index Entries	185
3.4.1	Retrieving scrap uses	186
3.4.2	Building the Automata	189
3.4.3	Searching the Scraps	193
3.5	Labels	199
3.6	Memory Management	202
3.6.1	Allocating Memory	203
3.6.2	Freeing Memory	204
4	Man page	205
5	Indices	210
5.1	Files	210
5.2	Fragments	210
5.3	Identifiers	214

Chapter 1

Introduction

In 1984, Knuth introduced the idea of *literate programming* and described a pair of tools to support the practise [3]. His approach was to combine Pascal code with T_EX documentation to produce a new language, WEB, that offered programmers a superior approach to programming. He wrote several programs in WEB, including `weave` and `tangle`, the programs used to support literate programming. The idea was that a programmer wrote one document, the web file, that combined documentation (written in T_EX [4]) with code (written in Pascal).

Running `tangle` on the web file would produce a complete Pascal program, ready for compilation by an ordinary Pascal compiler. The primary function of `tangle` is to allow the programmer to present elements of the program in any desired order, regardless of the restrictions imposed by the programming language. Thus, the programmer is free to present his program in a top-down fashion, bottom-up fashion, or whatever seems best in terms of promoting understanding and maintenance.

Running `weave` on the web file would produce a T_EX file, ready to be processed by T_EX. The resulting document included a variety of automatically generated indices and cross-references that made it much easier to navigate the code. Additionally, all of the code sections were automatically prettyprinted, resulting in a quite impressive document.

Knuth also wrote the programs for T_EX and METAFONT entirely in WEB, eventually publishing them in book form [5, 6]. These are probably the largest programs ever published in a readable form.

Inspired by Knuth's example, many people have experimented with WEB. Some people have even built web-like tools for their own favorite combinations of programming language and typesetting language. For example, CWEB, Knuth's current system of choice, works with a combination of C (or C++) and T_EX [8]. Another system, FunnelWeb, is independent of any programming language and only mildly dependent on T_EX [10]. Inspired by the versatility of FunnelWeb and by the daunting size of its documentation, Preston Briggs decided to write his own, very simple, tool for literate programming.¹

`weft` is a fork of `nuweb`. The original `nuweb` code, written by Preston Briggs and maintained over the years by several contributors, provided an excellent foundation: a simple, language-independent literate programming tool. `weft` builds on that foundation with a modernized codebase, a modular source structure organized by concept, the removal of the unmaintained HTML output path, and the addition of a Markdown weave output format. The goal is to continue evolving the tool while preserving the simplicity and philosophy that made `nuweb` valuable in the first place.

1.1 `weft`

`weft` works with any programming language and produces documentation in L^AT_EX [7] or Markdown format. L^AT_EX was the original choice because it supports a multi-level sectioning scheme and has facilities for drawing figures. Markdown support was added to allow literate programs to produce lightweight documentation

¹There is another system similar to `nuweb`, written by Norman Ramsey, called `noweb` [9]. It perhaps suffers from being overly Unix-dependent and requiring several programs to use. On the other hand, its command syntax is very nice. In any case, `nuweb` certainly owes its name and a number of features to his inspiration.

that renders well on platforms like GitHub, GitLab, and any Markdown viewer, without requiring a T_EX installation.

Support for arbitrary programming languages was a design goal because programmers work in many languages (and sometimes combinations of several languages), *e.g.*, C, Fortran, C++, yacc, lex, Scheme, assembly, Postscript, and so forth. The need to support arbitrary programming languages has many consequences:

No prettyprinting Both WEB and CWEB are able to prettyprint the code sections of their documents because they understand the language well enough to parse it. Since weft aims to support *any* language, this feature must be abandoned. However, weft does allow particular individual formulas or fragments of L^AT_EX code to be formatted and still be parts of output files. Also, keywords in scraps can be surrounded by @_ to have them be bold in the output.

No index of identifiers Because WEB knows about Pascal, it is able to construct an index of all the identifiers occurring in the code sections (filtering out keywords and the standard type identifiers). Unfortunately, this isn't as easy for a language-independent tool. weft doesn't know what an identifier looks like in each language and certainly doesn't know all the keywords. (On the other hand, see the end of Section 1.2.3)

Of course, there must be some compensation for these losses or the whole idea would be a waste. Here are the advantages:

Simplicity The majority of the commands in WEB are concerned with control of the automatic prettyprinting. Since weft doesn't prettyprint, many commands are eliminated. A further set of commands is subsumed by L^AT_EX and may also be eliminated. As a result, the set of commands is reduced to only four members (explained in the next section). This simplicity is also reflected in the size of this tool, which is quite a bit smaller than the tools used with other approaches.

No prettyprinting Everyone disagrees about how their code should look, so automatic formatting annoys many people. One approach is to provide ways to control the formatting. weft's approach is simpler—it performs no automatic formatting and therefore allows the programmer complete control of code layout. weft does allow individual scraps to be presented in either verbatim, math, or paragraph mode in the T_EX output.

Control weft also offers the programmer complete control of the layout of his output files (the files generated during tangling). Of course, this is essential for languages that are sensitive to layout; but it is also important in many practical situations, *e.g.*, debugging.

Speed Since weft doesn't do too much, the weft tool runs quickly. By default it performs only tangling; weaving is activated with -w. When weaving is requested, both operations execute in a single pass.

Page numbers Inspired by the example of noweb, weft refers to all scraps by page number to simplify navigation. If there are multiple scraps on a page (say, page 17), they are distinguished by lower-case letters (*e.g.*, 17a, 17b, and so forth).

Multiple file output The programmer may specify more than one output file in a single weft file. This is required when constructing programs in a combination of languages (say, Fortran and C). It's also an advantage when constructing very large programs that would require a lot of compile time.

This last point is very important. By allowing the creation of multiple output files, the need for monolithic programs is avoided. This supports the creation of very large programs by groups of people.

A further reduction in compilation time is achieved by first writing each output file to a temporary location, then comparing the temporary file with the old version of the file. If there is no difference, the temporary file can be deleted. If the files differ, the old version is deleted and the temporary file renamed. This approach works well in combination with **make** (or similar tools), since **make** will avoid recompiling untouched output files.

1.2 Writing weft

The bulk of a weft file will be ordinary L^AT_EX (or Markdown, when using the `-w md` flag). In fact, any L^AT_EX file can serve as input to weft and will be simply copied through, unchanged, to the documentation file—unless a weft command is discovered. Similarly, any Markdown file can be processed with `-w md` and will be copied through to the `.md` output. All weft commands begin with an “at-sign” (`@`). Therefore, a file without at-signs will be copied unchanged. weft commands are used to specify *output files*, define *fragments*, and delimit *scraps*. These are the basic features of interest to the weft tool—all else is simply text to be copied to the documentation file.

1.2.1 Source File Extension: `.weft`

weft source files use the `.weft` extension. This single, format-agnostic extension replaces the earlier `.w` and `.wmd` conventions. The prose surrounding scraps may be written in L^AT_EX or Markdown; the at-sign commands (`@o`, `@d`, `@<`, etc.) are the same in both cases.

The weave output format is specified in one of two ways:

In the source file The `@W` directive declares the intended format: `@W tex` for L^AT_EX, `@W md` for Markdown. This should appear in the preamble, before any `@o` or `@d` directives.

On the command line `-w md` or `-w tex` overrides the `@W` directive. Using `-w` alone (without a format token) reads the format from `@W`.

If no extension is given on the command line, weft appends `.weft` automatically.

Default Behavior: Tangle Only

By default, `weft myfile.weft` performs only tangling (producing the output files). No documentation is generated unless `-w` is given:

<code>weft programa.weft</code>	tangle only (default)
<code>weft -w programa.weft</code>	weave per <code>@W</code> (error if missing)
<code>weft -w md programa.weft</code>	weave Markdown (override)
<code>weft -w tex programa.weft</code>	weave LaTeX (override)

The `-t` flag suppresses weaving even if `-w` is given. The `-o` flag suppresses tangling.

Cosmetic Fences

When writing `.weft` files with Markdown prose, it is often convenient to surround scrap definitions with Markdown fenced code blocks (triple backticks) so that the source file renders nicely in Markdown previewers such as GitHub or VS Code. For example:

Some prose in Markdown.

```
'''  
@d My fragment  
@{code here  
@}  
'''
```

More prose.

weft silently strips these fence lines (lines that consist of three or more backticks, optionally followed by a language identifier) during Markdown weaving. They do not appear in the generated `.md` file and do not affect tangling. This is purely a cosmetic convenience for making the source file readable in any Markdown viewer.

Note that fences are only stripped in Markdown mode (`-w md`). In L^AT_EX mode, backtick lines are treated as ordinary text.

1.2.2 The Major Commands

Files and fragments are defined with the following commands:

@o *file-name flags scrap* Output a file. The file name is terminated by whitespace.

@d *fragment-name scrap* Define a fragment. The fragment name is terminated by a return or the beginning of a scrap.

@q *fragment-name scrap* Define a fragment. The fragment name is terminated by a return or the beginning of a scrap. This a quoted fragment.

A specific file may be specified several times, with each definition being written out, one after the other, in the order they appear. The definitions of fragments may be similarly specified piecemeal.

Fragment Names with Embedded Parameters

Fragment names can carry *parameters*—placeholder values embedded directly in the name using the **@'value@'** syntax. Inside the fragment body, **@1**, **@2**, ... **@9** refer to the first, second, ... ninth parameter in order of appearance.

For example, a reusable validation fragment:

```
@d Validate field @'name@' type @'string@'
@{if (!data.@1) throw new Error("Missing @1");
if (typeof data.@1 !== '@2')
    throw new TypeError("@1 must be @2");
@}
```

Each invocation supplies its own arguments:

```
<@Validate field @'email@' type @'string@'@>
<@Validate field @'age@' type @'number@'@>
```

The first invocation expands to:

```
if (!data.email) throw new Error("Missing email");
if (typeof data.email !== 'string')
    throw new TypeError("email must be string");
```

Note three key points:

- The values in the *definition* (**@'name@'**, **@'string@'**) serve as default arguments and as documentation—they make the fragment name readable.
- At each *invocation*, different values can be substituted (**@'email@'** replaces **@1**, **@'string@'** replaces **@2**).
- If a fragment name is abbreviated with ..., any missing arguments fall back to the defaults given in the definition.

The difference between a quoted fragment (**@q**) and an ordinary one (**@d**) is that inside a quoted fragment fragments are not expanded on output. Rather, they are formatted as uses of fragments so that the output file can itself be **weft** source. This allows you to create files containing fragments which can undergo further processing before the fragments are expanded, while also describing and documenting them in one place.

You can have both quoted and unquoted fragments with the same name. They are written out in order as usual, with those introduced by **@q** being quoted and those with **@d** expanded as normal.

In quoted fragments, the **@f** filename is quoted as well, so that when it is expanded it refers to the finally produced file, not any of the intermediate ones.

Scraps

Scraps have specific begin markers and end markers to allow precise control over the contents and layout. Note that any amount of whitespace (including carriage returns) may appear between a name and the beginning of a scrap. Scraps may also appear in the running text where they are formatted (almost) identically to their use in definitions, but don't appear in any code output.

`@{anything@}` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in verbatim mode. Using the `-l` option will cause the program to typeset the scrap with the help of L^AT_EX's `listings` package.

`@[anything@]` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in paragraph mode, allowing sections of T_EX documents to be scraps, but still be prettyprinted in the document.

`@(anything@)` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in math mode. This allows this scrap to contain a formula which will be typeset nicely.

Inside a scrap, several special sequences are recognized.

Fragment invocation.

`<fragment-name>` Expands the named fragment inline. It is an error to specify recursive invocations. If the fragment name contains embedded parameters, the invocation supplies arguments at the same positions (see above).

When the reference appears *mid-line*—that is, after at least one visible character on the same line—section markers, `#line` directives, and fragment-name comments are automatically suppressed. The fragment content is spliced seamlessly into the surrounding text. At start-of-line positions (including indented lines), markers are emitted normally. See Section 1.6 for details.

Argument substitution.

`@1`, `@2`, ..., `@9` Inside a fragment body, substitutes the *n*th argument. If the argument was not supplied, the empty string is used.

At each argument position in an invocation, four forms are accepted:

`@'string@'` A literal string, copied verbatim (only `@@` is interpreted, producing a single `@`). This is the most common form.

`<fragment-name>` The fragment is expanded and its output becomes the argument value.

`@{text@}` The text is expanded as an anonymous fragment (inheriting the caller's arguments) and the result is passed as the argument. Useful for combining text and arguments into a single value.

`@1`, `@2`, ..., `@9` Forwards an argument from the calling fragment to the called one.

Other in-scrap commands.

`@xlabel@x` Marks a place inside a scrap and associates it to the label (which can be any text not containing a `@`). Expands to the reference number of the scrap followed by a numeric value. Outside scraps it expands to the same value. It is used so that text outside scraps can refer to particular places within scraps.

`@f` Inside a scrap this is replaced by the name of the current output file.

`@t` Inside a scrap this is replaced by the title of the fragment as it is at the point it is used, with all parameters replaced by actual arguments.

@# At the beginning of a line in a scrap this will suppress the normal indentation for that line. Use this, for example, when you have a `#ifdef` inside a nested scrap. Writing `@##ifdef` will cause it to be lined up on the left rather than indented with the rest of its code.

@s Inside a scrap suppresses indentation for the following fragment expansion.

Note that fragment names may be abbreviated, either during invocation or definition. For example, it would be very tedious to have to type, repeatedly, the fragment name

```
@d Check for terminating at-sequence and return name if found
```

Therefore, `weft` provides a mechanism (borrowed from Knuth) of indicating abbreviated names.

```
@d Check for terminating...
```

Basically, the programmer need only type enough characters to identify the fragment name uniquely, followed by three periods. An abbreviation may even occur before the full version; `weft` simply preserves the longest version of a fragment name. Note also that blanks and tabs are insignificant within a fragment name; each string of them is replaced by a single blank.

Sometimes, for instance during program testing, it is convenient to comment out a few lines of code. In C or Fortran placing `/* ... */` around the relevant code is not a robust solution, as the code itself may contain comments. `weft` provides the command

```
@%
```

only to be used inside scraps. It behaves exactly the same as `%` in the normal L^AT_EX text body.

When scraps are written to a program file or a documentation file, tabs are expanded into spaces by default. Currently, `weft` assumes tab stops are set every eight characters. Furthermore, when a fragment is expanded in a scrap, the body of the fragment is indented to match the indentation of the fragment invocation. Therefore, care must be taken with languages (*e.g.*, Fortran) that are sensitive to indentation. These default behaviors may be changed for each output file (see below).

Flags

When defining an output file, the programmer has the option of using flags to control output of a particular file. The flags are intended to make life a little easier for programmers using certain languages. They introduce little language dependences; however, they do so only for a particular file. Thus it is still easy to mix languages within a single document. There are five “per-file” flags:

- d Forces the creation of `#line` directives in the output file. These are useful with C (and sometimes C++ and Fortran) on many Unix systems since they cause the compiler’s error messages to refer to the web file rather than to the output file. Similarly, they allow source debugging in terms of the web file.
- i Suppresses the indentation of fragments. That is, when a fragment is expanded within a scrap, it will *not* be indented to match the indentation of the fragment invocation. This flag would seem most useful for Fortran programmers.
- t Suppresses expansion of tabs in the output file. This feature seems important when generating `make` files.
- cx Puts comments in generated code documenting the fragment that generated that code. The *x* selects the comment style for the language in use: `c` for C (`/* */`), `+` for C++ (`//`), and `p` for shell/Perl (`#`). If the global `-x` cross-reference flag is set the comment includes the page reference for the first scrap that generated the code.
- s Suppresses section markers and `#line` directives in the tangled output for this file. The language is still detected (from the extension or a preceding `@1`), so the Markdown weaver can emit syntax-highlighting classes, but the tangled code is emitted without any tangle annotations. This is useful for bootstrap source files or generated output that should remain clean.

These flags are still available for explicit control, but in most cases the automatic language detection described in Section 1.6 is more convenient.

1.2.3 The Minor Commands

There are some very low-level utility commands that may appear anywhere in the web file.

`@@` Causes a single “at sign” to be copied into the output.

`@_` Causes the text between it and the next `@_` to be made bold (for keywords, etc.)

`@i file-name` Includes a file. Includes may be nested, though there is currently a limit of 10 levels. The file name should be complete (no extension will be appended) and should be terminated by a carriage return. Normally the current directory is searched for the file to be included, but this can be varied using the `-I` flag on the command line. Each such flag adds one directory to the search path and they are searched in the order given.

`@rx` Changes the escape character ‘@’ to ‘x’. This must appear before any scrap definitions.

`@v` Always replaced by the string established by the `-V` flag, or a default string if the flag isn’t given. This is intended to mark versions of the generated files.

`@l tag` Sets the language for the next `@o` or `@d` directive. The tag is looked up in the language table (Section 1.6) to determine the comment style for section markers and the `class` attribute for Markdown syntax highlighting. Placed on the line before the directive it applies to.

`@L name style [+d]` Defines a custom language mapping for the section marker system (see Section 1.6). This must appear in the document preamble, before any `@o` or `@d` directives that depend on it.

`@W format` Declares the weave output format for this source file. Valid formats are `md` (Markdown) and `tex` (L^AT_EX). Used by `-w` when no explicit format is given on the command line. A CLI format (`-w md` or `-w tex`) overrides this directive. Should appear in the document preamble.

In the text of the document, that is outside scraps, you may include scrap-like material.

`@{Anything@}` The included material, the *Anything*, is typeset as if it appeared inside a scrap. This is useful for referring to fragments in the text and for describing the literate programming process itself.

`@<Fragment name@>` The fragment named is expanded in place in the text. The expansion is presented verbatim, it is not interpreted for typesetting, so any special environment must be set up before and after this is used.

`@c` Outside of a scrap this is used to define a block comment using all the text until the next command. Inside a scrap the last block comment is expanded using the current indentation and breaking lines between words if 60 characters are exceeded.

Finally, there are three commands used to create indices to the fragment names, file definitions, and user-specified identifiers.

`@f` Create an index of file names.

`@m` Create an index of fragment names.

`@u` Create an index of user-specified identifiers.

These are usually placed in their own section in the L^AT_EX document; for example, see Chapter 5.

Identifiers must be explicitly specified for inclusion in the `@u` index. By convention, each identifier is marked at the point of its definition; all references to each identifier (inside scraps) will be discovered automatically. To “mark” an identifier for inclusion in the index, it must be mentioned at the end of a scrap. For example,

```
@d a scrap @{
  Suppose the variables FOO and BAR are being declared
  inside this scrap.
@| FOO BAR @}

```

Alphabetic identifiers have been used in this example, but any string of characters (not including whitespace or @ characters) will do. Therefore, it's possible to add index entries for things like <=< if desired. An identifier may be declared in more than one scrap.

In the generated index, each identifier appears with a list of all the scraps using and defining it, where the defining scraps are distinguished by underlining. Note that the identifier doesn't actually have to appear in the defining scrap; it just has to be in the list of definitions at the end of a scrap.

1.3 Sectioning commands

For larger documents the indexes and usage lists get rather unwieldy and problems arise in naming things so that different things in different parts of the document don't get confused. `weft` has a sectioning command which keeps the fragment names and user identifiers separate. Thus, you can give a fragment in one section the same name as a fragment in another and the two won't be confused or connected in any way. Nor will user identifiers defined in one section be referenced in another. Except for the fact that scraps in successive sections can go into the same output file, this is the same as if the sections came from separate input files.

However, occasionally you may really want fragments from one section to be used in another. More often, you will want to identify a user identifier in one section with the same identifier in another (as, for example, a header file defined in one section is included in code in another). Extra commands allow a fragment defined in one section to be accessible from all other sections. Similarly, you can have scraps which define user identifiers and export them so that they can be used in other sections.

`@s` Start a new section.

`@S` Close the current section and don't start another.

`@d+ fragment-name scrap` Define a fragment which is accessible in all sections, a global fragment.

`@D+` Likewise

`@q+` Likewise

`@Q+` Likewise

`@m+` Create an index of all such fragments.

`@u+` Create an index of globally accessible user identifiers.

There are two kinds of section, the base section which is where normally everything goes, and local sections which are introduced with the `@s` command. A local section comprises everything from the command which starts it to the one which ends it. A `@s` command will start a new local section. A `@S` command closes the current local section, but doesn't open another, so what follows goes into the base section. Note that fragments defined in the base section aren't global; they are accessible only in the base section, but they are accessible regardless of any local sections between their definition and their use.

Within a scrap:

`@<+fragment-name>` Expand the globally accessible fragment with that name, rather than any local fragment.

`@+` Like `@|` except that the identifiers defined are exported to the global realm and are not directly referenced in any scrap in any section (not even the one where they are defined).

@- Like @| except that the identifiers are imported to the local realm. The cross-references show where the global variables are defined and defines the same names as locally accesible. Uses of the names within the section will point to this scrap.

Note that the + signs above are part of the commands. They are not part of the fragment names. If you want a fragment whose name begins with a plus sign, leave a space between the command and the name.

1.4 Running weft

weft is invoked using the following command:

```
weft flags file-name...
```

One or more files may be processed at a time. If a file name has no extension, `.weft` will be appended. While a file name may specify a file in another directory, the resulting documentation file will always be created in the current directory. For example,

```
weft /foo/bar/quux
```

will take as input the file `/foo/bar/quux.weft` and will produce the tangled output files only. To also generate documentation, use `-w` (see below).

By default, weft performs only tangling—it generates the output files defined by `@o` directives and nothing else. Documentation (weaving) is produced only when the `-w` flag is given. When `-w` is present, both tangling and weaving execute in a single pass. Three flags control what weft produces:

- t Suppress weaving (useful to override `-w` in scripts).
- o Suppress generation of the output files (weave only).
- c Avoid testing output files for change before updating them.

Thus, the command

```
weft -to /foo/bar/quux
```

would simply scan the input and produce no output at all.

There are several additional command-line flags:

- v For “verbose”, causes weft to write information about its progress to `stderr`.
- n Forces scraps to be numbered sequentially from 1 (instead of using page numbers). This form is perhaps more desirable for small webs.
- s Doesn’t print list of scraps making up each file following each scrap.
- d Print “dangling” identifiers – user identifiers which are never referenced, in indices, etc.
- p *path* Prepend *path* to the filenames for all the output files.
- w [*format*] Activate weaving (documentation generation). Without `-w`, no documentation file is produced. The optional *format* argument selects the output format:
 - `-w md` — force Markdown output (`.md` file).
 - `-w tex` — force L^AT_EX output (`.tex` file).
 - `-w` alone — read the format from the `@W` directive in the source file. If `@W` is absent, weft reports an error.

A format given on the command line overrides any `@W` directive. When Markdown output is selected, sequential numbering (`-n`) is forced automatically, since Markdown has no concept of page numbers. The Markdown output uses HTML anchors (`<a id="...">`) for cross-references and `<pre><code>` blocks for scraps, allowing clickable navigation between definitions and references. Cosmetic fence lines (triple backticks) in the source are silently stripped.

- `-x` Include cross-reference numbers in scrap comments.
- `-h options` Provide options for the hyperref package.
- `-r` Turn on hyperlinks. You must include the `—usepackage—` options in the text.
- `-l` Use the `listings` package for formatting scraps. Use this if you want to have a pretty-printer for your scraps. In order to e.g. have pretty Perl scraps, include the following L^AT_EX commands in your document:


```
\usepackage{listings}
...
\lstset{extendedchars=true, keepspaces=true, language=perl}
```

 See the `listings` documentation for a list of formatting options. Be sure to include a `\usepackage{listings}` in your document.
- `-V string` Provide *string* as the replacement for the `@v` operation.
- `-m` Emit a JSON map of the project to stdout and exit. The map describes every output file and every fragment, their definition locations, usage relationships, and transitive dependencies. Neither weaving nor tangling is performed. See Section 2.6 for details.
- `-e name` Extract a single fragment by name and emit its code and dependency information as Markdown to stdout. The match uses the same prefix logic as `@<name...@>`. Neither weaving nor tangling is performed. See Section 2.6 for details.
- `-R file[:line]` Read section markers from a tangled output file and emit JSON mapping to stdout. If a line number is given, emit the single innermost scrap covering that line. Without a line number, emit all scrap regions in the file. No `.weft` source files are needed—the reverse map operates entirely on the tangled output. See Section 2.6 for details.

1.5 Markdown Output

When invoked with `-w md`, `weft` generates a Markdown file (`.md`) instead of a L^AT_EX file (`.tex`). This allows literate programs to be read on any platform that renders Markdown, including GitHub, GitLab, and local editors like VS Code.

The Markdown output preserves all the essential features of the L^AT_EX output—scrap definitions, cross-references, and indices—but uses a different set of formatting conventions:

Scrap headers File definitions appear as `***filename*** **N** =` and fragment definitions as `***<Name* N>*** =`, where *N* is the sequential scrap number.

Scrap bodies Code is wrapped in `<pre><code>` HTML blocks (not fenced code blocks). This allows fragment references within scraps to be rendered as clickable HTML links with italic and bold formatting, which would not be possible inside Markdown fenced blocks.

HTML escaping Characters `<`, `>`, and `&` inside scraps are escaped as `<`, `>`, and `&` respectively.

Cross-references Each scrap has an HTML anchor (`<a id="weftN">`) and cross-reference links use `<a href="#weftN">`. Footer notes (“defined by”, “referenced in”) appear as Markdown blockquotes.

Indices The `@f`, `@m`, and `@u` commands produce Markdown lists with links. In the user identifier index, defining scraps appear in bold.

Sequential numbering The `-n` flag is forced automatically, since Markdown has no page numbers. Scraps are numbered 1, 2, 3, ... without letter suffixes.

Prose Text outside scraps passes through unmodified. The author writes Markdown prose directly in the `.weft` source file; `weft` does not interpret or transform it.

Syntax highlighting When the language of a scrap can be determined—from a preceding `@l` tag, the `@L` command, or the file extension of an `@o` name—`weft` emits `<code class="language-X">` on the opening tag. This allows client-side highlighters (`highlight.js`, `Prism`) and platforms like GitHub to apply syntax colouring automatically.

Tangling (output file generation) works identically regardless of the weave format. When `-w` is given, a single invocation produces both the documentation and the tangled output files in one pass.

1.6 Section Markers and Language Auto-Detection

When debugging tangled code, it is useful to correlate lines in the generated file back to the original web source. `weft` can auto-detect the programming language and generate *section markers*—comments that delimit the opening and closing of each scrap in the tangled output, including the source file name and line number. For C and C++, it also generates `#line` directives automatically.

The key idea is simple:

file extension → language → comment style → section markers in tangled output

For example, `@o server.js` has the extension `.js`. `weft` looks up `js` in its language table, finds that JavaScript uses `//` comments, and wraps each scrap in the tangled `server.js` with `//` markers:

```
// {1: web.weft:12}
console.log("hello");
// {1: }
```

Similarly, `@o main.c` matches `c`, which uses `/* */` comments and also enables `#line` directives:

```
#line 5 "core.weft"
/* {1: core.weft:5} */
#include <stdio.h>
/* {1: } */
```

And `@o config.py` matches `py`, using `#` comments:

```
# {3: settings.weft:40}
DATABASE_URL = "postgres://localhost/app"
# {3: }
```

This happens automatically—no flags are needed. The extension alone is enough.

1.6.1 Overriding the Language with `@l`

Sometimes the file extension is not enough, or you want a different behaviour. The `@l` command (lowercase) placed on the line before an `@o` or `@d` directive overrides the extension:

```
@l sql
@o queries.txt
@{SELECT * FROM users;
@}
```

Here the file is `queries.txt`, whose extension `.txt` has no comment style. But `@l sql` tells weft to use `--` comments, so the tangled output gets:

```
-- {1: db.weft:5}
SELECT * FROM users;
-- {:1}
```

`@l` also works on `@d` (fragment) definitions. This is useful for fragments that will be included in files of a specific language:

```
@l python
@d Validate email
@{import re
def validate(email):
    return bool(re.match(r'^[~@]+@[~@]+$', email))
@}
```

The language tag is stored on the fragment and used both for section markers in the tangled output and for syntax highlighting in the Markdown weave output.

`@l` is format-agnostic: it works identically in $\text{\LaTeX}$ and Markdown source files. Unlike the earlier approach of encoding the language in Markdown fences (`'''python`) or $\text{\LaTeX}$ markers (`%%python`), `@l` uses the same at-sign syntax as every other weft command.

Note that cosmetic fences (`'''`) may still be used around `@o/@d` definitions for readability in Markdown previewers—they are silently stripped during Markdown weaving (see Section 1.2.1). But fences no longer set the language; use `@l` for that.

Priority rules. When multiple sources of language information are available, weft resolves them in this order:

1. **Explicit per-file flags** (`-d`, `-s`, `-cx`, etc.) on the `@o` line always win for tangle behaviour.
2. **@l tag** immediately before the directive—overrides the extension.
3. **File extension** of the `@o` name—the default when no `@l` is present.

1.6.2 Mid-Line Inline Expansion

When a fragment reference `@<name>` appears in the middle of a line—after at least one visible character—weft automatically suppresses all tangle annotations for that expansion: section markers, `#line` directives, and fragment-name comments. The fragment content is spliced directly into the surrounding text, producing clean inline output.

This makes small constant fragments practical:

```
@d port @{4000@}
@d host @"localhost"@

@l c
@o config.c -s
@{const char *host = @<host>;
int port = @<port>;
@}
```

Tangling produces:

```
const char *host = "localhost";
int port = 4000;
```

If the same reference appears at the *start* of a line (even if indented), it is treated as a block expansion and markers are emitted normally. No special flags or directives are needed—the position on the line is the only thing that determines the behaviour.

This works with any language and any fragment size. When a multi-line fragment is expanded mid-line, its content is still spliced without markers—the rule is based on the *reference site*, not the fragment body.

1.6.3 The @L Command: Defining New Languages

The built-in table covers over 250 entries, but it cannot anticipate every language or every project’s preferences. The @L command defines new language mappings—or overrides existing ones—directly in the document preamble.

```
@L name style [+d]
```

- *name* is the tag that will be recognised in @l commands and file extensions (@o file.name).
- *style* is the comment syntax: //, #, --, /*, or <!--.
- +d (optional) enables #line directive generation.

Examples:

```
@L wgs1 //          % WebGPU shading language: // comments
@L cedar //         % Cedar policy language
@L c //             % override built-in: use // instead of /* */
```

How @L interacts with the rest of the system. User-defined entries are searched *before* the built-in table, so @L c // effectively changes the comment style for all subsequent @o main.c directives (and for any @l c tag) from /* */ to //. Up to 32 user-defined languages may be registered.

@L should appear in the document preamble, before any @o or @d that depends on it. It is processed during pass 1 and silently skipped during weaving.

1.6.4 The Language Table

weft includes a built-in table of over 250 entries (languages, file extensions, and common aliases), organised by comment style. Each entry maps a tag to a comment style and optionally enables #line directives. The tag is matched against @l tags and file extensions alike—there is no distinction between them.

Comment style	Tags (representative)
/* */ + #line	c, h, cpp, cc, cxx, hpp, hxx, objc, objcpp, m, mm, ino, pde, cu, cuh, cilk
/* */	css, scss, sass, less, stylus, cl (OpenCL)
//	js, ts, jsx, tsx, mjs, cjs, mts, svelte, vue, java, scala, kotlin, kt, groovy, gradle, clj, cs, fs, fsx, vb, go, rust, rs, zig, swift, dart, odin, mojo, carbon, php, graphql, prisma, glsl, hlsl, wgs1, metal, vert, frag, proto, thrift, fbs, capnp, sol, reason, res, ml, sml, fut, apex, bicep, jsonnet, qml, p4
#	python, py, pyx, pyi, sh, bash, zsh, fish, ksh, nu, perl, pl, ruby, rb, r, tcl, awk, sed, coffee, makefile, mk, cmake, just, meson, bazel, bzl, julia, jl, nim, crystal, cr, elixir, ex, yaml, yml, toml, dockerfile, terraform, tf, hcl, nix, conf, ini, env, editorconfig, gitignore, powershell, ps1, xdc, sdc
--	lua, luau, sql, psql, plsql, plpgsql, mysql, sqlite, hive, haskell, hs, elm, purescript, purs, ada, adb, ads, vhdl, vhd, lean, lean4, agda, idris, idr, erlang
<!-- -->	html, htm, xml, svg, xhtml, xsl, xsd, dtd, rss, xaml, csproj, jsp, aspx, cshtml, erb, ejs, hbs, njk, twig, blade, pug, liquid, mustache, md, mdx, rst, tex, latex
(none)	json, csv, tsv, txt, diff, patch

For the full list, see the `lang_table` array in Section 3.3.1. If a tag is not found in either the user-defined (@L) or built-in table, weft defaults to `//` comments with no `#line` directives. Tags mapped to style 0 (e.g. `json`, `txt`) produce no markers at all.

1.6.5 Syntax Highlighting in Markdown Output

When weaving to Markdown (`-w md`), weft emits the language tag on the `<code>` element:

```
<pre><code class="language-python">def hello():
    print("hello")
</code></pre>
```

This allows client-side highlighters (highlight.js, Prism) and platforms like GitHub to apply syntax colouring automatically. The language tag comes from the same chain described above: @L tag or file extension.

For @d fragments, the tag comes from a preceding @L command (there is no file extension to infer from). If no language is known, weft emits a plain `<pre><code>` without a class.

1.6.6 Output Format of Section Markers

Each scrap in the tangled file is wrapped with an opening marker at the top and a closing marker at the bottom, using the comment syntax of the detected language. The opening marker includes the scrap number and the source location (file name and line number). When fragments are nested, each expansion gets its own pair of markers, producing a clear hierarchy in the output.

1.6.7 Debugging Tangled Code

When you work with literate programming, the code you execute is *tangled output*—generated files assembled from scraps scattered across `.w` files. An error at `server.js:47` does not correspond to line 47 of any file you wrote; it is line 47 of a file that weft assembled from fragments defined in `server.w`, `routes.w`, and `database.w`. Without markers, debugging means mentally reconstructing which scrap produced which line. With markers, the tangled file tells you directly.

Reading the markers. Each marker encodes three pieces of information:

Part	Meaning
{ or {:	Opening or closing delimiter
Number	Scrap ID—unique across the entire document
file:line	Source .w file and line number (opening only)

Examples by language. JavaScript (`//` comments):

```
// {3: routes.w:42}
app.get("/api/users", async (req, res) => {
  // {7: database.w:18}
  const users = await db.query("SELECT * FROM users");
  // {7}
  res.json(users);
});
// {3}
```

Python (`#` comments):

```
# {5: models.w:31}
class User:
    def __init__(self, name, email):
        self.name = name
```

```

        self.email = email
    # {:5}

```

C (`/* */` comments with `#line` directives):

```

#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) {
    void *ptr = malloc(size);
    if (!ptr) {
        /* {4: errors.w:8} */
        fprintf(stderr, "out of memory\n");
        exit(1);
        /* {4:4} */
    }
    return ptr;
}
/* {2:2} */

```

Lua (`--` comments):

```

-- {9: config.w:55}
local defaults = {
    port = 8080,
    host = "localhost",
}
-- {9:9}

```

HTML (`<!-- -->` comments):

```

<!-- {6: layout.w:20} -->
<nav class="sidebar">
    <!-- {11: navigation.w:7} -->
    <ul>
        <li><a href="/">Home</a></li>
    </ul>
    <!-- {11:11} -->
</nav>
<!-- {6:6} -->

```

Debugging workflow. The following four-step procedure traces any error in tangled output back to the `literate` source.

Step 1: Locate the error in tangled output. You receive an error such as:

```

TypeError: Cannot read property 'id' of undefined
    at Object.<anonymous> (server.js:47)

```

Open `server.js` and go to line 47.

Step 2: Read the nearest opening marker. Scan *upward* from the error line until you find an opening marker:

```

// {3: routes.w:42}

```

This tells you: scrap 3 generated this code, defined in `routes.w` starting at line 42.

Step 3: Go to the `literate` source. Open `routes.w` at line 42. You will find the scrap with the bug, surrounded by prose that explains its intent:

We fetch the user by ID from the database. If the user doesn't exist, we should return a 404, but currently we assume the user always exists --- this is the bug.

```
@d Handle user fetch
@{const user = await getUser(req.params.id);
  res.json(user.id);
@}
```

The prose makes the bug obvious: there is no null check. Fix it in the `.w` file, re-tangle, and the error is gone.

Step 4: Nested markers = nested fragments. When fragments invoke other fragments, markers nest:

```
// {3: routes.w:42}
app.get("/api/users/:id", async (req, res) => {
  // {7: database.w:18}
  const user = await db.query(
    "SELECT * FROM users WHERE id=$1",
    [req.params.id]);
  // {7}
  if (!user) return res.status(404).json({error: "not found"});
  res.json(user);
});
// {3}
```

If the bug is inside the `{7: ... {7}}` block, go to `database.w:18`, not `routes.w:42`. The *innermost enclosing markers* point to the most specific source location. Always look at the tightest pair around the error line.

Interpreting nested hierarchies. Complex tangled output can have several nesting levels. Read them like parentheses—each opening matches its corresponding close:

```
/* {1: main.w:10} */           outermost: main file structure
#include <stdio.h>
/* {2: io.w:25} */             middle: I/O subsystem
void print_report() {
  /* {5: format.w:8} */        innermost: formatting detail
  printf("Report v%d\n", VERSION);
  /* {5} */
}
/* {2} */
int main() {
  /* {3: init.w:14} */
  init_subsystems();
  /* {3} */
  print_report();
  return 0;
}
/* {1} */
```

Scrap numbers are *document-global*—scrap 1 is the first `@{...@}` in the entire woven document, scrap 5 is the fifth. They are stable as long as you do not reorder scraps.

C/C++: #line directives. For C-family languages, weft also generates `#line` directives *above* the opening marker. These make the C compiler itself report errors against the `.w` source file:

```
#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) { ... }
/* {:2} */
```

Now gcc will say:

```
memory.w:14: error: implicit declaration of function 'malloc'
```

instead of:

```
memory.c:47: error: implicit declaration of function 'malloc'
```

The `#line` directive and the section marker are complementary: `#line` makes the compiler report errors against the `.w` file directly; the `{N: file:line}` markers are human-readable when browsing the tangled output.

Common debugging scenarios. Runtime error with stack trace.

```
Error: ENOENT: no such file or directory, open '/data/config.json'
    at readConfig (server.js:123)
```

Open `server.js:123`, find the nearest marker `// {12: config.w:35}`, open `config.w:35`—the scrap that reads the config—and fix the path or add an existence check in the `.w` file.

Compiler error in C. With `#line` directives active, the compiler already reports:

```
parser.w:84: error: 'token_type' undeclared
```

Go directly to `parser.w:84`. The prose around the scrap explains what `token_type` should be and where it is defined.

Wrong output, no error. The program runs but produces wrong results. Open the tangled file and search for the relevant code. The markers tell you which `.w` file and scrap to examine:

```
# {15: transform.w:67}
result = [x * 2 for x in data]    # should be x * 3?
# {:15}
```

Go to `transform.w:67`. The prose says “triple each value” but the code says `* 2`. Bug found.

Identifying which fragment owns a line. Sometimes you know the line of code but not which `@d` definition produced it (multiple fragments with the same name accumulate). The scrap number is unique:

```
// {14: handlers.w:89}
validateInput(data);
// {:14}
// {22: handlers.w:112}
validatePermissions(user);
// {:22}
```

Scraps 14 and 22 are two different definitions (or continuations) of the same fragment. The line numbers point to each one independently.

Tips.

- *Never edit tangled files.* The markers remind you these are generated. Always trace back to the `.w` source and edit there.
- *Scrap numbers are stable* as long as you do not add or remove `@{...@}` blocks before them. If numbering shifts after edits, re-tangle and the markers update automatically.
- *Use `grep` on markers* to find all scraps from a specific file:

```
grep -n '{[0-9]*: database.w:' server.js
```
- *Closing markers as verification.* If you see `{:7}` but no `{7:` above it in your viewport, scroll up—the scrap is larger than your screen. If you see `{7:` but no `{:7}`, the scrap extends below.
- *Markers in test output.* If your test framework shows code snippets on failure, the markers point straight to the literate source.
- *Disable markers selectively.* Use explicit flags on `@o` (e.g. `@o file -t`) with no comment flags to suppress markers for a specific output file when comments would interfere—for example, binary formats, data files, or files where comments are not valid syntax.

1.7 Restrictions

Because weft is intended to be a simple tool, a few restrictions have been established. Over time, some of these may be eliminated; others seem fundamental.

- The handling of errors is not completely ideal. In some cases, weft simply warns of a problem and continues; in other cases it halts immediately. This behavior should be regularized.
- weft warns about references to fragments that haven't been defined, but doesn't halt. The name of the fragment is included in the output file surrounded by `<>` signs. This seems most convenient for development, but may change in the future.
- File names and index entries should not contain any `@` signs.
- Fragment names may be (almost) any well-formed $\text{\TeX}$ string. It makes sense to change fonts or use math mode; however, care should be taken to ensure matching braces, brackets, and dollar signs.
- Anything is allowed in the body of a scrap; however, very long scraps (horizontally or vertically) may not typeset well.
- Temporary files (created for comparison to the eventual output files) are placed in the current directory. Since they may be renamed to an output file name, all the output files should be on the same file system as the current directory. Alternatively, you can use the `-p` flag to specify where the files go.
- Because page numbers cannot be determined until the document has been typeset, weft must be rerun after $\text{\LaTeX}$ to obtain a clean version of the document (very similar to the way it is sometimes necessary to rerun $\text{\LaTeX}$ to obtain an up-to-date table of contents after significant edits). weft will warn (in most cases) when this needs to be done; in the remaining cases, $\text{\LaTeX}$ will warn that labels may have changed. This restriction does not apply to Markdown output (`-w md`), which uses sequential numbering and requires no additional processing passes.

Very long scraps may be allowed to break across a page if declared with `@O` or `@D` (instead of `@o` and `@d`). This doesn't work very well as a default, since far too many short scraps will be broken across pages; however, as a user-controlled option, it seems very useful.

1.8 Project Introspection

The `-m` and `-e` flags support machine-readable introspection of a weft project. They run after pass 1 has built the name trees and do *not* invoke the weave or tangle phases.

1.8.1 Map (`-m`)

The command

```
weft -m project.weft
```

writes a JSON object to stdout with two arrays, `"files"` and `"fragments"`. For each entry the map records the name, language tag, definition scraps (with source file and line), and dependency relationships. File entries list the fragments they use; fragment entries list the scraps that invoke them and the fragments they reference in turn.

This gives an AI assistant or any external tool a complete dependency graph of the project in a single invocation, without having to parse the weft source files.

1.8.2 Extract (`-e`)

The command

```
weft -e "fragment name" project.weft
```

finds the named fragment (using prefix matching), then emits its scrap code and dependency information as Markdown to stdout. Referenced fragments are followed transitively, so the output contains the full closure of the requested fragment.

This allows an AI assistant to retrieve exactly the code it needs to understand or modify a specific fragment, together with all its dependencies, in one step.

1.8.3 Reverse Map (`-R`)

The `-m` and `-e` flags require the `.weft` source files. The reverse map provides the opposite direction: given a tangled output file, it parses the section markers already embedded in that file and reports which `.weft` source and line produced each region. No source files are needed and pass 1 is never invoked.

Single-line query.

```
weft -R server.js:47
```

emits a JSON object identifying the innermost scrap covering line 47:

```
{"scrap": 3, "file": "routes.weft", "line": 42}
```

If the line is outside any scrap region, an empty object `{}` is returned.

Full-file mapping.

```
weft -R server.js
```

emits a JSON object listing every scrap region in the file:

```
{"source": "server.js", "regions": [
  {"start": 2, "end": 15, "scrap": 3,
   "file": "routes.weft", "line": 42},
  {"start": 5, "end": 8, "scrap": 7,
   "file": "database.weft", "line": 18}
]}
```

Regions may overlap when fragments are nested.

This enables CI/CD pipelines to translate compiler errors back to the literate source automatically, and lets AI agents resolve tangled-output locations without loading the `.weft` files at all.

1.9 Acknowledgements

Several people have contributed their time, ideas, and debugging skills. In particular, the contributions of the following people are gratefully acknowledged: Osman Buyukisik, Manuel Carriba, Adrian Clarke, Tim Harvey, Michael Lewis, Walter Ravenek, Rob Shillingsburg, Kayvan Sylvan, Dominique de Waleffe, and Scott Warren. Of course, most of these people would never have heard of nuweb (the original tool) without the efforts of George Greenwade.

Since maintenance has been taken over by Marc Mengel, Simon Wright and Keith Harwood online contributions have been made by:

- Walter Brown <wb@fnal.gov>
- Nicky van Foreest <n.d.vanforeest@math.utwente.nl>
- Javier Goizueta <jgoizueta@jazzfree.com>
- Alan Karp <karp@hp.com>

Chapter 2

The Overall Structure

Processing a web requires three major steps:

1. Read the source, accumulating file names, fragment names, scraps, and lists of cross-references.
2. Reread the source, copying out to the documentation file, with protection and cross-reference information for all the scraps.
3. Traverse the list of files names. For each file name:
 - (a) Dump all the defining scraps into a temporary file.
 - (b) If the file already exists and is unchanged, delete the temporary file; otherwise, rename the temporary file.

2.1 Files

I have divided the program into several files for quicker recompilation during development.

```
"global.h" 21≡  
  < Include files 22 >  
  < Type declarations 23a, ... >  
  < Limits 23b >  
  < Global variable declarations 28, ... >  
  < Function prototypes 42a, ... >  
  < Operating System Dependencies 26b >◇
```

We'll need at least five of the standard system include files. We also require `_POSIX_C_SOURCE` at level 200809L so that POSIX functions like `strdup` are visible when compiling under strict C11 mode (`-std=c11`).

⟨ *Include files 22* ⟩ ≡

```
#define _POSIX_C_SOURCE 200809L

#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <signal.h>
#include <locale.h>
◇
```

Fragment referenced in 21.

Defines: **exit** 26a, 31, 36d, 37b, 41, 47c, 48a, 70b, 117b, 121, 123b, 128, 129, 130a, 131a, 137, 138, 140, 146b, 148, 154, 172, 176, 177a, 178bc, 180, 181, 186a, 193b, 204, **fclose** 59b, 88b, 121, 123ab, 124b, 130b, 158a, **FILE** 59ab, 65a, 69b, 71c, 72, 79, 80b, 82b, 86a, 88ab, 93a, 96, 97a, 102, 103, 104b, 106a, 108b, 110a, 111b, 113, 114, 116, 121, 123a, 124b, 126ab, 132b, 133ab, 134b, 143, 147a, 156b, 158a, 195ab, 196cd, 199ab, 204, **fopen** 59b, 88b, 121, 123b, 124b, 129, 131a, 158a, **fprintf** 34, 37ab, 41, 42b, 45, 47c, 48a, 52a, 58a, 59b, 60b, 63, 65b, 68c, 69a, 70b, 76ab, 77, 79, 80c, 86b, 88b, 92a, 93b, 94c, 95b, 100bc, 101, 103, 104c, 108c, 110c, 112, 113, 114, 116, 117b, 121, 123ab, 124c, 128, 129, 130a, 131a, 133a, 134e, 137, 138, 140, 143, 146b, 148, 150bcd, 153, 154, 156ab, 163b, 170, 171, 172, 173ab, 176, 177a, 178bc, 180, 181, 186a, 193b, 196a, 197b, 199cd, 200ab, **fputs** 52a, 53a, 54d, 58a, 60a, 62a, 63, 64a, 65ab, 66abcd, 67a, 68abcd, 69ab, 71c, 72, 74a, 75a, 76ab, 77, 78, 79, 80ac, 81ab, 83b, 84ab, 85c, 86b, 91a, 92ab, 93ab, 94c, 95ab, 96, 97ab, 99a, 100ab, 101, 102, 103, 104acd, 105a, 106b, 107ab, 108ac, 110bc, 111ac, 112, 113, 114, 115a, 116, 121, 133a, 149, 153, 155a, 195c, 196ab, 197ab, **getc** 124b, 127a, 128, 129, 130ab, 131a, **getenv** 37a, **isgraph** 130a, 172, 180, **islower** 165a, **isspace** 67a, 93c, 140, 172, 173a, 176, 178c, **isupper** Never used, **malloc** 82b, 106a, 186a, 203a, **putc** 53ab, 60c, 61a, 72, 73c, 77, 79, 80c, 81a, 83b, 84ab, 86b, 89ab, 90a, 97ab, 98, 99a, 101, 103, 104c, 106b, 107a, 133a, 149, 151bd, 152a, 154, 155abc, **remove** 124ab, **setlocale** 37a, **size_t** 119, 201c, 202b, 203a, **stderr** 34, 37ab, 41, 42b, 45, 47c, 48a, 59b, 68c, 69a, 70b, 77, 79, 88b, 94c, 95b, 101, 103, 117b, 121, 123ab, 124c, 128, 129, 130a, 131a, 134e, 137, 138, 140, 146b, 148, 154, 163b, 170, 171, 172, 173ab, 176, 177a, 178bc, 180, 181, 186a, 193b, 199d, 200ab, **strlen** 53a, 121, 158b, 162ab, 188a, 198b, **toupper** 165a.

I like to use `TRUE` and `FALSE` in my code. I'd use an `enum` here, except that some systems seem to provide definitions of `TRUE` and `FALSE` by default. The following code seems to work on all the local systems.

```
< Type declarations 23a > ≡
    #ifndef FALSE
    #define FALSE 0
    #endif
    #ifndef TRUE
    #define TRUE 1
    #endif

    #if defined(__GNUC__) && __GNUC__ >= 7
    #define FALLTHROUGH __attribute__((fallthrough))
    #elif defined(__clang__)
    #define FALLTHROUGH __attribute__((fallthrough))
    #else
    #define FALLTHROUGH ((void)0)
    #endif
    ◇
```

Fragment defined by 23a, 54c, 122d, 160ab, 177c, 179b, 182d, 194c, 201a.

Fragment referenced in 21.

Defines: `FALLTHROUGH` 45, 61a, 74a, 90a, 127a, 163a, `FALSE` 28, 29a, 34, 35abc, 36abc, 41, 60c, 61a, 62b, 66c, 67d, 69b, 79, 84a, 85b, 86b, 89ab, 90a, 91b, 94a, 96, 103, 107a, 108c, 110c, 112, 117a, 128, 131a, 154, 158a, 164a, 167, 168, 170, 173a, 174a, 194a, 196a, 197b, 198ab, `TRUE` 28, 29a, 34, 36a, 41, 58a, 61a, 62b, 66ad, 69b, 79, 84a, 85b, 86b, 89ab, 90a, 91b, 93bc, 96, 103, 107a, 108c, 110c, 112, 116, 128, 133b, 134e, 154, 164a, 167, 168, 170, 173a, 194a, 196a, 197b, 198ab.

The `FALLTHROUGH` macro marks intentional switch case fall-throughs as a statement rather than a comment. GCC 7+ and Clang recognise `__attribute__((fallthrough))` and suppress `-Wimplicit-fallthrough` warnings; other compilers see a harmless `((void)0)`. The prose surrounding each switch already explains why the fall-through is intended, so no inline comment is needed.

Here we define the maximum length for names (of fragments etc).

```
< Limits 23b > ≡
    #ifndef MAX_NAME_LEN
    #define MAX_NAME_LEN 1024
    #endif
    ◇
```

Fragment referenced in 21.

Defines: `MAX_NAME_LEN` 140, 143, 146b, 172, 176, 180, 182a, 193b, 198c.

2.1.1 The Main Files

The code is divided into four main files (introduced here) and five support files (introduced in the next section). The file `main.c` will contain the driver for the whole program (see Section 2.2).

```
"main.c" 23c≡
    #include "global.h"
    ◇
```

File defined by 23c, 26a.

The first pass over the source file is contained in `pass1.c`. It handles collection of all the file names, fragments names, and scraps (see Section 2.3).

```
"pass1.c" 24a≡
    #include "global.h"
    ◇
```

File defined by [24a](#), [42b](#).

The `.tex` file is created during a second pass over the source file. The file `latex.c` contains the code controlling the construction of the `.tex` file (see [Section 2.4](#)).

```
"latex.c" 24b≡
    #include "global.h"
    static int scraps = 1;
    ◇
```

File defined by [24b](#), [59ab](#), [65a](#), [69b](#), [70ab](#), [71bc](#), [72](#), [73a](#), [79](#), [80b](#), [82ab](#), [85b](#), [86a](#).
Uses: `scraps` [132a](#).

The Markdown output is created by the code in `markdown.c` (see [Section 2.5](#)).

```
"markdown.c" 24c≡
    #include "global.h"
    static int scraps = 1;
    ◇
```

File defined by [24c](#), [88ab](#), [93a](#), [96](#), [97a](#), [102](#), [103](#), [104b](#), [105b](#), [106a](#), [108b](#), [110a](#), [111b](#).
Uses: `scraps` [132a](#).

The code controlling the creation of the output files is in `output.c` (see [Section 2.7](#)).

```
"output.c" 24d≡
    #include "global.h"
    ◇
```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

2.1.2 Support Files

The support files contain a variety of support routines used to define and manipulate the major data abstractions. The file `input.c` holds all the routines used for referring to source files (see Section 3.1).

```
"input.c" 25a≡
    #include "global.h"
    ◇
```

File defined by 25a, 126ab, 127ac, 131a.

Creation and lookup of scraps is handled by routines in `scraps.c` (see Section 3.2).

```
"scraps.c" 25b≡
    #include "global.h"
    ◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

The handling of file names and fragment names is detailed in `names.c` (see Section 3.3).

```
"names.c" 25c≡
    #include "global.h"
    ◇
```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Memory allocation and deallocation is handled by routines in `arena.c` (see Section 3.6).

```
"arena.c" 25d≡
    #include "global.h"
    ◇
```

File defined by 25d, 201d, 202ab, 203b.

Finally, for best portability, I seem to need a file containing (useless!) definitions of all the global variables.

```
"global.c" 25e≡
    #include "global.h"
    < Operating System Dependencies 26b >
    < Global variable definitions 29a, ... >
    ◇
```

File defined by 25e, 200d.

2.2 The Main Routine

The main routine is quite simple in structure. It wades through the optional command-line arguments, then handles any files listed on the command line.

"main.c" 26a≡

```
#include <stdlib.h>
int main(int argc, char **argv)
{
    int arg = 1;
    ⟨ Interpret command-line arguments 30c, ... ⟩
    ⟨ Set locale information 37a ⟩
    ⟨ Handle reverse map mode 36d ⟩
    initialise_delimit_scrap_array();
    ⟨ Process the remaining arguments (file names) 37b ⟩
    exit(0);
}
◇
```

File defined by 23c, 26a.

Defines: `main` 204.

Uses: `exit` 22.

We only have two major operating system dependencies; the separators for file names, and how to set environment variables. For now we assume the latter can be accomplished via `putenv()` in `stdlib.h`.

⟨ Operating System Dependencies 26b ⟩ ≡

```
#if defined(VMS)
#define PATH_SEP(c) (c==' '||c==':')
#define PATH_SEP_CHAR ""
#define DEFAULT_PATH ""
#elif defined(MSDOS)
#define PATH_SEP(c) (c=='\\')
#define PATH_SEP_CHAR "\\"
#define DEFAULT_PATH "."
#else
#define PATH_SEP(c) (c=='/')
#define PATH_SEP_CHAR "/"
#define DEFAULT_PATH "."
#endif
◇
```

Fragment referenced in 21, 25e.

2.2.1 Command-Line Arguments

There are numerous possible command-line arguments:

- t Suppresses generation of the `.tex` file.
- o Suppresses generation of the output files.
- d list dangling identifier references in indexes.
- c Forces output files to overwrite old files of the same name without comparing for equality first.
- v The verbose flag. Forces output of progress reports.
- n Forces sequential numbering of scraps (instead of page numbers).
- s Doesn't print list of scraps making up file at end of each scrap.

- x Include cross-reference numbers in scrap comments.
- p *path* Prepend *path* to the filenames for all the output files.
- h *options* Provide options for the `hyperref` package.
- r Turn on hyperlinks. You must include the `—usepackage—` options in the text.
- l Use the `listings` package for formatting scraps.

There are two ways to get hyper-links into your documentation. With one, the `-r` flag simply arranges for fragment cross-references to be hyperlinks. You have to provide all the rest of the `hyperref` material yourself. The `-h` flag does more work, but requires you provide the `hyperref` options on the command line. You would use the first if you know you will use the same tools all the time. You would use the second if you have to distribute the weft file and use the build system to provide the `hyperref` options.

The `-h` option is used as follows. If you want hyperlinks in your document give a string to this option which is a valid option for the `hyperref` package. For example, `"dvips,colorlinks=true"` sets it to use the `dvips` driver and mark the links in colour. For more information about these options, see the `hyperref` documentation.

Then in your document you put the command `\NWuseHyperlinks` immediately after your `usepackage` commands. If you do both of these you will get hyperlinks. If you miss out either then nothing interesting happens.

Global flags are declared for each of the arguments.

(*Global variable declarations* 28) ≡

```
extern int tex_flag;      /* if FALSE, don't emit the documentation file */
extern int output_flag;   /* if FALSE, don't emit the output files */
extern int compare_flag;  /* if FALSE, overwrite without comparison */
extern int verbose_flag;  /* if TRUE, write progress information */
extern int number_flag;   /* if TRUE, use a sequential numbering scheme */
extern int scrap_flag;    /* if FALSE, don't print list of scraps */
extern int dangling_flag; /* if FALSE, don't print dangling identifiers */
extern int xref_flag;     /* If TRUE, print cross-references in scrap comments */
extern int prepend_flag;  /* If TRUE, prepend a path to the output file names */
extern char * dirpath;    /* The prepended directory path */
extern char * path_sep;   /* How to join path to filename */
extern int listings_flag; /* if TRUE, use listings package for scrap formatting */
extern int version_info_flag; /* If TRUE, set up version string */
extern char * version_string; /* What to print for @v */
extern int hyperref_flag; /* Are we preparing for hyperref
                           package. */
extern int hyperopt_flag; /* Are we preparing for hyperref options */
extern char * hyperoptions; /* The options to pass to the
                             hyperref package */
extern int includepath_flag; /* Do we have an include path? */
extern struct incl{char * name; struct incl * next;} * include_list;
                           /* The list of include paths */
extern int markdown_flag; /* if TRUE, generate Markdown instead of LaTeX */
extern int weave_flag;    /* if TRUE, weave was requested via -w */
extern int weave_format;  /* 0=none, 1=tex, 2=md */
extern int weave_format_from_cli; /* if TRUE, CLI override of @W */
extern int map_flag;      /* if TRUE, emit JSON map to stdout */
extern int extract_flag;  /* if TRUE, next arg is extract name */
extern char *extract_name; /* fragment name to extract */
extern int reverse_map_flag; /* if TRUE, reverse map mode */
extern char *reverse_map_arg; /* file[:line] argument for -R */
◇
```

Fragment defined by 28, 29b, 30a, 46c, 51c, 67b, 125b, 135a, 161a, 201b.

Fragment referenced in 21.

Defines: compare_flag 29a, 34, 124a, dangling_flag 29a, 34, 86b, 108c, extract_flag 29a, 34, 36b, extract_name 29a, 36b, 41, hyperoptions 29a, 36a, 60ab, hyperopt_flag 29a, 34, 36a, hyperref_flag 29a, 34, 36a, 60a, incl 29a, 35b, 40, 129, includepath_flag 29a, 34, 35b, map_flag 29a, 34, 41, markdown_flag 29a, 34, 41, number_flag 29a, 34, 41, 158a, output_flag 29a, 34, 41, reverse_map_arg 29a, 36cd, reverse_map_flag 29a, 34, 36c, scrap_flag 29a, 34, 63, 94b, tex_flag 29a, 34, 41, 42b, 154, verbose_flag 29a, 34, 42b, 59b, 88b, 123a, version_info_flag 29a, 34, 35c, version_string 29a, 35c, 62a, 75a, 91a, weave_flag 29a, 34, 41, weave_format 29a, 34, 41, 171, weave_format_from_cli 29a, 34, 41, 171, xref_flag 29a, 34, 155ac.

Uses: FALSE 23a, next 131b, reverse 183d, scraps 132a, TRUE 23a.

The flags are all initialized for correct default behavior.

< Global variable definitions 29a > ≡

```

int tex_flag = TRUE;
int output_flag = TRUE;
int compare_flag = TRUE;
int verbose_flag = FALSE;
int number_flag = FALSE;
int scrap_flag = TRUE;
int dangling_flag = FALSE;
int xref_flag = FALSE;
int prepend_flag = FALSE;
char * dirpath = DEFAULT_PATH; /* Default directory path */
char * path_sep = PATH_SEP_CHAR;
int listings_flag = FALSE;
int version_info_flag = FALSE;
char default_version_string[] = "no version";
char * version_string = default_version_string;
int hyperref_flag = FALSE;
int hyperopt_flag = FALSE;
char * hyperoptions = "";
int includepath_flag = FALSE; /* Do we have an include path? */
struct incl * include_list = NULL;
/* The list of include paths */

int markdown_flag = FALSE;
int weave_flag = FALSE;
int weave_format = 0;
int weave_format_from_cli = FALSE;
int map_flag = FALSE;
int extract_flag = FALSE;
char *extract_name = NULL;
int reverse_map_flag = FALSE;
char *reverse_map_arg = NULL;

```

Fragment defined by 29ac, 30b, 46d, 51d, 67c, 125c, 135b, 161b.

Fragment referenced in 25e.

Uses: compare_flag 28, dangling_flag 28, extract_flag 28, extract_name 28, FALSE 23a, hyperoptions 28, hyperopt_flag 28, hyperref_flag 28, incl 28, includepath_flag 28, map_flag 28, markdown_flag 28, number_flag 28, output_flag 28, reverse_map_arg 28, reverse_map_flag 28, scrap_flag 28, tex_flag 28, TRUE 23a, verbose_flag 28, version_info_flag 28, version_string 28, weave_flag 28, weave_format 28, weave_format_from_cli 28, xref_flag 28.

A global variable `nw_char` will be used for the weft meta-character, which by default will be `@`.

< Global variable declarations 29b > ≡

```

extern int nw_char;

```

Fragment defined by 28, 29b, 30a, 46c, 51c, 67b, 125b, 135a, 161a, 201b.

Fragment referenced in 21.

Defines: `nw_char` 29c, 41, 42c, 45, 47c, 48a, 49b, 51b, 52b, 53d, 56, 57, 58a, 60c, 61a, 63, 70a, 71c, 72, 73a, 74a, 75d, 76a, 80c, 86b, 89a, 90a, 97a, 99a, 100b, 127a, 128, 131a, 137, 138, 139a, 140, 141cd, 143, 146b, 149, 151c, 152a, 153, 154, 156ab, 157a, 163b, 170, 171, 172, 176, 177ab, 178c, 180, 181, 182a, 193ab, 196a, 197b, 198ac.

Global variable definitions 29c $\equiv$

```
int nw_char='@';
```

◇

Fragment defined by 29ac, 30b, 46d, 51d, 67c, 125c, 135b, 161b.

Fragment referenced in 25e.

Defines: `nw_char` 29b, 41, 42c, 45, 47c, 48a, 49b, 51b, 52b, 53d, 56, 57, 58a, 60c, 61a, 63, 70a, 71c, 72, 73a, 74a, 75d, 76a, 80c, 86b, 89a, 90a, 97a, 99a, 100b, 127a, 128, 131a, 137, 138, 139a, 140, 141cd, 143, 146b, 149, 151c, 152a, 153, 154, 156ab, 157a, 163b, 170, 171, 172, 176, 177ab, 178c, 180, 181, 182a, 193ab, 196a, 197b, 198ac.

We save the invocation name of the command in a global variable `command_name` for use in error messages.

Global variable declarations 30a $\equiv$

```
extern char *command_name;
```

◇

Fragment defined by 28, 29b, 30a, 46c, 51c, 67b, 125b, 135a, 161a, 201b.

Fragment referenced in 21.

Defines: `command_name` 30bc, 32, 34, 37b, 41, 45, 47c, 48a, 59b, 69a, 77, 79, 88b, 95b, 101, 103, 117b, 123b, 124c, 128, 129, 130a, 131a, 134e, 137, 138, 140, 146b, 148, 154, 163b, 170, 171, 172, 173ab, 176, 177a, 178bc, 180, 181, 193b.

Global variable definitions 30b $\equiv$

```
char *command_name = NULL;
```

◇

Fragment defined by 29ac, 30b, 46d, 51d, 67c, 125c, 135b, 161b.

Fragment referenced in 25e.

Uses: `command_name` 30a.

The invocation name is conventionally passed in `argv[0]`.

Interpret command-line arguments 30c $\equiv$

```
command_name = argv[0];
```

◇

Fragment defined by 30cd.

Fragment referenced in 26a.

Uses: `command_name` 30a.

We need to examine the remaining entries in `argv`, looking for command-line arguments.

Interpret command-line arguments 30d $\equiv$

```
while (arg < argc) {
    char *s = argv[arg];
    if (*s++ == '-') {
        Check for long options 31
        Interpret the argument string s 34
        arg++;
        Perhaps get the prepend path 35a
        Perhaps get the version info string 35c
        Perhaps get the hyperref options 36a
        Perhaps add an include path 35b
        Perhaps get the extract name 36b
        Perhaps get the reverse map argument 36c
    }
    else break;
}
```

◇

Fragment defined by 30cd.

Fragment referenced in 26a.

2.2.2 Help Message

When the user passes `--help`, we display a comprehensive reference that serves both human users and AI assistants. The help text is structured in sections—usage, options grouped by purpose, examples, and an AI workflow summary—so that a language model can parse it and use `weft` correctly without additional context.

We intercept long options before the character-by-character flag parsing. Currently `--help` is the only long option.

```
< Check for long options 31> ≡  
    if (*s == '-') {  
        if (strcmp(s + 1, "help") == 0) {  
            < Print help message 32>  
            exit(0);  
        }  
    }  
}
```

Fragment referenced in 30d.

Uses: `exit` 22.

The help message itself. We write to `stdout` (not `stderr`) because `--help` is a deliberate request, not an error condition. Options are grouped by function: tangle control, weave control, analysis tools, L^AT_EX-specific, and general. The AI WORKFLOW section at the end documents the map-extract-edit-tangle-debug loop that makes `weft` particularly suited to AI-assisted development.

< Print help message 32 > ≡

```
printf("weft -- A Modern Literate Programming Tool\n\n");
printf("USAGE\n");
printf("  %s [options] file...\n\n", command_name);
printf("WHAT IS WEFT?\n");
printf("  weft is a literate programming tool.  You write .weft files where\n");
printf("  documentation and code live together as a single narrative.  weft\n");
printf("  then TANGLES the .weft source into executable output files, and\n");
printf("  optionally WEAVES it into formatted documentation (LaTeX or Markdown).\n\n");
printf("  The .weft files are the ONLY source of truth.  The tangled output\n");
printf("  files (*.c, *.go, *.js, etc.) are GENERATED -- never edit them.\n");
printf("  All changes must go through the .weft source.\n\n");
printf("  Default behavior: tangle only (generate output files from @o directives).\n\n");
printf("HOW IT WORKS\n");
printf("  A .weft file contains prose with embedded code chunks:\n");
printf("    @o file.c @{ ... @}      Output file: tangle writes this to file.c\n");
printf("    @d Name  @{ ... @}      Fragment: reusable code block, invoked by name\n");
printf("    @<Name@>                Invocation: inserts the named fragment here\n");
printf("    @@                        Literal @: escapes the meta-character\n");
printf("    @i other.weft            Include: pulls in another .weft file\n");
printf("  Fragments compose hierarchically.  An @o chunk may invoke @d fragments,\n");
printf("  which invoke other fragments, forming a tree that weft flattens into\n");
printf("  the final output file.\n\n");
printf("OPTIONS\n");
printf("  Tangle control:\n");
printf("    -o          Suppress output file generation (parse only)\n");
printf("    -c          Overwrite files without comparing first\n");
printf("    -p path     Prepend path to all output file names\n");
printf("    -I path     Add directory to include search path (repeatable)\n");
printf("    -x          Include cross-reference numbers in section markers\n");
printf("  Weave control:\n");
printf("    -w [format] Weave documentation (format: md or tex)\n");
printf("                  Without format, uses @W directive from source\n");
printf("    -t          Suppress weave output (even if -w given)\n");
printf("    -n          Sequential scrap numbering (auto for Markdown)\n");
printf("    -s          Omit scrap composition lists\n\n");
printf("  Analysis (AI-friendly tools):\n");
printf("    -m          Emit JSON structure map to stdout (all fragments,\n");
printf("                  files, definitions, and dependency graph)\n");
printf("    -e name     Extract named fragment and all its transitive\n");
printf("                  dependencies as self-contained Markdown to stdout\n");
printf("    -R file[:line] Reverse map: given a tangled output file (and\n");
printf("                  optional line number), prints the .weft source\n");
printf("                  location that generated it.  Needs NO .weft files\n");
printf("                  -- reads section markers embedded in the output\n");
printf("  LaTeX:\n");
printf("    -r          Enable hyperref cross-reference links\n");
printf("    -h options  Pass options to hyperref package\n");
printf("    -l          Use listings package for scrap formatting\n");
printf("    -d          List dangling identifier references\n");
printf("  General:\n");
printf("    -V string   Set version string (expands @v in source)\n");
printf("    -v          Verbose progress output\n");
printf("    --help     Show this help message\n");
printf("EXAMPLES\n");
printf("  %s project.weft          Tangle (default)\n", command_name);
printf("  %s -w md project.weft    Weave to Markdown\n", command_name);
printf("  %s -w tex project.weft    Weave to LaTeX\n", command_name);
printf("  %s -m project.weft | jq '.files'  JSON structure map\n", command_name);
printf("  %s -e \"Validate\" project.weft      Extract fragment\n", command_name);
printf("  %s -R server.js:47          Reverse map line\n", command_name);
printf("  %s -R server.js            All regions in file\n", command_name);
printf("  %s -v -p build/ project.weft Verbose + path prefix\n", command_name);
printf("  %s -o project.weft         Parse only, no output\n", command_name);
printf("RULES FOR AI ASSISTANTS\n");
printf("  1. NEVER edit tangled output files (*.c, *.go, *.js, etc.)\n");
printf("     They are generated.  Your changes will be overwritten.\n");
printf("  2. ALWAYS edit .weft source files instead.\n");
printf("     Use -R to find which .weft file generates a given output line.\n");
printf("  3. After editing .weft files, re-tangle\n");
```

2.2.3 Short Flags

Several flags can be stacked behind a single minus sign; therefore, we've got to loop through the string, handling them all. If this flag requires an argument we skip to getting its argument straight away. This allows arguments to butt up to their flags and also avoids ambiguity about which value goes with which flag.

⟨ Interpret the argument string s 34 ⟩ ≡

```

{
    char c = *s++;
    while (c) {
        switch (c) {
            case 'c': compare_flag = FALSE;
                       break;
            case 'd': dangling_flag = TRUE;
                       break;
            case 'h': hyperopt_flag = TRUE;
                       goto HasValue;
            case 'I': includepath_flag = TRUE;
                       goto HasValue;
            case 'l': listings_flag = TRUE;
                       break;
            case 'n': number_flag = TRUE;
                       break;
            case 'o': output_flag = FALSE;
                       break;
            case 'p': prepend_flag = TRUE;
                       goto HasValue;
            case 'r': hyperref_flag = TRUE;
                       break;
            case 's': scrap_flag = FALSE;
                       break;
            case 't': tex_flag = FALSE;
                       break;
            case 'e': extract_flag = TRUE;
                       goto HasValue;
            case 'R': reverse_map_flag = TRUE;
                       goto HasValue;
            case 'm': map_flag = TRUE;
                       break;
            case 'v': verbose_flag = TRUE;
                       break;
            case 'V': version_info_flag = TRUE;
                       goto HasValue;
            case 'w': goto HasWeaveFormat;
            case 'x': xref_flag = TRUE;
                       break;
            default: fprintf(stderr, "%s: unexpected argument ignored. "
                             "Try '%s --help' for usage information.\n",
                             command_name, command_name);
                     break;
        }
        c = *s++;
    }
    goto HasValue;
HasWeaveFormat:
{
    char *fmt = NULL;
    weave_flag = TRUE;
    if (*s != '\0')
        fmt = s;
    else if (arg + 1 < argc && argv[arg + 1][0] != '-') {
        /* Peek: only consume next arg if it looks like a format token */
        char *candidate = argv[arg + 1];
        if (strcmp(candidate, "md") == 0 || strcmp(candidate, "tex") == 0) {
            fmt = candidate;
            ++arg;
        }
    }
    if (fmt != NULL) {
        if (strcmp(fmt, "md") == 0) {
            weave_format = 2;
            markdown_flag = TRUE;
            weave_format_from_cli = TRUE;
        } else if (strcmp(fmt, "tex") == 0) {
            weave_format = 1;
            weave_format_from_cli = TRUE;
        }
    }
}

```

⟨ *Perhaps get the prepend path 35a* ⟩ ≡

```
if (prepend_flag)
{
    if (*s == '\\0')
        s = argv[arg++];
    dirpath = s;
    prepend_flag = FALSE;
}
◇
```

Fragment referenced in [30d](#).

Uses: [FALSE 23a](#).

⟨ *Perhaps add an include path 35b* ⟩ ≡

```
if (includepath_flag)
{
    struct incl * le
        = (struct incl *)arena_getmem(sizeof(struct incl));
    struct incl ** p = &include_list;

    if (*s == '\\0')
        s = argv[arg++];
    le->name = save_string(s);
    le->next = NULL;
    while (*p != NULL)
        p = &((*p)->next);
    *p = le;
    includepath_flag = FALSE;
}
◇
```

Fragment referenced in [30d](#).

Uses: [arena_getmem 202b](#), [FALSE 23a](#), [incl 28](#), [includepath_flag 28](#), [next 131b](#), [save_string 162b](#).

⟨ *Perhaps get the version info string 35c* ⟩ ≡

```
if (version_info_flag)
{
    if (*s == '\\0')
        s = argv[arg++];
    version_string = s;
    version_info_flag = FALSE;
}
◇
```

Fragment referenced in [30d](#).

Uses: [FALSE 23a](#), [version_info_flag 28](#), [version_string 28](#).

```

⟨ Perhaps get the hyperref options 36a ⟩ ≡
    if (hyperopt_flag)
    {
        if (*s == '\0')
            s = argv[arg++];
        hyperoptions = s;
        hyperopt_flag = FALSE;
        hyperref_flag = TRUE;
    }
    ◇

```

Fragment referenced in 30d.

Uses: FALSE 23a, hyperoptions 28, hyperopt_flag 28, hyperref_flag 28, TRUE 23a.

```

⟨ Perhaps get the extract name 36b ⟩ ≡
    if (extract_flag) {
        extract_name = (*s != '\0') ? s : argv[arg++];
        extract_flag = FALSE;
    }
    ◇

```

Fragment referenced in 30d.

Uses: extract_flag 28, extract_name 28, FALSE 23a.

```

⟨ Perhaps get the reverse map argument 36c ⟩ ≡
    if (reverse_map_flag) {
        reverse_map_arg = (*s != '\0') ? s : argv[arg++];
        reverse_map_flag = FALSE;
    }
    ◇

```

Fragment referenced in 30d.

Uses: FALSE 23a, reverse_map_arg 28, reverse_map_flag 28.

If the user gave `-R file[:line]`, we handle it immediately and exit. No `.weft` source files are required—only the tangled output file.

```

⟨ Handle reverse map mode 36d ⟩ ≡
    if (reverse_map_arg) {
        write_reverse_map(reverse_map_arg);
        exit(0);
    }
    ◇

```

Fragment referenced in 26a.

Uses: exit 22, reverse_map_arg 28, write_reverse_map 121.

In order to be able to process files in foreign languages we set the locale information. `isgraph()` and friends need this (see Section 3.3). Try to read `LC_CTYPE` from the environment. Use `LC_ALL` if that fails. Don't set the locale if reading `LC_ALL` fails, too. Print a warning if setting the program's locale fails.

⟨ Set locale information 37a ⟩ ≡

```
{
    /* try to get locale information */
    char *s=getenv("LC_CTYPE");
    if (s==NULL) s=getenv("LC_ALL");

    /* set it */
    if (s!=NULL)
        if(setlocale(LC_CTYPE, s)==NULL)
            fprintf(stderr, "Setting locale failed\n");
}
◇
```

Fragment referenced in 26a.

Uses: fprintf 22, getenv 22, setlocale 22, stderr 22.

2.2.4 File Names

We expect at least one file name. While a missing file name might be ignored without causing any problems, we take the opportunity to report the usage convention.

⟨ Process the remaining arguments (file names) 37b ⟩ ≡

```
{
    if (arg >= argc) {
        fprintf(stderr, "%s: expected a file name.\n"
            "Usage: %s [options] file...\n"
            "Try '%s --help' for more information.\n",
            command_name, command_name, command_name);
        exit(-1);
    }
    do {
        ⟨ Handle the file name in argv[arg] 38 ⟩
        arg++;
    } while (arg < argc);
}
◇
```

Fragment referenced in 26a.

Uses: command_name 30a, exit 22, fprintf 22, stderr 22.

The code to handle a particular file name is rather more tedious than the actual processing of the file. A file name may be an arbitrarily complicated path name, with an optional extension. If no extension is present, we add `.w` as a default. The extended path name will be kept in a local variable `source_name`. The resulting documentation file will be written in the current directory; its name will be kept in the variable `tex_name`.

```

⟨ Handle the file name in argv[arg] 38 ⟩ ≡
{
    char source_name[FILENAME_MAX];
    char tex_name[FILENAME_MAX];
    char aux_name[FILENAME_MAX];
    char *trim;
    char *dot;
    ⟨ Build source_name and tex_name 39 ⟩
    ⟨ Process a file 41 ⟩
}◊

```

Fragment referenced in 37b.

Uses: `source_name` 125b.

I bump the pointer `p` through all the characters in `argv[arg]`, copying all the characters into `source_name` (via the pointer `q`).

At each slash, I update `trim` to point just past the slash in `source_name`. The effect is that `trim` will point at the file name without any leading directory specifications.

The pointer `dot` is made to point at the file name extension, if present. If there is no extension, we add `.weft` to the source name. The `tex_name` (weave output name) is computed later in “Process a file” once the weave format is known.

```

< Build source_name and tex_name 39 > ≡
{
    char *p = argv[arg];
    char *q = source_name;
    char c;
    trim = q;
    dot = NULL;
    c = *p++;
    while (c) {
        *q++ = c;
        if (PATH_SEP(c)) {
            trim = q;
            dot = NULL;
        }
        else if (c == '.')
            dot = q - 1;
        c = *p++;
    }
    < Add the source path to the include path list 40 >
    *q = '\0';
    if (!dot) {
        *q++ = '.';
        *q++ = 'w';
        *q++ = 'e';
        *q++ = 'f';
        *q++ = 't';
        *q = '\0';
    }
    /* Find trim again for tex_name computation later */
    {
        char *t = source_name;
        trim = t;
        dot = NULL;
        while (*t) {
            if (PATH_SEP(*t)) { trim = t + 1; dot = NULL; }
            else if (*t == '.') dot = t;
            t++;
        }
    }
}
}◊

```

Fragment referenced in [38](#).

Uses: `source_name` [125b](#).

If the source file has a directory part we add that directory to the end of the search path so that the path of last resort is the same as the source path, not, as one person put it, something irrelevant like the directory weft was invoked from.

```

⟨ Add the source path to the include path list 40 ⟩ ≡
    if (trim != source_name) {
        struct incl * le
            = (struct incl *)arena_getmem(sizeof(struct incl));
        struct incl ** p = &include_list;
        char sv = *trim;

        *trim = '\0';
        le->name = save_string(source_name);
        le->next = NULL;
        while (*p != NULL)
            p = &((*p)->next);
        *p = le;
        *trim = sv;
    }
    ◇

```

Fragment referenced in [39](#).

Uses: [arena_getmem 202b](#), [incl 28](#), [next 131b](#), [save_string 162b](#), [source_name 125b](#).

Now that we're finally ready to process a file, it's not really too complex. We bundle most of the work into three routines `pass1` (see [Section 2.3](#)), `write_tex` (see [Section 2.4](#)), and `write_files` (see [Section 2.7](#)). After we're finished with a particular file, we must remember to release its storage (see [Section 3.6](#)).

The default behavior is to tangle only. Weaving happens only when `-w` is given. The weave format is determined by (in priority order): a CLI argument (`-w md` or `-w tex`), or the `@W` directive in the source file. If `-w` is given without a format and no `@W` is present, weft reports an error and exits. The `-t` flag suppresses weaving even if `-w` was given.

⟨ *Process a file* 41 ⟩ ≡

```
{
    int do_weave = FALSE;
    int effective_format = 0;

    /* Reset weave_format for this file unless CLI locked it */
    if (!weave_format_from_cli)
        weave_format = 0;

    pass1(source_name);
    current_sector = 1;
    prev_sector = 1;

    if (map_flag) {
        write_map(file_names, macro_names);
        /* skip weave and tangle */
    } else if (extract_name) {
        write_extract(extract_name, file_names, macro_names);
        /* skip weave and tangle */
    } else {
        /* Determine whether to weave and with which format */
        if (weave_flag && !tex_flag) {
            /* -t suppresses weave: tex_flag==FALSE means user passed -t */
        } else if (weave_flag) {
            effective_format = weave_format;
            if (effective_format == 0) {
                fprintf(stderr, "%s: -w requires a weave format.\n"
                    "    Use -w md, -w tex, or add %cW md/%cW tex to the source file.\n",
                    command_name, nw_char, nw_char);
                exit(-1);
            }
            do_weave = TRUE;
        }

        if (do_weave) {
            /* Compute weave output name now that format is known */
            const char *weave_ext = (effective_format == 2) ? ".md" : ".tex";
            if (dot) {
                *dot = '\0';
                sprintf(tex_name, "%s%s%s", dirpath, path_sep, trim, weave_ext);
                sprintf(aux_name, "%s%s%s.aux", dirpath, path_sep, trim);
                *dot = '.';
            } else {
                sprintf(tex_name, "%s%s%s", dirpath, path_sep, trim, weave_ext);
                sprintf(aux_name, "%s%s%s.aux", dirpath, path_sep, trim);
            }

            if (effective_format == 2) {
                markdown_flag = TRUE;
                number_flag = TRUE;
                collect_numbers(aux_name);
                write_md(source_name, tex_name, 0);
            } else {
                collect_numbers(aux_name);
                write_tex(source_name, tex_name, 0);
            }
        }

        if (output_flag)
            write_files(file_names);
    }
    arena_free();
}
```

41

Fragment referenced in 38.

Uses: [arena_free](#) 203b, [collect_numbers](#) 158a, [command_name](#) 30a, [current_sector](#) 46c, [exit](#) 22, [extract_name](#) 28, [FALSE](#) 23a, [file_names](#) 161a, [fprintf](#) 22, [macro_names](#) 161a, [map_flag](#) 28, [markdown_flag](#) 28, [number_flag](#) 28, [nw_char](#) 29bc, [output_flag](#) 28, [pass1](#) 42b, [prev_sector](#) 46c, [source_name](#) 125b, [stderr](#) 22, [tex_flag](#) 28, [TRUE](#) 23a, [weave_flag](#) 28, [weave_format](#) 28, [weave_format_from_cli](#) 28, [write_extract](#) 117b, [write_files](#) 122c, [write_map](#) 115a, [write_md](#) 88b, [write_tex](#) 59b.

2.3 Pass One

During the first pass, we scan the file, recording the definitions of each fragment and file and accumulating all the scraps.

```
< Function prototypes 42a > ≡  
    extern void pass1(char *file_name);  
    ◇
```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: file_name 131c, pass1 42b.

The routine `pass1` takes a single argument, the name of the source file. It opens the file, then initializes the scrap structures (see Section 3.2) and the roots of the file-name tree, the fragment-name tree, and the tree of user-specified index entries (see Section 3.3). After completing all the necessary preparation, we make a pass over the file, filling in all our data structures. Next, we search all the scraps for references to the user-specified index entries. Finally, we must reverse all the cross-reference lists accumulated while scanning the scraps.

```
"pass1.c" 42b ≡  
    void pass1(char *file_name)  
    {  
        if (verbose_flag)  
            fprintf(stderr, "reading %s\n", file_name);  
        source_open(file_name);  
        init_scraps();  
        macro_names = NULL;  
        file_names = NULL;  
        user_names = NULL;  
        < Scan the source file, looking for at-sequences 42c >  
        if (tex_flag)  
            search();  
        < Reverse cross-reference lists 54b >  
    }  
    ◇
```

File defined by 24a, 42b.

Defines: `pass1` 41, 42a, 74a, 99a.

Uses: file_name 131c, file_names 161a, fprintf 22, init_scraps 132c, macro_names 161a, search 189a, source_open 131a, stderr 22, tex_flag 28, user_names 161a, verbose_flag 28.

The only thing we look for in the first pass are the command sequences. All ordinary text is skipped entirely.

```
< Scan the source file, looking for at-sequences 42c > ≡  
    {  
        int c = source_get();  
        while (c != EOF) {  
            if (c == nw_char)  
                < Scan at-sequence 45 >  
            c = source_get();  
        }  
    }◇
```

Fragment referenced in 42b.

Uses: nw_char 29bc, source_get 127a.

The `@l` command (lowercase) sets the language for the next `@o` or `@d` directive by storing the tag in `pending_lang`. It replaces the earlier mechanism of detecting Markdown fences (‘‘`lang`’) and L^AT_EX markers (‘‘`lang`’) during pass1. A typical usage is:

```
@l python
@d Validate input
@{import re
...
@}
```

```
< Collect language apply in pass1 43a > ≡
{
    char lang[64];
    char *lp = lang;
    int lc = source_get();
    /* skip optional whitespace */
    while (lc == ' ' || lc == '\t')
        lc = source_get();
    /* read language tag */
    while (lc != EOF && lc != '\n' && lc != ' ' && lc != '\t'
           && (lp - lang) < 62) {
        *lp++ = lc;
        lc = source_get();
    }
    *lp = '\0';
    if (lang[0] != '\0') {
        strncpy(pending_lang, lang, 63);
        pending_lang[63] = '\0';
    }
    /* consume rest of line */
    while (lc != EOF && lc != '\n')
        lc = source_get();
}◇
```

Fragment referenced in 45.

Uses: `pending_lang` 161a, `source_get` 127a.

The `@L` command (Section 3.3.2) lets users register custom language mappings. During pass1, the actual parsing and registration is handled by `collect_lang_def()` in `names.c`. During the weave passes (L^AT_EX and Markdown), we simply consume the rest of the line so that the arguments do not leak into the typeset output.

```
< Collect user language definition in pass1 43b > ≡
    collect_lang_def();◇
```

Fragment referenced in 45.

Uses: `collect_lang_def` 170.

```
< Collect weave format declaration in pass1 43c > ≡
    collect_weave_format();◇
```

Fragment referenced in 45.

Uses: `collect_weave_format` 171.

```

⟨ Skip at-L language definition line 44 ⟩ ≡
{ int lc = source_get();
  while (lc != '\n' && lc != EOF)
    lc = source_get();
}
c = source_get();◇

```

Fragment referenced in [61a](#), [90a](#).

Uses: [source_get 127a](#).

Six at-sequences are interesting during the first pass: [@o](#) and [@O](#) (file definitions), [@d](#), [@D](#), and [@q](#) (fragment definitions), [@c](#) (block comments), [@s](#)/[@S](#) (sectors), [@l](#) (language apply), and [@L](#) (language definitions). We skip past all others immediately, warning if unexpected sequences are discovered.

⟨ Scan at-sequence 45 ⟩ ≡

```
{
    char quoted = 0;

    c = source_get();
    switch (c) {
        case 'r':
            c = source_get();
            nw_char = c;
            update_delimit_scrap();
            break;
        case '0':
        case 'o': ⟨ Build output file definition 46e ⟩
            break;
        case 'Q':
        case 'q': quoted = 1;
            FALLTHROUGH;
        case 'D':
        case 'd': ⟨ Build fragment definition 47a ⟩
            break;
        case 's':
            ⟨ Step to next sector 46a ⟩
            break;
        case 'S':
            ⟨ Close the current sector 46b ⟩
            break;
        case '<':
        case '(':
        case '[':
        case '{': ⟨ Skip over an in-text scrap 47c ⟩
            break;
        case 'c': ⟨ Collect a block comment 48b ⟩
            break;
        case 'l': ⟨ Collect language apply in pass1 43a ⟩
            break;
        case 'L': ⟨ Collect user language definition in pass1 43b ⟩
            break;
        case 'W': ⟨ Collect weave format declaration in pass1 43c ⟩
            break;
        case 'x':
        case 'v':
        case 'u':
        case 'm':
        case 'f': /* ignore during this pass */
            break;
        default: if (c==nw_char) /* ignore during this pass */
            break;
            fprintf(stderr,
                "%s: unexpected %c sequence ignored (%s, line %d)\n",
                command_name, nw_char, source_name, source_line);
            break;
    }
}◊
```

Fragment referenced in 42c.

Uses: command_name 30a, FALLTHROUGH 23a, fprintf 22, nw_char 29bc, source_get 127a, source_line 125b, source_name 125b, stderr 22, update_delimit_scrap 73a.

⟨ Step to next sector 46a ⟩ ≡

```
prev_sector += 1;
current_sector = prev_sector;
c = source_get();
◇
```

Fragment referenced in [45](#), [61a](#), [90a](#).

Uses: `current_sector` [46c](#), `prev_sector` [46c](#), `source_get` [127a](#).

⟨ Close the current sector 46b ⟩ ≡

```
current_sector = 1;
c = source_get();
◇
```

Fragment referenced in [45](#), [61a](#), [90a](#).

Uses: `current_sector` [46c](#), `source_get` [127a](#).

⟨ Global variable declarations 46c ⟩ ≡

```
extern unsigned char current_sector;
extern unsigned char prev_sector;
◇
```

Fragment defined by [28](#), [29b](#), [30a](#), [46c](#), [51c](#), [67b](#), [125b](#), [135a](#), [161a](#), [201b](#).

Fragment referenced in [21](#).

Defines: `current_sector` [41](#), [46abd](#), [81b](#), [85c](#), [105a](#), [108a](#), [136c](#), [140](#), [176](#), [180](#), `prev_sector` [41](#), [46ad](#).

⟨ Global variable definitions 46d ⟩ ≡

```
unsigned char current_sector = 1;
unsigned char prev_sector = 1;
◇
```

Fragment defined by [29ac](#), [30b](#), [46d](#), [51d](#), [67c](#), [125c](#), [135b](#), [161b](#).

Fragment referenced in [25e](#).

Uses: `current_sector` [46c](#), `prev_sector` [46c](#).

2.3.1 Accumulating Definitions

There are three steps required to handle a definition:

1. Build an entry for the name so we can look it up later.
2. Collect the scrap and save it in the table of scraps.
3. Attach the scrap to the name.

We go through the same steps for both file names and fragment names.

⟨ Build output file definition 46e ⟩ ≡

```
{
    Name *name = collect_file_name(); /* returns a pointer to the name entry */
    int scrap = collect_scrap();      /* returns an index to the scrap */
    ⟨ Add scrap to name's definition list 47b ⟩
}◇
```

Fragment referenced in [45](#).

Uses: `collect_file_name` [172](#), `collect_scrap` [136b](#), `Name` [160b](#).

⟨ *Build fragment definition 47a* ⟩ ≡

```
{
    Name *name = collect_macro_name();
    int scrap = collect_scrap();
    ⟨ Add scrap to name's definition list 47b ⟩
}◇
```

Fragment referenced in [45](#).

Uses: [collect_macro_name 176](#), [collect_scrap 136b](#), [Name 160b](#).

Since a file or fragment may be defined by many scraps, we maintain them in a simple linked list. The list is actually built in reverse order, with each new definition being added to the head of the list.

⟨ *Add scrap to name's definition list 47b* ⟩ ≡

```
{
    Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    def->scrap = scrap;
    def->quoted = quoted;
    def->next = name->defs;
    name->defs = def;
}◇
```

Fragment referenced in [46e](#), [47a](#).

Uses: [arena_getmem 202b](#), [defs 131c](#), [next 131b](#), [Scrap_Node 160a](#).

⟨ *Skip over an in-text scrap 47c* ⟩ ≡

```
{
    int c;
    int depth = 1;
    while ((c = source_get()) != EOF) {
        if (c == nw_char)
            ⟨ Skip over at-sign or go to skipped 48a ⟩
    }
    fprintf(stderr, "%s: unexpected EOF in text at (%s, %d)\n",
               command_name, source_name, source_line);
    exit(-1);

    skipped: ;
}
◇
```

Fragment referenced in [45](#).

Uses: [command_name 30a](#), [exit 22](#), [fprintf 22](#), [nw_char 29bc](#), [source_get 127a](#), [source_line 125b](#), [source_name 125b](#), [stderr 22](#).

⟨ *Skip over at-sign or go to skipped 48a* ⟩ ≡

```
{
  c = source_get();
  switch (c) {
    case '{': case '[': case '(': case '<':
      depth += 1;
      break;
    case '}': case ']': case ')': case '>':
      if (--depth == 0)
        goto skipped;
    case 'x': case '|': case ',':
    case '%': case '1': case '2':
    case '3': case '4': case '5': case '6':
    case '7': case '8': case '9': case '_':
    case 'f': case '#': case '+': case '-':
    case 'v': case '*': case 'c': case '\\':
    case 's':
      break;
    default:
      if (c != nw_char) {
        fprintf(stderr, "%s: unexpected %c%c in text at (%s, %d)\n",
                  command_name, nw_char, c, source_name, source_line);
        exit(-1);
      }
      break;
  }
}
```

◇

Fragment referenced in [47c](#).

Uses: [command_name 30a](#), [exit 22](#), [fprintf 22](#), [nw_char 29bc](#), [source_get 127a](#), [source_line 125b](#), [source_name 125b](#), [stderr 22](#).

2.3.2 Block Comments

⟨ *Collect a block comment 48b* ⟩ ≡

```
{
  char * p = blockBuff;
  char * e = blockBuff + (sizeof(blockBuff)/sizeof(blockBuff[0])) - 1;

  ⟨ Skip whitespace 49a ⟩
  while (p < e)
  {
    ⟨ Add one char to the block buffer 49b ⟩
  }
  if (p == e)
  {
    ⟨ Skip to the next nw-char 51b ⟩
  }
  *p = '\000';
}
```

◇

Fragment referenced in [45](#).

Uses: [blockBuff 51c](#).

```

⟨ Skip whitespace 49a ⟩ ≡
    while (source_peek == ' ',
           || source_peek == '\t',
           || source_peek == '\n')
        (void)source_get();
    ◇

```

Fragment referenced in [48b](#).

Uses: `source_get` [127a](#), `source_peek` [127a](#).

```

⟨ Add one char to the block buffer 49b ⟩ ≡
    int c = source_get();

    if (c == nw_char)
    {
        ⟨ Add an at character to the block or break 50a ⟩
    }
    else if (c == EOF)
    {
        source_ungetc(&c);
        break;
    }
    else
    {
        ⟨ Add any other character to the block 50b ⟩
    }
    ◇

```

Fragment referenced in [48b](#).

Uses: `nw_char` [29bc](#), `source_get` [127a](#), `source_ungetc` [127c](#).

⟨ Add an at character to the block or break 50a ⟩ ≡

```
int cc = source_peek;

if (cc == 'c')
{
    do
        c = source_get();
    while (c <= ' ');

    break;
}
else if (cc == 'd'
        || cc == 'D'
        || cc == 'q'
        || cc == 'Q'
        || cc == 'o'
        || cc == 'O'
        || cc == EOF)
{
    source_ungetc(&c);
    break;
}
else
{
    *p++ = c;
    *p++ = source_get();
}
◇
```

Fragment referenced in [49b](#).

Uses: `source_get` [127a](#), `source_peek` [127a](#), `source_ungetc` [127c](#).

⟨ Add any other character to the block 50b ⟩ ≡

```
    ⟨ Perhaps skip white-space 51a ⟩
    *p++ = c;
◇
```

Fragment referenced in [49b](#).

⟨ Perhaps skip white-space 51a ⟩ $\equiv$

```

    if (c == ' ')
    {
        while (source_peek == ' ')
            c = source_get();
    }
    if (c == '\n')
    {
        if (source_peek == '\n')
        {
            do
                c = source_get();
            while (source_peek == '\n');
        }
        else
            c = ' ';
    }
    ◇

```

Fragment referenced in [50b](#).

Uses: `source_get` [127a](#), `source_peek` [127a](#).

⟨ Skip to the next nw-char 51b ⟩ $\equiv$

```

    int c;

    while ((c = source_get()), c != nw_char && c != EOF) /* Skip */
        source_ungetc(&c); ◇

```

Fragment referenced in [48b](#).

Uses: `nw_char` [29bc](#), `source_get` [127a](#), `source_ungetc` [127c](#).

⟨ Global variable declarations 51c ⟩ $\equiv$

```

    extern char blockBuff[6400];
    ◇

```

Fragment defined by [28](#), [29b](#), [30a](#), [46c](#), [51c](#), [67b](#), [125b](#), [135a](#), [161a](#), [201b](#).

Fragment referenced in [21](#).

Defines: `blockBuff` [48b](#), [51d](#), [52b](#).

⟨ Global variable definitions 51d ⟩ $\equiv$

```

    char blockBuff[6400];
    ◇

```

Fragment defined by [29ac](#), [30b](#), [46d](#), [51d](#), [67c](#), [125c](#), [135b](#), [161b](#).

Fragment referenced in [25e](#).

Uses: `blockBuff` [51c](#).

We don't show block comments inside scraps in the $\text{\TeX}$ file, but we do show where they go.

⟨ Show presence of a block comment 52a ⟩ ≡

```
{
    fputs(delimit_scrap[scrap_type][1],file);
    fprintf(file, "\\hbox{\\sffamily\\slshape (Comment)}");
    fputs(delimit_scrap[scrap_type][0], file);
}
◇
```

Fragment referenced in [74a](#).

Uses: [fprintf 22](#), [fputs 22](#), [scrap_type 71b](#), [72](#).

The block comment is presently in the block buffer. Here we copy it into the scrap whence we shall copy it into the output file.

⟨ Include block comment in a scrap 52b ⟩ ≡

```
{
    char * p = blockBuff;

    push(nw_char, &writer);
    do
    {
        push(c, &writer);
        c = *p++;
    } while (c != '\\0');
}◇
```

Fragment referenced in [138](#).

Uses: [blockBuff 51c](#), [nw_char 29bc](#), [push 135d](#).

⟨ Copy block comment from scrap 53a ⟩ ≡

```
{
    int bgn = indent + global_indent;
    int posn = bgn + strlen(comment_begin[comment_flag]);
    int i;

    ⟨ Perhaps put a delayed indent 155b ⟩
    c = pop(&reader);
    fputs(comment_begin[comment_flag], file);
    while (c != '\0')
    {
        ⟨ Move a word to the file 53b ⟩
        ⟨ If we break the line at this word 53c ⟩
        {
            putchar('\n', file);
            for (i = 0; i < bgn ; i++)
                putchar(' ', file);
            c = pop(&reader);
            if (c != '\0')
            {
                posn = bgn + strlen(comment_mid[comment_flag]);
                fputs(comment_mid[comment_flag], file);
            }
        }
    }
    fputs(comment_end[comment_flag], file);
}
◇
```

Fragment referenced in 152a.

Uses: comment_begin 175, comment_end 175, comment_mid 175, fputs 22, pop 142c, putchar 22, strlen 22.

⟨ Move a word to the file 53b ⟩ ≡

```
do
{
    putchar(c, file);
    posn += 1;
    c = pop(&reader);
} while (c > ' ');
◇
```

Fragment referenced in 53a.

Uses: pop 142c, putchar 22.

⟨ If we break the line at this word 53c ⟩ ≡

```
if (c == '\n' || (c == ' ' && posn > 60))◇
```

Fragment referenced in 53a.

⟨ Skip over a block comment 53d ⟩ ≡

```
if (last == nw_char && c == 'c')
    while ((c = pop(reader.m)) != '\0')
        /* Skip */;
◇
```

Fragment referenced in 192.

Uses: nw_char 29bc, pop 142c.

```

⟨ Begin or end a block comment 54a ⟩ ≡
    if (inBlock)
    {
        ⟨ End block 66c ⟩
    }
    else
    {
        ⟨ Start block 66a ⟩
    }
}◊

```

Fragment referenced in [61a](#).

2.3.3 Fixing the Cross References

Since the definition and reference lists for each name are accumulated in reverse order, we take the time at the end of `pass1` to reverse them all so they'll be simpler to print out prettily. The code for `reverse_lists` appears in [Section 3.3](#).

```

⟨ Reverse cross-reference lists 54b ⟩ ≡
{
    reverse_lists(file_names);
    reverse_lists(macro_names);
    reverse_lists(user_names);
}◊

```

Fragment referenced in [42b](#).

Uses: `file_names` [161a](#), `macro_names` [161a](#), `reverse_lists` [183c](#), `user_names` [161a](#).

2.3.4 Dealing with fragment parameters

Fragment parameters were added on later in nuweb's development. There still is not, for example, an index of fragment parameters. We need a data type to keep track of fragment parameters.

```

⟨ Type declarations 54c ⟩ ≡
    typedef int *Parameters;
    ◊

```

Fragment defined by [23a](#), [54c](#), [122d](#), [160ab](#), [177c](#), [179b](#), [182d](#), [194c](#), [201a](#).

Fragment referenced in [21](#).

Defines: `Parameters` [57](#), [62b](#), [91b](#), [132b](#), [146b](#), [147a](#), [149](#).

When we are copying a scrap to the output, we check for an embedded parameter. If we find it we copy it. Otherwise it's an old-style parameter and we can then pull the *n*th string from the `Parameters` list when we see an `@1 @2`, etc.

```

⟨ Handle macro parameter substitution 54d ⟩ ≡

    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9':
    {
        Arglist * args;
        Name * name;

        lookup(c - '1', inArgs, inParams, &name, &args);
    }

```

```

if (name == (Name *)1) {
    Embed_Node * q = (Embed_Node *)args;
    indent = write_scraps(file, spelling, q->defs,
                          global_indent + indent,
                          indent_chars, debug_flag,
                          tab_flag, indent_flag,
                          0, 0,
                          q->args, inParams,
                          local_parameters, "");
}
else if (name != NULL) {
    int i, narg;
    char * p = name->spelling;
    Arglist *q = args;

    < Perhaps comment this macro 155a >
    indent = write_scraps(file, spelling, name->defs,
                          global_indent + indent,
                          indent_chars, debug_flag,
                          tab_flag, indent_flag,
                          comment_flag, location_flag,
                          args, name->arg,
                          local_parameters, p);
}
else if (args != NULL) {
    if (delayed_indent) {
        < Put out the indent 151b >
    }
    fputs((char *)args, file);
}
else if ( parameters && parameters[c - '1'] ) {
    Scrap_Node param_defs;
    param_defs.scrap = parameters[c - '1'];
    param_defs.next = 0;
    write_scraps(file, spelling, &param_defs,
                  global_indent + indent,
                  indent_chars, debug_flag,
                  tab_flag, indent_flag,
                  comment_flag, location_flag,
                  NULL, NULL, 0, "");
} else if (delayed_indent) {
    < Put out the indent 151b >
}
}
}

```

Fragment referenced in [152a](#).

Uses: Arglist [179b](#), defs [131c](#), delayed_indent [147b](#), Embed_Node [182d](#), fputs [22](#), lookup [144b](#), Name [160b](#), next [131b](#), Scrap_Node [160a](#), write_scraps [147a](#).

Now onto actually parsing fragment parameters from a call. We start off checking for fragment parameters, an @ (sequence followed by parameters separated by @, sequences, and terminated by a @) sequence.

We collect separate scraps for each parameter, and write the scrap numbers down in the text. For example, if the file has:

```
@<foo @( param1 @, param2 @)@>
```

we actually make new scraps, say 10 and 11, for param1 and param2, and write in the collected scrap:

@<foo @(10@,11@)@>

⟨ Save macro parameters 56 ⟩ ≡

```
{
    int param_scrap;
    char param_buf[10];

    push(nw_char, &writer);
    push('(', &writer);
    do {
        param_scrap = collect_scrap();
        sprintf(param_buf, "%d", param_scrap);
        pushs(param_buf, &writer);
        push(nw_char, &writer);
        push(scrap_ended_with, &writer);
        add_to_use(name, current_scrap);
    } while( scrap_ended_with == ',' );
    do
        c = source_get();
    while( ' ' == c );
    if (c == nw_char) {
        c = source_get();
    }
    if (c != '>') {
        /* ZZZ print error */;
    }
}◊
```

Fragment referenced in [141c](#).

Uses: [add_to_use 142a](#), [collect_scrap 136b](#), [nw_char 29bc](#), [push 135d](#), [pushs 136a](#), [source_get 127a](#).

If we get inside, we have at least one parameter, which will be at the beginning of the parms buffer, and we prime the pump with the first character.

⟨ *Check for macro parameters 57* ⟩ ≡

```
if (c == '(') {
    Parameters res = arena_getmem(10 * sizeof(int));
    int *p2 = res;
    int count = 0;
    int scrapnum;

    while( c && c != ')' ) {
        scrapnum = 0;
        c = pop(manager);
        while( '0' <= c && c <= '9' ) {
            scrapnum = scrapnum * 10 + c - '0';
            c = pop(manager);
        }
        if ( c == nw_char ) {
            c = pop(manager);
        }
        *p2++ = scrapnum;
    }
    while (count < 10) {
        *p2++ = 0;
        count++;
    }
    while( c && c != nw_char ) {
        c = pop(manager);
    }
    if ( c == nw_char ) {
        c = pop(manager);
    }
    *parameters = res;
}
◇
```

Fragment referenced in [146c](#).

Uses: [arena_getmem 202b](#), [nw_char 29bc](#), [Parameters 54c](#), [pop 142c](#).

These are used in `write_tex` to output the argument list for a fragment.

⟨ *Format macro parameters 58a* ⟩ ≡

```
char sep;

sep = '(';
do {
    fputc(sep,file);

    fputs("{\\footnotesize ", file);
    write_single_scrap_ref(file, scraps + 1);
    fprintf(file, "\\label{scrap%d}\\n", scraps + 1);
    fputs(" }", file);

    source_last = '{';
    copy_scrap(file, TRUE, NULL);

    ++scraps;

    sep = ',';
} while ( source_last != ')' && source_last != EOF );
fputs(" ) ",file);
do
    c = source_get();
while(c != nw_char && c != EOF);
if (c == nw_char) {
    c = source_get();
}
```

◇

Fragment referenced in 77, 79.

Uses: `copy_scrap` 72, `fprintf` 22, `fputs` 22, `nw_char` 29bc, `scraps` 132a, `source_get` 127a, `source_last` 127a, `TRUE` 23a, `write_single_scrap_ref` 133b.

2.4 Writing the Latex File

The second pass (invoked via a call to `write_tex`) copies most of the text from the source file straight into a `.tex` file. Definitions are formatted slightly and cross-reference information is printed out.

Note that all the formatting is handled in this section. If you don't like the format of definitions or indices or whatever, it'll be in this section somewhere. Similarly, if someone wanted to modify `weft` to work with a different typesetting system, this would be the place to look.

⟨ *Function prototypes 58b* ⟩ ≡

```
extern void write_tex(char *file_name, char *tex_name, unsigned char sector);
```

◇

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: `file_name` 131c, `sector` 131c, `write_tex` 59b.

We need a few local function declarations before we get into the body of `write_tex`.

"latex.c" 59a≡

```
static void copy_scrap(FILE *file, int prefix, Name *name);           /* formats the body of a scrap */
static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps); /* formats a list of scrap numbers */
static void format_entry(Name *name, FILE *tex_file, unsigned char sector); /* formats an index entry */
static void format_file_entry(Name *name, FILE *tex_file);           /* formats a file index entry */
static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector);
static void write_arg(FILE *tex_file, char *p);
static void write_literal(FILE *tex_file, char *p, int mode);
static void write_ArglistElement(FILE *file, Arglist *args, char **params);
◇
```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.

Uses: Arglist 179b, copy_scrap 72, FILE 22, format_entry 82b, format_file_entry 80b, format_user_entry 86a, Name 160b, print_scrap_numbers 69b, scraps 132a, Scrap_Node 160a, sector 131c, write_arg 65a, write_ArglistElement 79, write_literal 71c.

The routine `write_tex` takes two file names as parameters: the name of the web source file and the name of the `.tex` output file.

"latex.c" 59b≡

```
void write_tex(char *file_name, char *tex_name, unsigned char sector)
{
    (void)sector;
    FILE *tex_file = fopen(tex_name, "w");
    if (tex_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", tex_name);
        source_open(file_name);
        ⟨ Write LaTeX limbo definitions 60a ⟩
        ⟨ Copy source_file into tex_file 60c ⟩
        fclose(tex_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, tex_name);
}
◇
```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.

Defines: `write_tex` 41, 58b.

Uses: `command_name` 30a, `fclose` 22, `FILE` 22, `file_name` 131c, `fopen` 22, `fprintf` 22, `sector` 131c, `source_open` 131a, `stderr` 22, `verbose_flag` 28.

Now that the `\NW...` macros are used, it seems convenient to write default definitions for those macros so that source files need not define anything new. If a user wants to change any of the macros (to use `hyperref` or to write in some language other than english) he or she can redefine the commands.

```

⟨ Write LaTeX limbo definitions 60a ⟩ ≡
    if (hyperref_flag) {
        fputs("\\newcommand{\\NWtarget}[2]{\\hypertarget{#1}{#2}}\\n", tex_file);
        fputs("\\newcommand{\\NWlink}[2]{\\hyperlink{#1}{#2}}\\n", tex_file);
    } else {
        fputs("\\newcommand{\\NWtarget}[2]{#2}\\n", tex_file);
        fputs("\\newcommand{\\NWlink}[2]{#2}\\n", tex_file);
    }
    fputs("\\newcommand{\\NWtxtMacroDefBy}{Fragment defined by}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtMacroRefIn}{Fragment referenced in}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtMacroNoRef}{Fragment never referenced}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtDefBy}{Defined by}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtRefIn}{Referenced in}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtNoRef}{Not referenced}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtFileDefBy}{File defined by}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtIdsentsUsed}{Uses:}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtIdsentsNotUsed}{Never used}\\n", tex_file);
    fputs("\\newcommand{\\NWtxtIdsentsDefed}{Defines:}\\n", tex_file);
    fputs("\\newcommand{\\NWsep}{\\$\\diamond}\\n", tex_file);
    fputs("\\newcommand{\\NWnotglobal}{(not defined globally)}\\n", tex_file);
    fputs("\\newcommand{\\NWuseHyperlinks}{", tex_file);
    if (hyperoptions[0] != '\\0')
    {
        ⟨ Write the hyperlink usage macro 60b ⟩
    }
    fputs("}\\n", tex_file);
    ◇

```

Fragment referenced in 59b.

Uses: fputs 22, hyperoptions 28, hyperref_flag 28, Uses 194c.

```

⟨ Write the hyperlink usage macro 60b ⟩ ≡
    fprintf(tex_file, "\\usepackage[%s]{hyperref}", hyperoptions);◇

```

Fragment referenced in 60a.

Uses: fprintf 22, hyperoptions 28.

We make our second (and final) pass through the source web, this time copying characters straight into the .tex file. However, we keep an eye peeled for @ characters, which signal a command sequence.

```

⟨ Copy source_file into tex_file 60c ⟩ ≡
{
    int inBlock = FALSE;
    int c = source_get();
    while (c != EOF) {
        if (c == nw_char)
        {
            ⟨ Interpret at-sequence 61a ⟩
        }
        else {
            putc(c, tex_file);
            c = source_get();
        }
    }
}◇

```

Fragment referenced in 59b.

Uses: FALSE 23a, nw_char 29bc, putc 22, source_get 127a.

Interpret at-sequence 61a $\equiv$

```
{
  int big_definition = FALSE;
  c = source_get();
  switch (c) {
    case 'r':
      c = source_get();
      nw_char = c;
      update_delimit_scrap();
      break;
    case 'O': big_definition = TRUE;
              FALLTHROUGH;
    case 'o': < Write output file definition 63 >
              break;
    case 'Q':
    case 'D': big_definition = TRUE;
              FALLTHROUGH;
    case 'q':
    case 'd': < Write macro definition 64a >
              break;
    case 's':
              < Step to next sector 46a >
              break;
    case 'S':
              < Close the current sector 46b >
              break;
    case '{':
    case '[':
    case '(': < Write in-text scrap 67d >
              break;
    case '<': < Expand macro into tex_file 62b >
              break;
    case 'x': < Copy label from source into (61b tex_file ) 75b >
              c = source_get();
              break;
    case 'c': < Begin or end a block comment 54a >
              c = source_get();
              break;
    case 'f': < Write index of file names 80a >
              break;
    case 'm': < Write index of macro names 81b >
              break;
    case 'u': < Write index of user-specified names 85c >
              break;
    case 'v': < Copy version info into tex file 62a >
              break;
    case 'l':
    case 'L': < Skip at-L language definition line 44 >
              break;
    case 'W': < Skip at-L language definition line 44 >
              break;
    default:
      if (c==nw_char)
        putc(c, tex_file);
      c = source_get();
      break;
  }
}
```

Fragment referenced in [60c](#).

```

⟨ Copy version info into tex file 62a ⟩ ≡
    fputs(version_string, tex_file);
    c = source_get();
    ◇

```

Fragment referenced in 61a.

Uses: fputs 22, source_get 127a, version_string 28.

```

⟨ Expand macro into file 62b ⟩ ≡
{
    Parameters local_parameters = 0;
    int changed;
    char indent_chars[MAX_INDENT];
    Arglist *a;
    Name *name;
    Arglist * args;
    a = collect_scrap_name(0);
    name = a->name;
    args = instance(a->args, NULL, NULL, &changed);
    name->mark = TRUE;
    write_scraps(file, tex_name, name->defs, 0, indent_chars, 0, 0, 1, 0, 0,
        args, name->arg, local_parameters, tex_name);
    name->mark = FALSE;
    c = source_get();
}
◇

```

Fragment referenced in 61a.

Uses: Arglist 179b, collect_scrap_name 180, defs 131c, FALSE 23a, instance 145a, MAX_INDENT 122d, Name 160b, Parameters 54c, source_get 127a, TRUE 23a, write_scraps 147a.

2.4.1 Formatting Definitions

We go through a fair amount of effort to format a file definition. I've derived most of the L^AT_EX commands experimentally; it's quite likely that an expert could do a better job. The L^AT_EX for the previous fragment definition should look like this (perhaps modulo the scrap references):

```

\begin{flushleft} \small
\begin{minipage}{\linewidth}\label{scrap70}\raggedright\small
$\langle$Interpret at-sequence {\footnotesize 18}$\rangle$equiv$
\vspace{-1ex}
\begin{list}{}{} \item
\mbox{}\verb@{\@
\mbox{}\verb@ int big_definition = FALSE;@
\mbox{}\verb@ c = source_get();@
\mbox{}\verb@ switch (c) {@
\mbox{}\verb@ case '0': big_definition = TRUE;@
\mbox{}\verb@ case 'o': @$\langle$Write output file definition {\footnotesize 19a}$\rangle$\verb@@
:
\mbox{}\verb@ case '@{\tt @}\verb@': putc(c, tex_file);@
\mbox{}\verb@ default: c = source_get();@
\mbox{}\verb@ break;@
\mbox{}\verb@ }@
\mbox{}\verb@ @$\diamond$
\end{list}

```

```

\vspace{-1ex}
\footnotesize\addtolength{\baselineskip}{-1ex}
\begin{list}{}{\setlength{\itemsep}{-\parsep}\setlength{\itemindent}{-\leftmargin}}
\item Fragment referenced in scrap 17b.
\end{list}
\end{minipage}\vspace{4ex}
\end{flushleft}

```

The *flushleft* environment is used to avoid L^AT_EX warnings about underful lines. The *minipage* environment is used to avoid page breaks in the middle of scraps. The *verb* command allows arbitrary characters to be printed (however, note the special handling of the @ case in the switch statement).

Fragment and file definitions are formatted nearly identically. I've factored the common parts out into separate scraps.

```

⟨ Write output file definition 63 ⟩ ≡
{
    Name *name = collect_file_name();
    ⟨ Begin the scrap environment 65b ⟩
    fputs("\NWtarget{weft", tex_file);
    write_single_scrap_ref(tex_file, scraps);
    fputs("{}{} ", tex_file);
    fprintf(tex_file, "\\verb%c\\\"%s\\\"%c\\nobreak\\ {\\footnotesize {", nw_char, name->spelling, nw_char);
    write_single_scrap_ref(tex_file, scraps);
    fputs("{}$\\equiv$\n", tex_file);
    ⟨ Fill in the middle of the scrap environment 66d ⟩
    ⟨ Begin the cross-reference environment 68a ⟩
    if ( scrap_flag ) {
        ⟨ Write file defs 68c ⟩
    }
    format_defs_refs(tex_file, scraps);
    format_uses_refs(tex_file, scraps++);
    ⟨ Finish the cross-reference environment 68b ⟩
    ⟨ Finish the scrap environment 67a ⟩
}◊

```

Fragment referenced in 61a.

Uses: collect_file_name 172, format_defs_refs 196d, format_uses_refs 195b, fprintf 22, fputs 22, Name 160b, nw_char 29bc, scraps 132a, scrap_flag 28, write_single_scrap_ref 133b.

I don't format a fragment name at all specially, figuring the programmer might want to use italics or bold face in the midst of the name.

```

⟨ Write macro definition 64a ⟩ ≡
{
    Name *name = collect_macro_name();

    ⟨ Begin the scrap environment 65b ⟩
    fputs("\\NWtarget{weft", tex_file);
    write_single_scrap_ref(tex_file, scraps);
    fputs("{} {} $\\langle\\$,\\{\\itshape ", tex_file);
    ⟨ Write the macro's name 64b ⟩
    fputs("\\nobreak\\ {\\footnotesize {", tex_file);
    write_single_scrap_ref(tex_file, scraps);
    fputs("}}$\\,\\rangle\\equiv$\\n", tex_file);
    ⟨ Fill in the middle of the scrap environment 66d ⟩
    ⟨ Begin the cross-reference environment 68a ⟩
    ⟨ Write macro defs 68d ⟩
    ⟨ Write macro refs 69a ⟩
    format_defs_refs(tex_file, scraps);
    format_uses_refs(tex_file, scraps++);
    ⟨ Finish the cross-reference environment 68b ⟩
    ⟨ Finish the scrap environment 67a ⟩
}◊

```

Fragment referenced in 61a.

Uses: collect_macro_name 176, format_defs_refs 196d, format_uses_refs 195b, fputs 22, Name 160b, scraps 132a, write_single_scrap_ref 133b.

```

⟨ Write the macro's name 64b ⟩ ≡
{
    char * p = name->spelling;
    int i = 0;

    while (*p != '\\000') {
        if (*p == ARG_CHR) {
            write_arg(tex_file, name->arg[i++]);
            p++;
        }
        else
            fputc(*p++, tex_file);
    }
}◊

```

Fragment referenced in 64a, 83b.

Uses: ARG_CHR 177c, write_arg 65a.

```

"latex.c" 65a≡
static void write_arg(FILE * tex_file, char * p)
{
    fputs("\\hbox{\\slshape\\sffamily ", tex_file);
    while (*p)
    {
        switch (*p)
        {
            case '$':
            case '_':
            case '^':
            case '#':
                fputc('\\', tex_file);
                break;
            default:
                break;
        }
        fputc(*p, tex_file);
        p++;
    }

    fputs("\\\\}", tex_file);
}
◇

```

File defined by [24b](#), [59ab](#), [65a](#), [69b](#), [70ab](#), [71bc](#), [72](#), [73a](#), [79](#), [80b](#), [82ab](#), [85b](#), [86a](#).
 Defines: `write_arg` [59a](#), [64b](#), [74a](#), [79](#).
 Uses: `FILE` [22](#), `fputs` [22](#).

```

⟨ Begin the scrap environment 65b ⟩ ≡
{
    if (big_definition)
    {
        if (inBlock)
        {
            ⟨ End block 66c ⟩
        }
        fputs("\\begin{flushleft} \\small", tex_file);
    }
    else
    {
        if (inBlock)
        {
            ⟨ Switch block 66b ⟩
        }
        else
        {
            ⟨ Start block 66a ⟩
        }
    }
    fprintf(tex_file, "\\label{scrap%d}\\raggedright\\small\\n", scraps);
}◇

```

Fragment referenced in [63](#), [64a](#).
 Uses: `fprintf` [22](#), `fputs` [22](#), `scraps` [132a](#).

⟨ Start block 66a ⟩ ≡

```
fputs("\\begin{flushleft} \\small\\n\\begin{minipage}{\\linewidth}", tex_file);
inBlock = TRUE;◇
```

Fragment referenced in 54a, 65b.
Uses: fputs 22, TRUE 23a.

⟨ Switch block 66b ⟩ ≡

```
fputs("\\par\\vspace{\\baselineskip}\\n", tex_file);◇
```

Fragment referenced in 65b.
Uses: fputs 22.

⟨ End block 66c ⟩ ≡

```
fputs("\\end{minipage}\\vspace{4ex}\\n", tex_file);
fputs("\\end{flushleft}\\n", tex_file);
inBlock = FALSE;◇
```

Fragment referenced in 54a, 65b, 67a.
Uses: FALSE 23a, fputs 22.

The interesting things here are the ◇ inserted at the end of each scrap and the various spacing commands. The diamond helps to clearly indicate the end of a scrap. The spacing commands were derived empirically; they may be adjusted to taste.

⟨ Fill in the middle of the scrap environment 66d ⟩ ≡

```
{
  fputs("\\vspace{-1ex}\\n\\begin{list}{}{} \\item\\n", tex_file);
  extra_scraps = 0;
  copy_scrap(tex_file, TRUE, name);
  fputs("{\\NWsep}\\n\\end{list}\\n", tex_file);
}◇
```

Fragment referenced in 63, 64a.
Uses: copy_scrap 72, extra_scraps 67b, fputs 22, TRUE 23a.

We've got one last spacing command, controlling the amount of white space after a scrap.

Note also the whitespace eater. I use it to remove any blank lines that appear after a scrap in the source file. This way, text following a scrap will not be indented. Again, this is a matter of personal taste.

```
< Finish the scrap environment 67a > ≡
{
    scraps += extra_scraps;
    if (big_definition)
        fputs("\\vspace{4ex}\\n\\end{flushleft}\\n", tex_file);
    else
    {
        < End block 66c >
    }
    do
        c = source_get();
    while (isspace(c));
}◇
```

Fragment referenced in 63, 64a.

Uses: [extra_scraps 67b](#), [fputs 22](#), [isspace 22](#), [scraps 132a](#), [source_get 127a](#).

```
< Global variable declarations 67b > ≡
extern int extra_scraps;
◇
```

Fragment defined by 28, 29b, 30a, 46c, 51c, 67b, 125b, 135a, 161a, 201b.

Fragment referenced in 21.

Defines: [extra_scraps 66d](#), [67ac](#), 79, 93bc, 103.

```
< Global variable definitions 67c > ≡
int extra_scraps = 0;
◇
```

Fragment defined by 29ac, 30b, 46d, 51d, 67c, 125c, 135b, 161b.

Fragment referenced in 25e.

Uses: [extra_scraps 67b](#).

```
< Write in-text scrap 67d > ≡
copy_scrap(tex_file, FALSE, NULL);
c = source_get();
◇
```

Fragment referenced in 61a.

Uses: [copy_scrap 72](#), [FALSE 23a](#), [source_get 127a](#).

Formatting Cross References

⟨ Begin the cross-reference environment 68a ⟩ ≡

```
{
  fputs("\\vspace{-1.5ex}\\n", tex_file);
  fputs("\\footnotesize\\n", tex_file);
  fputs("\\begin{list}{\\setlength{\\itemsep}{-\\parsep}",
    tex_file);
  fputs("\\setlength{\\itemindent}{-\\leftmargin}\\n", tex_file);}
}◊
```

Fragment referenced in [63](#), [64a](#).

Uses: `fputs` [22](#).

There may (unusually) be nothing to output in the cross-reference section, so output an empty list item anyway to avoid having an empty list.

⟨ Finish the cross-reference environment 68b ⟩ ≡

```
{
  fputs("\\n\\item{}", tex_file);
  fputs("\\n\\end{list}\\n", tex_file);
}◊
```

Fragment referenced in [63](#), [64a](#).

Uses: `fputs` [22](#).

⟨ Write file defs 68c ⟩ ≡

```
{
  if (name->defs) {
    if (name->defs->next) {
      fputs("\\item \\NWtxtFileDefBy\\ ", tex_file);
      print_scrap_numbers(tex_file, name->defs);
    }
  } else {
    fprintf(stderr,
      "would have crashed in 'Write file defs' for '%s'\\n",
      name->spelling);
  }
}◊
```

Fragment referenced in [63](#).

Uses: `defs` [131c](#), `fprintf` [22](#), `fputs` [22](#), `next` [131b](#), `print_scrap_numbers` [69b](#), `stderr` [22](#).

⟨ Write macro defs 68d ⟩ ≡

```
{
  if (name->defs->next) {
    fputs("\\item \\NWtxtMacroDefBy\\ ", tex_file);
    print_scrap_numbers(tex_file, name->defs);
  }
}◊
```

Fragment referenced in [64a](#).

Uses: `defs` [131c](#), `fputs` [22](#), `next` [131b](#), `print_scrap_numbers` [69b](#).

< Write macro refs 69a> ≡

```
{
  if (name->uses) {
    if (name->uses->next) {
      fputs("\\item \\NWtxtMacroRefIn\\ ", tex_file);
      print_scrap_numbers(tex_file, name->uses);
    }
    else {
      fputs("\\item \\NWtxtMacroRefIn\\ ", tex_file);
      fputs("\\NWlink{weft", tex_file);
      write_single_scrap_ref(tex_file, name->uses->scrap);
      fputs("{", tex_file);
      write_single_scrap_ref(tex_file, name->uses->scrap);
      fputs("}", tex_file);
      fputs(".\\n", tex_file);
    }
  }
  else {
    fputs("\\item {\\NWtxtMacroNoRef}.\\n", tex_file);
    fprintf(stderr, "%s: <%s> never referenced.\\n",
              command_name, name->spelling);
  }
}
}◊
```

Fragment referenced in 64a.

Uses: `command_name` 30a, `fprintf` 22, `fputs` 22, `next` 131b, `print_scrap_numbers` 69b, `stderr` 22, `uses` 131c, `write_single_scrap_ref` 133b.

"`latex.c`" 69b≡

```
static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps)
{
  int page;
  fputs("\\NWlink{weft", tex_file);
  write_scrap_ref(tex_file, scraps->scrap, -1, &page);
  fputs("{", tex_file);
  write_scrap_ref(tex_file, scraps->scrap, TRUE, &page);
  fputs("}", tex_file);
  scraps = scraps->next;
  while (scraps) {
    fputs("\\NWlink{weft", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, -1, &page);
    fputs("{", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, FALSE, &page);
    scraps = scraps->next;
    fputs("}", tex_file);
  }
  fputs(".\\n", tex_file);
}
}◊
```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.

Defines: `print_scrap_numbers` 59a, 68cd, 69a, 81a, 84b.

Uses: `FALSE` 23a, `FILE` 22, `fputs` 22, `next` 131b, `page` 131c, `scraps` 132a, `Scrap_Node` 160a, `TRUE` 23a, `write_scrap_ref` 133a.

Formatting a Scrap

We add a `\mbox{}` at the beginning of each line to avoid problems with older versions of $\TeX$. This is the only place we really care whether a scrap is delimited with `@{...@}`, `@[...@]`, or `@(...@)`, and we base our output sequences on that.

We have an array `delimit_scrap` where we store our strings that perform the formatting for one line of a scrap. Upon initialisation we copy our strings from the source `orig_delimit_scrap` to it to have the strings writeable. We might change the strings later when we encounter the command to change the ‘weft character’.

```
"latex.c" 70a≡
static char *orig_delimit_scrap[3][5] = {
    /* {} mode: begin, end, insert nw_char, prefix, suffix */
    { "\\verb@", "@", "@{\\tt @}\\verb@", "\\mbox{}", "\\\"\\\"\\\"\\\" \" },
    /* [] mode: begin, end, insert nw_char, prefix, suffix */
    { "", "", "@", "", "" },
    /* () mode: begin, end, insert nw_char, prefix, suffix */
    { "$", "$", "@", "", "" },
};

static char *delimit_scrap[3][5];
◇
```

File defined by [24b](#), [59ab](#), [65a](#), [69b](#), [70ab](#), [71bc](#), [72](#), [73a](#), [79](#), [80b](#), [82ab](#), [85b](#), [86a](#).
Uses: [nw_char](#) [29bc](#).

The function `initialise_delimit_scrap_array` does the copying. If we want to have the listings package do the formatting we have to replace only two of those strings: the `verb` command has to be replaced by the package’s `lstinline` command.

```

"latex.c" 70b≡
void initialise_delimit_scrap_array(void) {
    int i,j;
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 5; j++) {
            if((delimit_scrap[i][j] = strdup(orig_delimit_scrap[i][j])) == NULL) {
                fprintf(stderr, "Not enough memory for string allocation\n");
                exit(EXIT_FAILURE);
            }
        }
    }
}

/* replace verb by lstinline */
if (listings_flag) {
    free(delimit_scrap[0][0]);
    if((delimit_scrap[0][0]=strdup("\\lstinline@")) == NULL) {
        fprintf(stderr, "Not enough memory for string allocation\n");
        exit(EXIT_FAILURE);
    }
    free(delimit_scrap[0][2]);
    if((delimit_scrap[0][2]=strdup("@{\\tt @}\\lstinline@")) == NULL) {
        fprintf(stderr, "Not enough memory for string allocation\n");
        exit(EXIT_FAILURE);
    }
}
}
}
◇

```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.
 Uses: exit 22, fprintf 22, stderr 22.

```

⟨ Function prototypes 71a ⟩ ≡
void initialise_delimit_scrap_array(void);
◇

```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c,
 195a, 196c, 199b, 201c.
 Fragment referenced in 21.

```

"latex.c" 71b≡
int scrap_type = 0;
◇

```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.
 Defines: scrap_type 52a, 72, 74a, 76ab, 77, 79.

```

"latex.c" 71c≡
static void write_literal(FILE * tex_file, char * p, int mode)
{
    fputs(delimit_scrap[mode][0], tex_file);
    while (*p!= '\000') {
        if (*p == nw_char)
            fputs(delimit_scrap[mode][2], tex_file);
        else
            fputc(*p, tex_file);
        p++;
    }
    fputs(delimit_scrap[mode][1], tex_file);
}
◇

```

File defined by [24b](#), [59ab](#), [65a](#), [69b](#), [70ab](#), [71bc](#), [72](#), [73a](#), [79](#), [80b](#), [82ab](#), [85b](#), [86a](#).

Defines: `write_literal` [59a](#), [77](#), [79](#).

Uses: `FILE` [22](#), `fputs` [22](#), `nw_char` [29bc](#).

```

"latex.c" 72≡
static void copy_scrap(FILE *file, int prefix, Name *name)
{
    int indent = 0;
    int c;
    char ** params = name->arg;
    if (source_last == '{') scrap_type = 0;
    if (source_last == '[') scrap_type = 1;
    if (source_last == '(') scrap_type = 2;
    c = source_get();
    if (prefix) fputs(delimit_scrap[scrap_type][3], file);
    fputs(delimit_scrap[scrap_type][0], file);
    while (1) {
        switch (c) {
            case '\n': fputs(delimit_scrap[scrap_type][1], file);
                       if (prefix) fputs(delimit_scrap[scrap_type][4], file);
                       fputs("\n", file);
                       if (prefix) fputs(delimit_scrap[scrap_type][3], file);
                       fputs(delimit_scrap[scrap_type][0], file);
                       indent = 0;
                       break;
            case '\t': < Expand tab into spaces 73c >
                       break;
            default:
                if (c==nw_char)
                {
                    < Check at-sequence for end-of-scrap 74a >
                    break;
                }
                putc(c, file);
                indent++;
                break;
        }
        c = source_get();
    }
}
◇

```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.

Defines: copy_scrap 58a, 59a, 66d, 67d, scrap_type 52a, 71b, 74a, 76ab, 77, 79.

Uses: FILE 22, fputs 22, Name 160b, nw_char 29bc, putc 22, source_get 127a, source_last 127a.

When we encounter the command to change the ‘weft character’ we call this function. It updates the scrap formatting directives accordingly.

```

"latex.c" 73a≡
void update_delimit_scrap(void)
{
    /* {}-mode begin */
    if (listings_flag) {
        delimit_scrap[0][0][10] = nw_char;
    } else {
        delimit_scrap[0][0][5] = nw_char;
    }
    /* {}-mode end */
    delimit_scrap[0][1][0] = nw_char;
    /* {}-mode insert nw_char */

    delimit_scrap[0][2][0] = nw_char;
    delimit_scrap[0][2][6] = nw_char;

    if (listings_flag) {
        delimit_scrap[0][2][18] = nw_char;
    } else {
        delimit_scrap[0][2][13] = nw_char;
    }

    /* []-mode insert nw_char */
    delimit_scrap[1][2][0] = nw_char;

    /* ()-mode insert nw_char */
    delimit_scrap[2][2][0] = nw_char;
}
◇

```

File defined by [24b](#), [59ab](#), [65a](#), [69b](#), [70ab](#), [71bc](#), [72](#), [73a](#), [79](#), [80b](#), [82ab](#), [85b](#), [86a](#).
 Defines: `update_delimit_scrap` [45](#), [61a](#), [73b](#), [90a](#).
 Uses: `nw_char` [29bc](#).

```

⟨ Function prototypes 73b ⟩ ≡
    void update_delimit_scrap(void);
◇

```

Fragment defined by [42a](#), [58b](#), [71a](#), [73b](#), [85a](#), [87](#), [122ab](#), [125a](#), [127b](#), [132b](#), [134b](#), [142b](#), [145b](#), [157b](#), [161c](#), [164b](#), [178d](#), [188c](#),
[195a](#), [196c](#), [199b](#), [201c](#).
 Fragment referenced in [21](#).
 Uses: `update_delimit_scrap` [73a](#).

```

⟨ Expand tab into spaces 73c ⟩ ≡
{
    int delta = 8 - (indent % 8);
    indent += delta;
    while (delta > 0) {
        putc(' ', file);
        delta--;
    }
}
◇

```

Fragment referenced in [72](#), [151d](#).
 Uses: `putc` [22](#).

< Check at-sequence for end-of-scrap 74a > ≡

```
{
  c = source_get();
  switch (c) {
    case 'c': < Show presence of a block comment 52a >
      break;
    case 'x': < Copy label from source into (74b file ) 75b >
      break;
    case 'v': < Copy version info into file 75a >
    case 's':
      break;
    case '+':
    case '-':
    case '*':
    case '|': < Skip over index entries 75d >
      FALLTHROUGH;
    case ',':
    case ')':
    case ']':
    case '}': fputs(delimit_scrap[scrap_type][1], file);
      return;
    case '<': < Format macro name 77 >
      break;
    case '%': < Skip commented-out code 75e >
      break;
    case '_': < Bold Keyword 76a >
      break;
    case 't': < Italic "fragment title" 76b >
      break;
    case 'f': < Italic "file name" 76b >
      break;
    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9':
      if (name == NULL
          || name->arg[c - '1'] == NULL) {
        fputs(delimit_scrap[scrap_type][2], file);
        fputc(c, file);
      }
      else {
        fputs(delimit_scrap[scrap_type][1], file);
        write_arg(file, name->arg[c - '1']);
        fputs(delimit_scrap[scrap_type][0], file);
      }
      break;
    default:
      if (c==nw_char)
      {
        fputs(delimit_scrap[scrap_type][2], file);
        break;
      }
      /* ignore these since pass1 will have warned about them */
      break;
  }
}
```

Fragment referenced in [72](#).

```

< Copy version info into file 75a > ≡
    fputs(version_string, file);
    ◇

```

Fragment referenced in [74a](#), [99a](#), [152a](#).
 Uses: `fputs` [22](#), `version_string` [28](#).

```

< Copy label from source into 75b > ≡
{
    < Get label from (75c source_get() ) 198c >
    write_label(label_name, @1);
}◇

```

Fragment referenced in [61a](#), [74a](#), [90a](#), [99a](#).
 Uses: `source_get` [127a](#).

There's no need to check for errors here, since we will have already pointed out any during the first pass.

```

< Skip over index entries 75d > ≡
{
    do {
        do
            c = source_get();
            while (c != nw_char);
            c = source_get();
        } while (c != '}' && c != ']' && c != ')' );
    }◇

```

Fragment referenced in [74a](#), [99a](#).
 Uses: `nw_char` [29bc](#), `source_get` [127a](#).

```

< Skip commented-out code 75e > ≡
{
    do
        c = source_get();
        while (c != '\n');
    }◇

```

Fragment referenced in [74a](#), [99a](#), [138](#).
 Uses: `source_get` [127a](#).

This scrap helps deal with bold keywords:

⟨ *Bold Keyword* 76a ⟩ ≡

```
{
    fputs(delimit_scrap[scrap_type][1],file);
    fprintf(file, "\\hbox{\\sffamily\\bfseries ");
    c = source_get();
    do {
        fputc(c, file);
        c = source_get();
    } while (c != nw_char);
    c = source_get();
    fprintf(file, "}");
    fputs(delimit_scrap[scrap_type][0], file);
}◊
```

Fragment referenced in 74a.

Uses: fprintf 22, fputs 22, nw_char 29bc, scrap_type 71b, 72, source_get 127a.

⟨ *Italic "whatever"* 76b ⟩ ≡

```
{
    fputs(delimit_scrap[scrap_type][1],file);
    fprintf(file, "\\hbox{\\sffamily\\slshape whatever}");
    fputs(delimit_scrap[scrap_type][0], file);
}◊
```

Fragment referenced in 74a.

Uses: fprintf 22, fputs 22, scrap_type 71b, 72.

< Format macro name 77 > ≡

```
{
    Arglist *args = collect_scrap_name(-1);
    Name *name = args->name;
    char * p = name->spelling;
    Arglist *q = args->args;
    int narg = 0;

    fputs(delimit_scrap[scrap_type][1],file);
    if (prefix)
        fputs("\\hbox{", file);
    fputs("$\\langle\\,${\\itshape ", file);
    while (*p != '\\000') {
        if (*p == ARG_CHR) {
            if (q == NULL) {
                write_literal(file, name->arg[narg], scrap_type);
            }
            else {
                write_ArglistElement(file, q, params);
                q = q->next;
            }
            p++;
            narg++;
        }
        else
            fputc(*p++, file);
    }
    fputs("}\\nobreak\\ ", file);
    if (scrap_name_has_parameters) {
        < Format macro parameters 58a >
    }
    fprintf(file, "{\\footnotesize ");
    if (name->defs)
        < Write abbreviated definition list 78 >
    else {
        putc('?', file);
        fprintf(stderr, "%s: never defined <%s>\\n",
            command_name, name->spelling);
    }
    fputs("$\\,\\rangle$", file);
    if (prefix)
        fputs("}", file);
    fputs(delimit_scrap[scrap_type][0], file);
}◊
```

Fragment referenced in [74a](#).

Uses: Arglist [179b](#), ARG_CHR [177c](#), collect_scrap_name [180](#), command_name [30a](#), defs [131c](#), fprintf [22](#), fputs [22](#), Name [160b](#), next [131b](#), putc [22](#), scrap_type [71b](#), [72](#), stderr [22](#), write_ArglistElement [79](#), write_literal [71c](#).

< Write abbreviated definition list 78 > ≡

```
{
    Scrap_Node *p = name->defs;
    fputs("\\NWlink{weft", file);
    write_single_scrap_ref(file, p->scrap);
    fputs("{}{", file);
    write_single_scrap_ref(file, p->scrap);
    fputs("}", file);
    p = p->next;
    if (p)
        fputs(", \\ldots\\ ", file);
}
```

Fragment referenced in [77](#), [79](#).

Uses: [defs 131c](#), [fputs 22](#), [next 131b](#), [Scrap_Node 160a](#), [write_single_scrap_ref 133b](#).

```

"latex.c" 79≡
static void
write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        char * p = (char*)q;

        if (p[0] == ARG_CHR) {
            write_arg(file, params[p[1] - '1']);
        } else {
            write_literal(file, (char *)q, 0);
        }
    } else if (name == (Name *)1) {
        Scrap_Node * qq = (Scrap_Node *)q;
        qq->quoted = TRUE;
        fputs(delimit_scrap[scrap_type][0], file);
        write_scraps(file, "", qq,
                     -1, "", 0, 0, 0, 0, 0,
                     NULL, params, 0, "");
        fputs(delimit_scrap[scrap_type][1], file);
        extra_scraps++;
        qq->quoted = FALSE;
    } else {
        char * p = name->spelling;
        fputs("$\\langle\\,${\\itshape ", file);
        while (*p != '\\000') {
            if (*p == ARG_CHR) {
                write_ArglistElement(file, q, params);
                q = q->next;
                p++;
            }
            else
                fputc(*p++, file);
        }
        fputs("\\nobreak\\ ", file);
        if (scrap_name_has_parameters) {
            int c;

            < Format macro parameters 58a >
        }
        fprintf(file, "{\\footnotesize ");
        if (name->defs)
            < Write abbreviated definition list 78 >
        else {
            putc('?', file);
            fprintf(stderr, "%s: never defined <%s>\n",
                    command_name, name->spelling);
        }
        fputs("$\\,\\rangle$", file);
    }
}
}
◇

```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.

Defines: write_ArglistElement 59a, 77.

Uses: Arglist 179b, ARG_CHR 177c, command_name 30a, defs 131c, extra_scraps 67b, FALSE 23a, FILE 22, fprintf 22, fputs 22, Name 160b, next 131b, putc 22, Scrap_Node 160a, scrap_type 71b, 72, stderr 22, TRUE 23a, write_arg 65a, write_literal 71c, write_scraps 147a.

2.4.2 Generating the Indices

< Write index of file names 80a > ≡

```
{
  if (file_names) {
    fputs("\n{\small\begin{list}{}{\setlength{\itemsep}{-\parsep}",
          tex_file);
    fputs("\setlength{\itemindent}{-\leftmargin}}\n", tex_file);
    format_file_entry(file_names, tex_file);
    fputs("\end{list}}", tex_file);
  }
  c = source_get();
}◊
```

Fragment referenced in [61a](#).

Uses: [file_names 161a](#), [format_file_entry 80b](#), [fputs 22](#), [source_get 127a](#).

"[latex.c](#)" [80b](#)≡

```
static void format_file_entry(Name *name, FILE *tex_file)
{
  while (name) {
    format_file_entry(name->llink, tex_file);
    < Format a file index entry 80c >
    name = name->rlink;
  }
}
◊
```

File defined by [24b](#), [59ab](#), [65a](#), [69b](#), [70ab](#), [71bc](#), [72](#), [73a](#), [79](#), [80b](#), [82ab](#), [85b](#), [86a](#).

Defines: [format_file_entry 59a](#), [80a](#).

Uses: [FILE 22](#), [Name 160b](#).

< Format a file index entry 80c > ≡

```
fputs("\item ", tex_file);
fprintf(tex_file, "\\verb%c\\%s\\%c ", nw_char, name->spelling, nw_char);
< Write file's defining scrap numbers 81a >
putc('\\n', tex_file);◊
```

Fragment referenced in [80b](#).

Uses: [fprintf 22](#), [fputs 22](#), [nw_char 29bc](#), [putc 22](#).

< Write file's defining scrap numbers 81a > ≡

```
{
    Scrap_Node *p = name->defs;
    fputs("{\\footnotesize {\\NWtxtDefBy}", tex_file);
    if (p->next) {
        /* fputs("s ", tex_file); */
        putc(' ', tex_file);
        print_scrap_numbers(tex_file, p);
    }
    else {
        putc(' ', tex_file);
        fputs("\\NWlink{weft", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("{", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("}", tex_file);
        putc('.', tex_file);
    }
    putc('}', tex_file);
}◇
```

Fragment referenced in [80c](#).

Uses: [defs 131c](#), [fputs 22](#), [next 131b](#), [print_scrap_numbers 69b](#), [putc 22](#), [Scrap_Node 160a](#), [write_single_scrap_ref 133b](#).

< Write index of macro names 81b > ≡

```
{
    unsigned char sector = current_sector;
    int c = source_get();
    if (c == '+')
        sector = 0;
    else
        source_ungetc(&c);
    if (has_sector(macro_names, sector)) {
        fputs("\\n{\\small\\begin{list}{\\setlength{\\itemsep}{-\\parsep}",
            tex_file);
        fputs("\\setlength{\\itemindent}{-\\leftmargin}}\\n", tex_file);
        format_entry(macro_names, tex_file, sector);
        fputs("\\end{list}}", tex_file);
    } else {
        fputs("None.\\n", tex_file);
    }
}
c = source_get();
◇
```

Fragment referenced in [61a](#).

Uses: [current_sector 46c](#), [format_entry 82b](#), [fputs 22](#), [has_sector 85b](#), [macro_names 161a](#), [sector 131c](#), [source_get 127a](#), [source_ungetc 127c](#).

```
"latex.c" 82a≡
static int load_entry(Name * name, Name ** nms, int n)
{
    while (name) {
        n = load_entry(name->llink, nms, n);
        nms[n++] = name;
        name = name->rlink;
    }
    return n;
}
◇
```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.
 Defines: load_entry 82b.
 Uses: Name 160b.

```
"latex.c" 82b≡
static void format_entry(Name *name, FILE *tex_file, unsigned char sector)
{
    Name ** nms = malloc(num_scraps()*sizeof(Name *));
    int n = load_entry(name, nms, 0);
    int i;

    ⟨ Sort nms of size n for ⟨ Rob's ordering 82c ⟩ 83a ⟩
    for (i = 0; i < n; i++)
    {
        Name * name = nms[i];

        ⟨ Format an index entry 83b ⟩
    }
}
◇
```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.
 Defines: format_entry 59a, 81b.
 Uses: FILE 22, load_entry 82a, malloc 22, Name 160b, num_scraps 132a, sector 131c.

```
⟨ Rob's ordering 82c ⟩ ≡
    robs_strcmp(ki->spelling, kj->spelling) < 0◇
```

Fragment referenced in 82b, 106a.
 Uses: robs_strcmp 165a.

⟨ Sort key of size n for ordering 83a ⟩ ≡

```

int j;
for (j = 1; j < n; j++)
{
    int i = j - 1;
    Name * kj = key[j];

    do
    {
        Name * ki = key[i];

        if (ordering)
            break;
        key[i + 1] = ki;
        i -= 1;
    } while (i >= 0);
    key[i + 1] = kj;
}
◇

```

Fragment referenced in [82b](#), [106a](#).

Uses: Name [160b](#).

⟨ Format an index entry 83b ⟩ ≡

```

if (name->sector == sector){
    fputs("\\item ", tex_file);
    fputs("$\\langle\\$, ", tex_file);
    ⟨ Write the macro's name 64b ⟩
    fputs("\\nobreak\\ {\\footnotesize ", tex_file);
    ⟨ Write defining scrap numbers 84a ⟩
    fputs("}\\$,\\rangle$ ", tex_file);
    ⟨ Write referencing scrap numbers 84b ⟩
    putc('\\n', tex_file);
}◇

```

Fragment referenced in [82b](#).

Uses: fputs [22](#), putc [22](#), sector [131c](#).

⟨ Write defining scrap numbers 84a ⟩ ≡

```
{
  Scrap_Node *p = name->defs;
  if (p) {
    int page;
    fputs("\\NWlink{weft", tex_file);
    write_scrap_ref(tex_file, p->scrap, -1, &page);
    fputs("{}{", tex_file);
    write_scrap_ref(tex_file, p->scrap, TRUE, &page);
    fputs("}", tex_file);
    p = p->next;
    while (p) {
      fputs("\\NWlink{weft", tex_file);
      write_scrap_ref(tex_file, p->scrap, -1, &page);
      fputs("{}{", tex_file);
      write_scrap_ref(tex_file, p->scrap, FALSE, &page);
      fputs("}", tex_file);
      p = p->next;
    }
  }
  else
    putc('?', tex_file);
}◊
```

Fragment referenced in 83b.

Uses: defs 131c, FALSE 23a, fputs 22, next 131b, page 131c, putc 22, Scrap_Node 160a, TRUE 23a, write_scrap_ref 133a.

⟨ Write referencing scrap numbers 84b ⟩ ≡

```
{
  Scrap_Node *p = name->uses;
  fputs("{\\footnotesize ", tex_file);
  if (p) {
    fputs("{\\NWtxtRefIn}", tex_file);
    if (p->next) {
      /* fputs("s ", tex_file); */
      putc(' ', tex_file);
      print_scrap_numbers(tex_file, p);
    }
    else {
      putc(' ', tex_file);
      fputs("\\NWlink{weft", tex_file);
      write_single_scrap_ref(tex_file, p->scrap);
      fputs("{}{", tex_file);
      write_single_scrap_ref(tex_file, p->scrap);
      fputs("}", tex_file);
      putc('.', tex_file);
    }
  }
  else
    fputs("{\\NWtxtNoRef}.", tex_file);
  putc('}', tex_file);
}◊
```

Fragment referenced in 83b.

Uses: fputs 22, next 131b, print_scrap_numbers 69b, putc 22, Scrap_Node 160a, uses 131c, write_single_scrap_ref 133b.

< Function prototypes 85a > ≡

```
extern int has_sector(Name *, unsigned char);
◇
```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: has_sector 85b, Name 160b.

"latex.c" 85b≡

```
int has_sector(Name * name, unsigned char sector)
{
    while(name) {
        if (name->sector == sector)
            return TRUE;
        if (has_sector(name->llink, sector))
            return TRUE;
        name = name->rlink;
    }
    return FALSE;
}
◇
```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.

Defines: has_sector 81b, 85ac, 105a, 108a.

Uses: FALSE 23a, Name 160b, sector 131c, TRUE 23a.

< Write index of user-specified names 85c > ≡

```
{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    if (has_sector(user_names, sector)) {
        fputs("\n{\small\begin{list}{}{\setlength{\itemsep}{-\parsep}",
            tex_file);
        fputs("\setlength{\itemindent}{-\leftmargin}}\n", tex_file);
        format_user_entry(user_names, tex_file, sector);
        fputs("\end{list}}", tex_file);
    }
}
}◇
```

Fragment referenced in 61a.

Uses: current_sector 46c, format_user_entry 86a, fputs 22, has_sector 85b, sector 131c, source_get 127a, user_names 161a.

```
"latex.c" 86a≡
static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector)
{
    while (name) {
        format_user_entry(name->llink, tex_file, sector);
        ⟨Format a user index entry 86b⟩
        name = name->rlink;
    }
}
◇
```

File defined by 24b, 59ab, 65a, 69b, 70ab, 71bc, 72, 73a, 79, 80b, 82ab, 85b, 86a.

Defines: format_user_entry 59a, 85c.

Uses: FILE 22, Name 160b, sector 131c.

```
⟨Format a user index entry 86b⟩ ≡
if (name->sector == sector){
    Scrap_Node *uses = name->uses;
    if ( uses || dangling_flag ) {
        int page;
        Scrap_Node *defs = name->defs;
        fprintf(tex_file, "\\item \\verb%c%s%c: ", nw_char,name->spelling,nw_char);
        if (!uses) {
            fputs("\\underline{", tex_file);
            fputs("\\NWlink{weft", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("}{", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("}}", tex_file);
            page = -2;
            defs = defs->next;
        }
        else
            if (!defs || uses->scrap < defs->scrap) {
                fputs("\\NWlink{weft", tex_file);
                write_scrap_ref(tex_file, uses->scrap, -1, &page);
                fputs("}{", tex_file);
                write_scrap_ref(tex_file, uses->scrap, TRUE, &page);
                fputs("}", tex_file);
                uses = uses->next;
            }
        else {
            if (defs->scrap == uses->scrap)
                uses = uses->next;
            fputs("\\underline{", tex_file);

            fputs("\\NWlink{weft", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("}{", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("}}", tex_file);
            page = -2;
            defs = defs->next;
        }
    }
    while (uses || defs) {
        if (uses && (!defs || uses->scrap < defs->scrap)) {
            fputs("\\NWlink{weft", tex_file);
            write_scrap_ref(tex_file, uses->scrap, -1, &page);
```

```

        fputs("{}{", tex_file);
        write_scrap_ref(tex_file, uses->scrap, FALSE, &page);
        fputs("}", tex_file);
        uses = uses->next;
    }
    else {
        if (uses && defs->scrap == uses->scrap)
            uses = uses->next;
        fputs(", \\underline{", tex_file);

        fputs("\\NWlink{weft", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("{}{", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("}", tex_file);

        putc('}', tex_file);
        page = -2;
        defs = defs->next;
    }
}
fputs(".\\n", tex_file);
}
}◊

```

Fragment referenced in 86a.

Uses: [dangling_flag 28](#), [defs 131c](#), [FALSE 23a](#), [fprintf 22](#), [fputs 22](#), [next 131b](#), [nw_char 29bc](#), [page 131c](#), [putc 22](#), [Scrap_Node 160a](#), [sector 131c](#), [TRUE 23a](#), [uses 131c](#), [write_scrap_ref 133a](#), [write_single_scrap_ref 133b](#).

2.5 Writing the Markdown File

This section implements the Markdown weave output, activated by the `-w md` command-line flag. It is structurally parallel to the L^AT_EX output in Section 2.4, but emits Markdown with embedded HTML for scraps (using `<pre><code>` blocks so that fragment references inside code can be clickable links).

⟨ Function prototypes 87 ⟩ ≡

```
extern void write_md(char *file_name, char *md_name, unsigned char sector);
```

◊

Fragment defined by [42a](#), [58b](#), [71a](#), [73b](#), [85a](#), [87](#), [122ab](#), [125a](#), [127b](#), [132b](#), [134b](#), [142b](#), [145b](#), [157b](#), [161c](#), [164b](#), [178d](#), [188c](#), [195a](#), [196c](#), [199b](#), [201c](#).

Fragment referenced in 21.

Uses: [file_name 131c](#), [sector 131c](#), [write_md 88b](#).

We need a few local function declarations before we get into the body of `write_md`.

"markdown.c" 88a≡

```
static void md_copy_scrap(FILE *file, int prefix, Name *name);
static void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps);
static void md_format_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_format_file_entry(Name *name, FILE *md_file);
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_write_arg(FILE *md_file, char *p);
static void md_write_literal(FILE *md_file, char *p);
static void md_write_ArglistElement(FILE *file, Arglist *args, char **params);
static void md_format_uses_refs(FILE *md_file, int scrap);
static void md_format_defs_refs(FILE *md_file, int scrap);
◇
```

File defined by 24c, 88ab, 93a, 96, 97a, 102, 103, 104b, 105b, 106a, 108b, 110a, 111b.

Uses: Arglist 179b, FILE 22, md_copy_scrap 97a, md_format_defs_refs 111b, md_format_entry 106a, md_format_file_entry 104b, md_format_user_entry 108b, md_format_uses_refs 110a, md_print_scrap_numbers 96, md_write_arg 93a, md_write_ArglistElement 103, md_write_literal 102, Name 160b, scraps 132a, Scrap_Node 160a, sector 131c.

The routine write_md takes two file names as parameters: the name of the web source file and the name of the .md output file.

"markdown.c" 88b≡

```
void write_md(char *file_name, char *md_name, unsigned char sector)
{
    (void)sector;
    FILE *md_file = fopen(md_name, "w");
    if (md_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", md_name);
        source_open(file_name);
        ⟨ Copy source_file into md_file 89a ⟩
        fclose(md_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, md_name);
}
◇
```

File defined by 24c, 88ab, 93a, 96, 97a, 102, 103, 104b, 105b, 106a, 108b, 110a, 111b.

Defines: write_md 41, 87.

Uses: command_name 30a, fclose 22, FILE 22, file_name 131c, fopen 22, fprintf 22, sector 131c, source_open 131a, stderr 22, verbose_flag 28.

We make our second pass through the source web, copying characters straight into the .md file. We watch for @ characters which signal a command sequence. We also detect and discard cosmetic fence lines (lines that are exactly ‘‘‘ optionally followed by a language tag).

```

< Copy source_file into md_file 89a> ≡
{
    int c = source_get();
    int at_line_start = TRUE;
    while (c != EOF) {
        if (c == nw_char)
        {
            < Interpret md at-sequence 90a>
        }
        else if (c == ''' && at_line_start) {
            < Check for cosmetic fence line 89b>
        }
        else {
            if (c == '\n') at_line_start = TRUE;
            else at_line_start = FALSE;
            putc(c, md_file);
            c = source_get();
        }
    }
}◊

```

Fragment referenced in 88b.

Uses: FALSE 23a, nw_char 29bc, putc 22, source_get 127a, TRUE 23a.

Cosmetic fence lines are lines that start with `'''`. We peek ahead to see if the line matches; if so, we skip to end of line. Otherwise we output what we already consumed and continue.

```

< Check for cosmetic fence line 89b> ≡
{
    int c2 = source_get();
    int c3 = (c2 == ''' ? source_get() : 0;
    if (c2 == ''' && c3 == ''' ) {
        /* Skip rest of fence line */
        int fc = source_get();
        while (fc != '\n' && fc != EOF)
            fc = source_get();
        c = source_get();
        at_line_start = TRUE;
    } else {
        /* Not a fence, output what we consumed */
        putc(''', md_file);
        if (c2 == ''' ) {
            putc(''', md_file);
            c = c3;
        } else {
            c = c2;
        }
        at_line_start = FALSE;
    }
}◊

```

Fragment referenced in 89a.

Uses: FALSE 23a, putc 22, source_get 127a, TRUE 23a.

⟨ Interpret md at-sequence 90a ⟩ ≡

```
{
    int big_definition = FALSE;
    c = source_get();
    switch (c) {
        case 'r':
            c = source_get();
            nw_char = c;
            update_delimit_scrap();
            break;
        case 'O': big_definition = TRUE;
            FALLTHROUGH;
        case 'o': ⟨ Write md output file definition 92a ⟩
            break;
        case 'Q':
        case 'D': big_definition = TRUE;
            FALLTHROUGH;
        case 'q':
        case 'd': ⟨ Write md macro definition 92b ⟩
            break;
        case 's':
            ⟨ Step to next sector 46a ⟩
            break;
        case 'S':
            ⟨ Close the current sector 46b ⟩
            break;
        case '{':
        case '[':
        case '(': ⟨ Write md in-text scrap 94a ⟩
            break;
        case '<': ⟨ Expand md macro into md_file 91b ⟩
            break;
        case 'x': ⟨ Copy label from source into (90b md_file ) 75b ⟩
            c = source_get();
            break;
        case 'c': /* Block comments: no special handling in Markdown */
            c = source_get();
            break;
        case 'f': ⟨ Write md index of file names 104a ⟩
            break;
        case 'm': ⟨ Write md index of macro names 105a ⟩
            break;
        case 'u': ⟨ Write md index of user-specified names 108a ⟩
            break;
        case 'v': ⟨ Copy version info into md file 91a ⟩
            break;
        case 'l':
        case 'L': ⟨ Skip at-L language definition line 44 ⟩
            break;
        case 'W': ⟨ Skip at-L language definition line 44 ⟩
            break;
        default:
            if (c==nw_char)
                putc(c, md_file);
            c = source_get();
            break;
    }
}
```

Fragment referenced in [89a](#).

```

< Copy version info into md file 91a> ≡
    fputs(version_string, md_file);
    c = source_get();
    ◇

```

Fragment referenced in 90a.

Uses: fputs 22, source_get 127a, version_string 28.

```

< Expand md macro into file 91b> ≡
{
    Parameters local_parameters = 0;
    int changed;
    char indent_chars[MAX_INDENT];
    Arglist *a;
    Name *name;
    Arglist * args;
    a = collect_scrap_name(0);
    name = a->name;
    args = instance(a->args, NULL, NULL, &changed);
    name->mark = TRUE;
    write_scraps(file, md_name, name->defs, 0, indent_chars, 0, 0, 1, 0, 0,
        args, name->arg, local_parameters, md_name);
    name->mark = FALSE;
    c = source_get();
}
◇

```

Fragment referenced in 90a.

Uses: Arglist 179b, collect_scrap_name 180, defs 131c, FALSE 23a, instance 145a, MAX_INDENT 122d, Name 160b, Parameters 54c, source_get 127a, TRUE 23a, write_scraps 147a.

2.5.1 Formatting Definitions in Markdown

File and fragment definitions use HTML anchors and formatting. A file definition header looks like:

```

<a id="weft1"></a>
**"filename" ** **1** =
<pre><code>...code...</code></pre>

```

A fragment definition header looks like:

```

<a id="weft1"></a>
**&lt;*Fragment Name* 1&gt;** =
<pre><code>...code...</code></pre>

```

⟨ Write md output file definition 92a ⟩ ≡

```
{
    Name *name = collect_file_name();
    (void)big_definition;
    fputs("\n<a id=\"weft\", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("></a>\n", md_file);
    fprintf(md_file, "**\"%s\"", name->spelling);
    fputs(" **", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("** =\n", md_file);
    ⟨ Fill in the md scrap 93b ⟩
    ⟨ Write md file cross-refs 94b ⟩
    ⟨ Finish md scrap 93c ⟩
}
```

Fragment referenced in 90a.

Uses: collect_file_name 172, fprintf 22, fputs 22, Name 160b, scraps 132a, write_single_scrap_ref 133b.

⟨ Write md macro definition 92b ⟩ ≡

```
{
    Name *name = collect_macro_name();
    (void)big_definition;
    fputs("\n<a id=\"weft\", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("></a>\n", md_file);
    fputs("**&lt;*", md_file);
    ⟨ Write the md macro's name 92c ⟩
    fputs("* ", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("&gt;** =\n", md_file);
    ⟨ Fill in the md scrap 93b ⟩
    ⟨ Write md macro cross-refs 94d ⟩
    ⟨ Finish md scrap 93c ⟩
}
```

Fragment referenced in 90a.

Uses: collect_macro_name 176, fputs 22, Name 160b, scraps 132a, write_single_scrap_ref 133b.

⟨ Write the md macro's name 92c ⟩ ≡

```
{
    char * p = name->spelling;
    int i = 0;

    while (*p != '\000') {
        if (*p == ARG_CHR) {
            md_write_arg(md_file, name->arg[i++]);
            p++;
        }
        else
            fputc(*p++, md_file);
    }
}
```

Fragment referenced in 92b, 106b.

Uses: ARG_CHR 177c, md_write_arg 93a.

```
"markdown.c" 93a≡
static void md_write_arg(FILE * md_file, char * p)
{
    fputs("<i>", md_file);
    while (*p)
    {
        switch (*p)
        {
            case '<':
                fputs("&lt;", md_file);
                break;
            case '>':
                fputs("&gt;", md_file);
                break;
            case '&':
                fputs("&amp;", md_file);
                break;
            default:
                fputc(*p, md_file);
                break;
        }
        p++;
    }
    fputs("</i>", md_file);
}
◇
```

File defined by [24c](#), [88ab](#), [93a](#), [96](#), [97a](#), [102](#), [103](#), [104b](#), [105b](#), [106a](#), [108b](#), [110a](#), [111b](#).
 Defines: [md_write_arg](#) [88a](#), [92c](#), [99a](#), [103](#).
 Uses: [FILE](#) [22](#), [fputs](#) [22](#).

```
< Fill in the md scrap 93b > ≡
{
    if (name->lang[0] != '\0')
        fprintf(md_file, "<pre><code class=\"language-%s\">", name->lang);
    else
        fputs("<pre><code>", md_file);
    extra_scraps = 0;
    md_copy_scrap(md_file, TRUE, name);
    fputs("</code></pre>\n", md_file);
}◇
```

Fragment referenced in [92ab](#).
 Uses: [extra_scraps](#) [67b](#), [fprintf](#) [22](#), [fputs](#) [22](#), [md_copy_scrap](#) [97a](#), [TRUE](#) [23a](#).

```
< Finish md scrap 93c > ≡
{
    scraps += extra_scraps;
    do
        c = source_get();
        while (isspace(c));
        at_line_start = TRUE;
    }◇
```

Fragment referenced in [92ab](#).
 Uses: [extra_scraps](#) [67b](#), [isspace](#) [22](#), [scraps](#) [132a](#), [source_get](#) [127a](#), [TRUE](#) [23a](#).

```

< Write md in-text scrap 94a > ≡
    md_copy_scrap(md_file, FALSE, NULL);
    c = source_get();
    ◇

```

Fragment referenced in [90a](#).

Uses: [FALSE 23a](#), [md_copy_scrap 97a](#), [source_get 127a](#).

Cross References in Markdown

Cross-references appear as blockquotes after each scrap.

```

< Write md file cross-refs 94b > ≡
{
    if ( scrap_flag ) {
        < Write md file defs 94c >
    }
    md_format_defs_refs(md_file, scraps);
    md_format_uses_refs(md_file, scraps++);
}◇

```

Fragment referenced in [92a](#).

Uses: [md_format_defs_refs 111b](#), [md_format_uses_refs 110a](#), [scraps 132a](#), [scrap_flag 28](#).

```

< Write md file defs 94c > ≡
{
    if (name->defs) {
        if (name->defs->next) {
            fputs("> *File defined by* ", md_file);
            md_print_scrap_numbers(md_file, name->defs);
        }
    } else {
        fprintf(stderr,
            "would have crashed in 'Write md file defs' for '%s'\n",
            name->spelling);
    }
}◇

```

Fragment referenced in [94b](#).

Uses: [defs 131c](#), [fprintf 22](#), [fputs 22](#), [md_print_scrap_numbers 96](#), [next 131b](#), [stderr 22](#).

```

< Write md macro cross-refs 94d > ≡
{
    < Write md macro defs 95a >
    < Write md macro refs 95b >
    md_format_defs_refs(md_file, scraps);
    md_format_uses_refs(md_file, scraps++);
}◇

```

Fragment referenced in [92b](#).

Uses: [md_format_defs_refs 111b](#), [md_format_uses_refs 110a](#), [scraps 132a](#).

⟨ *Write md macro defs* 95a ⟩ ≡

```
{
    if (name->defs->next) {
        fputs("> *Fragment defined by* ", md_file);
        md_print_scrap_numbers(md_file, name->defs);
    }
}
```

Fragment referenced in 94d.

Uses: [defs](#) 131c, [fputs](#) 22, [md_print_scrap_numbers](#) 96, [next](#) 131b.

⟨ *Write md macro refs* 95b ⟩ ≡

```
{
    if (name->uses) {
        if (name->uses->next) {
            fputs("> *Fragment referenced in* ", md_file);
            md_print_scrap_numbers(md_file, name->uses);
        }
        else {
            fputs("> *Fragment referenced in* ", md_file);
            fputs("<a href=\"#weft\"", md_file);
            write_single_scrap_ref(md_file, name->uses->scrap);
            fputs("\>", md_file);
            write_single_scrap_ref(md_file, name->uses->scrap);
            fputs("</a>", md_file);
            fputs(".\n", md_file);
        }
    }
    else {
        fputs("> *Fragment never referenced*.\\n", md_file);
        fprintf(stderr, "%s: <%s> never referenced.\\n",
            command_name, name->spelling);
    }
}
```

Fragment referenced in 94d.

Uses: [command_name](#) 30a, [fprintf](#) 22, [fputs](#) 22, [md_print_scrap_numbers](#) 96, [next](#) 131b, [stderr](#) 22, [uses](#) 131c, [write_single_scrap_ref](#) 133b.

```

"markdown.c" 96≡
static void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps)
{
    int page;
    fputs("<a href=\"#weft\", md_file);
    write_scrap_ref(md_file, scraps->scrap, -1, &page);
    fputs(">", md_file);
    write_scrap_ref(md_file, scraps->scrap, TRUE, &page);
    fputs("</a>", md_file);
    scraps = scraps->next;
    while (scraps) {
        fputs("<a href=\"#weft\", md_file);
        write_scrap_ref(md_file, scraps->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, scraps->scrap, FALSE, &page);
        fputs("</a>", md_file);
        scraps = scraps->next;
    }
    fputs(".\n", md_file);
}
◇

```

File defined by [24c](#), [88ab](#), [93a](#), [96](#), [97a](#), [102](#), [103](#), [104b](#), [105b](#), [106a](#), [108b](#), [110a](#), [111b](#).

Defines: `md_print_scrap_numbers` [88a](#), [94c](#), [95ab](#), [104d](#), [107b](#).

Uses: `FALSE` [23a](#), `FILE` [22](#), `fputs` [22](#), `next` [131b](#), `page` [131c](#), `scraps` [132a](#), `Scrap_Node` [160a](#), `TRUE` [23a](#), `write_scrap_ref` [133a](#).

Formatting a Scrap in Markdown

Scraps are output inside `<pre><code>` blocks. Characters `<`, `>`, and `&` are HTML-escaped. At-sequences within scraps (macro references, keywords, parameters, etc.) emit HTML markup directly.

```

"markdown.c" 97a≡
static int md_scrap_type = 0;

static void md_copy_scrap(FILE *file, int prefix, Name *name)
{
    int indent = 0;
    int c;
    char ** params = name ? name->arg : NULL;
    if (source_last == '{') md_scrap_type = 0;
    if (source_last == '[') md_scrap_type = 1;
    if (source_last == '(') md_scrap_type = 2;
    c = source_get();
    while (1) {
        switch (c) {
            case '\n': fputs("\n", file);
                        indent = 0;
                        break;
            case '\t': <Expand md tab into spaces 98>
                        break;
            default:
                if (c==nw_char)
                {
                    <Check md at-sequence for end-of-scrap 99a>
                    break;
                }
                if (md_scrap_type == 0 && prefix) {
                    <HTML-escape character c 97b>
                } else {
                    putc(c, file);
                }
                indent++;
                break;
        }
        c = source_get();
    }
}
◇

```

File defined by 24c, 88ab, 93a, 96, 97a, 102, 103, 104b, 105b, 106a, 108b, 110a, 111b.

Defines: md_copy_scrap 88a, 93b, 94a, md_scrap_type 99a.

Uses: FILE 22, fputs 22, Name 160b, nw_char 29bc, putc 22, source_get 127a, source_last 127a.

```

<HTML-escape character c 97b> ≡
switch (c) {
    case '<': fputs("&lt;", file); break;
    case '>': fputs("&gt;", file); break;
    case '&': fputs("&amp;", file); break;
    default: putc(c, file); break;
}◇

```

Fragment referenced in 97a.

Uses: fputs 22, putc 22.

```
< Expand md tab into spaces 98 > ≡  
  {  
    int delta = 8 - (indent % 8);  
    indent += delta;  
    while (delta > 0) {  
      putc(' ', file);  
      delta--;  
    }  
  }◇
```

Fragment referenced in [97a](#).

Uses: `putc` [22](#).

< Check md at-sequence for end-of-scrap 99a > ≡

```

{
  c = source_get();
  switch (c) {
    case 'c': < Show md presence of a block comment 100a >
      break;
    case 'x': < Copy label from source into (99b file ) 75b >
      break;
    case 'v': < Copy version info into file 75a >
    case 's':
      break;

    case '+':
    case '-':
    case '*':
    case '|': < Skip over index entries 75d >
    case ',':
    case ')':
    case ']':
    case '}': return;
    case '<': < Format md macro name 101 >
      break;
    case '%': < Skip commented-out code 75e >
      break;
    case '_': < Bold md Keyword 100b >
      break;
    case 't': < Italic md "fragment title" 100c >
      break;
    case 'f': < Italic md "file name" 100c >
      break;
    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9':
      if (name == NULL
          || name->arg[c - '1'] == NULL) {
        if (md_scrap_type == 0 && prefix)
          fputs("&", file);
        else
          putc(nw_char, file);
        fputc(c, file);
      }
      else {
        md_write_arg(file, name->arg[c - '1']);
      }
      break;
    default:
      if (c==nw_char)
      {
        if (md_scrap_type == 0 && prefix)
          fputs("&", file);
        else
          putc(nw_char, file);
        break;
      }
      /* ignore these since pass1 will have warned about them */
      break;
  }
}
}◊

```

Fragment referenced in [97a](#).

⟨ *Show md presence of a block comment* 100a ⟩ ≡

```
{  
    fputs("<i>(Comment)</i>", file);  
}  
◇
```

Fragment referenced in [99a](#).

Uses: [fputs](#) [22](#).

⟨ *Bold md Keyword* 100b ⟩ ≡

```
{  
    fprintf(file, "<b>");  
    c = source_get();  
    do {  
        if (c == '<') fputs("&lt;", file);  
        else if (c == '>') fputs("&gt;", file);  
        else if (c == '&') fputs("&amp;", file);  
        else fputc(c, file);  
        c = source_get();  
    } while (c != nw_char);  
    c = source_get();  
    fprintf(file, "</b>");  
}◇
```

Fragment referenced in [99a](#).

Uses: [fprintf](#) [22](#), [fputs](#) [22](#), [nw_char](#) [29bc](#), [source_get](#) [127a](#).

⟨ *Italic md "whatever"* 100c ⟩ ≡

```
{  
    fprintf(file, "<i>whatever</i>");  
}◇
```

Fragment referenced in [99a](#).

Uses: [fprintf](#) [22](#).

⟨ *Format md macro name 101* ⟩ ≡

```
{
    Arglist *args = collect_scrap_name(-1);
    Name *name = args->name;
    char * p = name->spelling;
    Arglist *q = args->args;
    int narg = 0;

    fputs("<<a href=\"#weft\"", file);
    if (name->defs) {
        write_single_scrap_ref(file, name->defs->scrap);
    } else {
        putc('?', file);
    }
    fputs("><i>", file);
    while (*p != '\000') {
        if (*p == ARG_CHR) {
            if (q == NULL) {
                md_write_literal(file, name->arg[narg]);
            }
            else {
                md_write_ArglistElement(file, q, params);
                q = q->next;
            }
            p++;
            narg++;
        }
        else {
            if (*p == '<') fputs("<", file);
            else if (*p == '>') fputs(">", file);
            else if (*p == '&') fputs("&", file);
            else fputc(*p, file);
            p++;
        }
    }
    fputs("</i> ", file);
    if (name->defs) {
        Scrap_Node *dp = name->defs;
        write_single_scrap_ref(file, dp->scrap);
        dp = dp->next;
        if (dp)
            fputs(", ...", file);
    }
    else {
        putc('?', file);
        fprintf(stderr, "%s: never defined <%s>\n",
            command_name, name->spelling);
    }
    fputs("</a>&gt;", file);
}◊
```

Fragment referenced in [99a](#).

Uses: [Arglist 179b](#), [ARG_CHR 177c](#), [collect_scrap_name 180](#), [command_name 30a](#), [defs 131c](#), [fprintf 22](#), [fputs 22](#), [md_write_ArglistElement 103](#), [md_write_literal 102](#), [Name 160b](#), [next 131b](#), [putc 22](#), [Scrap_Node 160a](#), [stderr 22](#), [write_single_scrap_ref 133b](#).

"markdown.c" 102≡

```
static void md_write_literal(FILE * md_file, char * p)
{
    while (*p!= '\000') {
        if (*p == '<') fputs("&lt;", md_file);
        else if (*p == '>') fputs("&gt;", md_file);
        else if (*p == '&') fputs("&amp;", md_file);
        else fputc(*p, md_file);
        p++;
    }
}
```

◇

File defined by [24c](#), [88ab](#), [93a](#), [96](#), [97a](#), [102](#), [103](#), [104b](#), [105b](#), [106a](#), [108b](#), [110a](#), [111b](#).

Defines: [md_write_literal](#) [88a](#), [101](#), [103](#).

Uses: [FILE](#) [22](#), [fputs](#) [22](#).

"markdown.c" 103≡

```
static void
md_write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        char * p = (char*)q;

        if (p[0] == ARG_CHR) {
            md_write_arg(file, params[p[1] - '1']);
        } else {
            md_write_literal(file, (char *)q);
        }
    } else if (name == (Name *)1) {
        Scrap_Node * qq = (Scrap_Node *)q;
        qq->quoted = TRUE;
        write_scraps(file, "", qq,
                     -1, "", 0, 0, 0, 0, 0,
                     NULL, params, 0, "");
        extra_scraps++;
        qq->quoted = FALSE;
    } else {
        char * p = name->spelling;
        fputs("<i>", file);
        while (*p != '\000') {
            if (*p == ARG_CHR) {
                md_write_ArglistElement(file, q, params);
                q = q->next;
                p++;
            }
            else {
                if (*p == '<') fputs("<", file);
                else if (*p == '>') fputs(">", file);
                else if (*p == '&') fputs("&", file);
                else fputc(*p, file);
                p++;
            }
        }
        fputs("</i> ", file);
        if (name->defs) {
            Scrap_Node *dp = name->defs;
            write_single_scrap_ref(file, dp->scrap);
            dp = dp->next;
            if (dp)
                fputs(", ...", file);
        }
        else {
            putc('?', file);
            fprintf(stderr, "%s: never defined <%s>\n",
                    command_name, name->spelling);
        }
        fputs(">", file);
    }
}
◇
```

File defined by [24c](#), [88ab](#), [93a](#), [96](#), [97a](#), [102](#), [103](#), [104b](#), [105b](#), [106a](#), [108b](#), [110a](#), [111b](#).

Defines: [md_write_ArglistElement](#) [88a](#), [101](#).

Uses: [Arglist](#) [179b](#), [ARG_CHR](#) [177c](#), [command_name](#) [30a](#), [defs](#) [131c](#), [extra_scraps](#) [67b](#), [FALSE](#) [23a](#), [FILE](#) [22](#), [fprintf](#) [22](#), [fputs](#) [22](#), [md_write_arg](#) [93a](#), [md_write_literal](#) [102](#), [Name](#) [160b](#), [next](#) [131b](#), [putc](#) [22](#), [Scrap_Node](#) [160a](#), [stderr](#) [22](#), [TRUE](#) [23a](#), [write_scraps](#) [147a](#), [write_single_scrap_ref](#) [1043b](#).

2.5.2 Generating the Indices in Markdown

Write md index of file names 104a ≡

```
{
    if (file_names) {
        fputs("\n", md_file);
        md_format_file_entry(file_names, md_file);
        fputs("\n", md_file);
    }
    c = source_get();
}◇
```

Fragment referenced in 90a.

Uses: file_names 161a, fputs 22, md_format_file_entry 104b, source_get 127a.

"markdown.c" 104b≡

```
static void md_format_file_entry(Name *name, FILE *md_file)
{
    while (name) {
        md_format_file_entry(name->llink, md_file);
        Format a md file index entry 104c
        name = name->rlink;
    }
}
◇
```

File defined by 24c, 88ab, 93a, 96, 97a, 102, 103, 104b, 105b, 106a, 108b, 110a, 111b.

Defines: md_format_file_entry 88a, 104a.

Uses: FILE 22, Name 160b.

Format a md file index entry 104c ≡

```
fputs("- ", md_file);
fprintf(md_file, "'\"%s\"' ", name->spelling);
Write md file's defining scrap numbers 104d
putc('\n', md_file);◇
```

Fragment referenced in 104b.

Uses: fprintf 22, fputs 22, putc 22.

Write md file's defining scrap numbers 104d ≡

```
{
    Scrap_Node *p = name->defs;
    fputs("*Defined by* ", md_file);
    if (p->next) {
        md_print_scrap_numbers(md_file, p);
    }
    else {
        fputs("<a href=\"#weft\"", md_file);
        write_single_scrap_ref(md_file, p->scrap);
        fputs(">", md_file);
        write_single_scrap_ref(md_file, p->scrap);
        fputs("</a>.", md_file);
    }
}◇
```

Fragment referenced in 104c.

Uses: defs 131c, fputs 22, md_print_scrap_numbers 96, next 131b, Scrap_Node 160a, write_single_scrap_ref 133b.

⟨ *Write md index of macro names* 105a ⟩ ≡

```
{
    unsigned char sector = current_sector;
    int c = source_get();
    if (c == '+')
        sector = 0;
    else
        source_ungetc(&c);
    if (has_sector(macro_names, sector)) {
        fputs("\n", md_file);
        md_format_entry(macro_names, md_file, sector);
        fputs("\n", md_file);
    } else {
        fputs("None.\n", md_file);
    }
}
c = source_get();
◇
```

Fragment referenced in 90a.

Uses: `current_sector` 46c, `fputs` 22, `has_sector` 85b, `macro_names` 161a, `md_format_entry` 106a, `sector` 131c, `source_get` 127a, `source_ungetc` 127c.

"markdown.c" 105b ≡

```
static int md_load_entry(Name * name, Name ** nms, int n)
{
    while (name) {
        n = md_load_entry(name->llink, nms, n);
        nms[n++] = name;
        name = name->rlink;
    }
    return n;
}
◇
```

File defined by 24c, 88ab, 93a, 96, 97a, 102, 103, 104b, 105b, 106a, 108b, 110a, 111b.

Defines: `md_load_entry` 106a.

Uses: `Name` 160b.

```

"markdown.c" 106a≡
static void md_format_entry(Name *name, FILE *md_file, unsigned char sector)
{
    Name ** nms = malloc(num_scraps()*sizeof(Name *));
    int n = md_load_entry(name, nms, 0);
    int i;

    ⟨ Sort nms of size n for ⟨ Rob's ordering 82c ⟩ 83a ⟩
    for (i = 0; i < n; i++)
    {
        Name * name = nms[i];

        ⟨ Format a md index entry 106b ⟩
    }
    free(nms);
}
◇

```

File defined by 24c, 88ab, 93a, 96, 97a, 102, 103, 104b, 105b, 106a, 108b, 110a, 111b.

Defines: md_format_entry 88a, 105a.

Uses: FILE 22, malloc 22, md_load_entry 105b, Name 160b, num_scraps 132a, sector 131c.

```

⟨ Format a md index entry 106b ⟩ ≡
    if (name->sector == sector){
        fputs("-", md_file);
        fputs("<*", md_file);
        ⟨ Write the md macro's name 92c ⟩
        fputs("* ", md_file);
        ⟨ Write md defining scrap numbers 107a ⟩
        fputs("> ", md_file);
        ⟨ Write md referencing scrap numbers 107b ⟩
        putc('\n', md_file);
    }◇

```

Fragment referenced in 106a.

Uses: fputs 22, putc 22, sector 131c.

⟨ Write md defining scrap numbers 107a ⟩ ≡

```
{
    Scrap_Node *p = name->defs;
    if (p) {
        int page;
        fputs("<a href=\"#weft\", md_file);
        write_scrap_ref(md_file, p->scrap, -1, &page);
        fputs("\>", md_file);
        write_scrap_ref(md_file, p->scrap, TRUE, &page);
        fputs("</a>", md_file);
        p = p->next;
        while (p) {
            fputs("<a href=\"#weft\", md_file);
            write_scrap_ref(md_file, p->scrap, -1, &page);
            fputs("\>", md_file);
            write_scrap_ref(md_file, p->scrap, FALSE, &page);
            fputs("</a>", md_file);
            p = p->next;
        }
    }
    else
        putc('?', md_file);
}◊
```

Fragment referenced in 106b.

Uses: defs 131c, FALSE 23a, fputs 22, next 131b, page 131c, putc 22, Scrap_Node 160a, TRUE 23a, write_scrap_ref 133a.

⟨ Write md referencing scrap numbers 107b ⟩ ≡

```
{
    Scrap_Node *p = name->uses;
    if (p) {
        fputs("*Referenced in* ", md_file);
        if (p->next) {
            md_print_scrap_numbers(md_file, p);
        }
        else {
            fputs("<a href=\"#weft\", md_file);
            write_single_scrap_ref(md_file, p->scrap);
            fputs("\>", md_file);
            write_single_scrap_ref(md_file, p->scrap);
            fputs("</a>.", md_file);
        }
    }
    else
        fputs("*Not referenced*.", md_file);
}◊
```

Fragment referenced in 106b.

Uses: fputs 22, md_print_scrap_numbers 96, next 131b, Scrap_Node 160a, uses 131c, write_single_scrap_ref 133b.

⟨ Write md index of user-specified names 108a ⟩ ≡

```
{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    if (has_sector(user_names, sector)) {
        fputs("\n", md_file);
        md_format_user_entry(user_names, md_file, sector);
        fputs("\n", md_file);
    }
}
}◊
```

Fragment referenced in 90a.

Uses: current_sector 46c, fputs 22, has_sector 85b, md_format_user_entry 108b, sector 131c, source_get 127a, user_names 161a.

"markdown.c" 108b≡

```
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector)
{
    while (name) {
        md_format_user_entry(name->llink, md_file, sector);
        ⟨ Format a md user index entry 108c ⟩
        name = name->rlink;
    }
}
◊
```

File defined by 24c, 88ab, 93a, 96, 97a, 102, 103, 104b, 105b, 106a, 108b, 110a, 111b.

Defines: md_format_user_entry 88a, 108a.

Uses: FILE 22, Name 160b, sector 131c.

⟨ Format a md user index entry 108c ⟩ ≡

```
if (name->sector == sector){
    Scrap_Node *uses = name->uses;
    if ( uses || dangling_flag ) {
        int page;
        Scrap_Node *defs = name->defs;
        fprintf(md_file, "- '%s': ", name->spelling);
        if (!uses) {
            fputs("(**", md_file);
            fputs("<a href=\"#weft\"", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs("\>", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs("</a>(**", md_file);
            page = -2;
            defs = defs->next;
        }
        else
            if (!defs || uses->scrap < defs->scrap) {
                fputs("<a href=\"#weft\"", md_file);
                write_scrap_ref(md_file, uses->scrap, -1, &page);
                fputs("\>", md_file);
                write_scrap_ref(md_file, uses->scrap, TRUE, &page);
```

```

        fputs("</a>", md_file);
        uses = uses->next;
    }
    else {
        if (defs->scrap == uses->scrap)
            uses = uses->next;
        fputs("**", md_file);

        fputs("<a href=\"#weft", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs(">", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("</a>**", md_file);
        page = -2;
        defs = defs->next;
    }
    while (uses || defs) {
        if (uses && (!defs || uses->scrap < defs->scrap)) {
            fputs("<a href=\"#weft", md_file);
            write_scrap_ref(md_file, uses->scrap, -1, &page);
            fputs(">", md_file);
            write_scrap_ref(md_file, uses->scrap, FALSE, &page);
            fputs("</a>", md_file);
            uses = uses->next;
        }
        else {
            if (uses && defs->scrap == uses->scrap)
                uses = uses->next;
            fputs(", **", md_file);

            fputs("<a href=\"#weft", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs(">", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs("</a>**", md_file);
            page = -2;
            defs = defs->next;
        }
    }
    fputs(".\n", md_file);
}
}◊

```

Fragment referenced in [108b](#).

Uses: [dangling_flag 28](#), [defs 131c](#), [FALSE 23a](#), [fprintf 22](#), [fputs 22](#), [next 131b](#), [page 131c](#), [Scrap_Node 160a](#), [sector 131c](#), [TRUE 23a](#), [uses 131c](#), [write_scrap_ref 133a](#), [write_single_scrap_ref 133b](#).

2.5.3 Identifier Cross-References in Markdown

These functions are the Markdown equivalents of `format_uses_refs` and `format_defs_refs` from `scraps.c`, but emit Markdown blockquote format.

"markdown.c" 110a≡

```
static void
md_format_uses_refs(FILE * md_file, int scrap)
{
    Uses * p = get_scrap_uses(scrap);
    if (p != NULL)
        ⟨ Write md uses references 110b ⟩
}
◇
```

File defined by 24c, 88ab, 93a, 96, 97a, 102, 103, 104b, 105b, 106a, 108b, 110a, 111b.
Defines: md_format_uses_refs 88a, 94bd.
Uses: FILE 22, get_scrap_uses 133c, Uses 194c.

⟨ Write md uses references 110b ⟩ ≡

```
{
    char join = ' ';
    fputs("> *Uses:* ", md_file);
    do {
        ⟨ Write one md use reference 110c ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".\n", md_file);
}◇
```

Fragment referenced in 110a.
Uses: fputs 22, next 131b, Uses 194c.

⟨ Write one md use reference 110c ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c '%s' ",
        join, name->spelling);
if (defs)
{
    do {
        ⟨ Write one md referenced scrap 111a ⟩
        first = FALSE;
        defs = defs->next;
    }while (defs!= NULL);
}
else
{
    fputs("(not defined globally)*", md_file);
}
◇
```

Fragment referenced in 110b.
Uses: defs 131c, FALSE 23a, first 202a, fprintf 22, fputs 22, Name 160b, next 131b, page 131c, Scrap_Node 160a, TRUE 23a.

```

< Write one md referenced scrap 111a> ≡
    fputs("<a href=\"#weft\", md_file);
    write_scrap_ref(md_file, defs->scrap, -1, &page);
    fputs("\>", md_file);
    write_scrap_ref(md_file, defs->scrap, first, &page);
    fputs("</a>", md_file);◇

```

Fragment referenced in [110c](#), [112](#).

Uses: [defs 131c](#), [first 202a](#), [fputs 22](#), [page 131c](#), [write_scrap_ref 133a](#).

"markdown.c" 111b≡

```

static void
md_format_defs_refs(FILE * md_file, int scrap)
{
    Uses * p = get_scrap_defs(scrap);
    if (p != NULL)
        < Write md defs references 111c>
}
◇

```

File defined by [24c](#), [88ab](#), [93a](#), [96](#), [97a](#), [102](#), [103](#), [104b](#), [105b](#), [106a](#), [108b](#), [110a](#), [111b](#).

Defines: [md_format_defs_refs 88a](#), [94bd](#).

Uses: [FILE 22](#), [get_scrap_defs 134a](#), [Uses 194c](#).

```

< Write md defs references 111c> ≡
{
    char join = ' ';
    fputs("> *Defines:", md_file);
    do {
        < Write one md def reference 112>
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".\n", md_file);
}◇

```

Fragment referenced in [111b](#).

Uses: [fputs 22](#), [next 131b](#).

```

< Write one md def reference 112> ≡
Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c '%s' ",
        join, name->spelling);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
    fputs("*Never used*", md_file);
}
else {
    do {
        if (defs->scrap != scrap) {
            < Write one md referenced scrap 111a>
            first = FALSE;
        }
        defs = defs->next;
    }while (defs!= NULL);
}
◇

```

Fragment referenced in 111c.

Uses: defs 131c, FALSE 23a, first 202a, fprintf 22, fputs 22, Name 160b, next 131b, page 131c, Scrap_Node 160a, TRUE 23a, uses 131c.

2.6 Map and Extract Output

Two modes support AI-assisted navigation of weft projects:

- m (map) emits a JSON object to stdout describing every output file and every fragment, their definition locations, usage relationships, and transitive dependencies.
- e *name* (extract) emits the scrap code and dependency information for a single fragment (matched by prefix) as Markdown to stdout.

Both modes run after `pass1` has built the name trees and do *not* invoke the weave or tangle phases.

2.6.1 JSON Map Output

The `write_map` function performs an in-order traversal of the `file_names` and `macro_names` binary search trees, emitting a JSON object with two arrays: "files" and "fragments".

For each `Name` node, it records:

- The spelling (name of the file or fragment).
- The sector number.
- The language tag (if any).
- Each definition scrap with its source file and line.
- Each usage scrap (scraps that reference this name).
- For definition scraps, the list of fragments referenced inside (via `get_scrap_uses`).

A helper function escapes strings for JSON output.

```

"output.c" 113≡
static void json_puts(FILE *f, const char *s)
{
    fputc('"', f);
    while (*s) {
        switch (*s) {
            case '"': fputs("\\\"", f); break;
            case '\\': fputs("\\\\", f); break;
            case '\n': fputs("\\n", f); break;
            case '\r': fputs("\\r", f); break;
            case '\t': fputs("\\t", f); break;
            default:
                if ((unsigned char)*s < 0x20)
                    fprintf(f, "\\u%04x", (unsigned char)*s);
                else
                    fputc(*s, f);
        }
        s++;
    }
    fputc('"', f);
}
◇

```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

Defines: [json_puts](#) [114](#), [121](#).

Uses: FILE [22](#), [fprintf](#) [22](#), [fputs](#) [22](#).

The `emit_name_json` helper emits a single name entry as a JSON object. The `is_file` flag distinguishes file names (which use `"uses_fragments"` for fragments referenced inside their definition scraps) from macro fragments (which use `"used_by"` for the scraps that invoke them, and `"uses"` for fragments they invoke).

"output.c" 114≡

```
static void emit_name_json(FILE *f, Name *name, int is_file, int *first)
{
    Scrap_Node *sn;
    int first_inner;

    if (!name) return;
    emit_name_json(f, name->llink, is_file, first);

    if (!*first) fputs("\n", f);
    *first = 0;

    fputs("    {\n", f);
    fputs("        \"name\": ", f); json_puts(f, name->spelling);
    if (name->lang[0]) {
        fputs(",\n        \"lang\": ", f); json_puts(f, name->lang);
    }
    fprintf(f, ",\n        \"sector\": %d", name->sector);

    /* defs array */
    fputs(",\n        \"defs\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
        if (!first_inner) fputs(" ", f);
        first_inner = 0;
        fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
        json_puts(f, scrap_file_name(sn->scrap));
        fprintf(f, ", \"line\": %d}", scrap_file_line(sn->scrap));
    }
    fputs("]", f);

    if (is_file) {
        /* uses_fragments: fragments referenced inside this file's scraps */
        fputs(",\n        \"uses_fragments\": [", f);
        first_inner = 1;
        for (sn = name->defs; sn; sn = sn->next) {
            Uses *u;
            for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
                if (!first_inner) fputs(" ", f);
                first_inner = 0;
                json_puts(f, u->defn->spelling);
            }
        }
        fputs("]", f);
    } else {
        /* used_by: scraps that invoke this fragment */
        fputs(",\n        \"used_by\": [", f);
        first_inner = 1;
        for (sn = name->uses; sn; sn = sn->next) {
            if (!first_inner) fputs(" ", f);
            first_inner = 0;
            fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
            json_puts(f, scrap_file_name(sn->scrap));
            fprintf(f, ", \"line\": %d}", scrap_file_line(sn->scrap));
        }
        fputs("]", f);

        /* uses: fragments referenced inside this fragment's def scraps */
        fputs(",\n        \"uses\": [", f);
        first_inner = 1;
        for (sn = name->defs; sn; sn = sn->next) {
            Uses *u;
            for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
                if (!first_inner) fputs(" ", f);
                first_inner = 0;
                json_puts(f, u->defn->spelling);
            }
        }
        fputs("]", f);
    }
}
```

```

"output.c" 115a≡
void write_map(Name *files, Name *macros)
{
    int first;

    fputs("{\n  \"files\": [\n", stdout);
    first = 1;
    emit_name_json(stdout, files, 1, &first);
    fputs("\n ],\n  \"fragments\": [\n", stdout);
    first = 1;
    emit_name_json(stdout, macros, 0, &first);
    fputs("\n ]\n}\n", stdout);
}
◇

```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

Defines: [write_map](#) [41](#), [122b](#).

Uses: [emit_name_json](#) [114](#), [first](#) [202a](#), [fputs](#) [22](#), [Name](#) [160b](#).

2.6.2 Fragment Extraction

The `write_extract` function finds a fragment by name (using prefix matching, like `@<name...@>` in the source), then emits the scrap code and dependency information as Markdown. It recurses into referenced fragments to provide the full transitive closure of the requested fragment.

To find a name in the BST without creating a new entry, we use a read-only search function:

```

"output.c" 115b≡
static Name *find_name(Name *node, const char *spelling)
{
    Name *found;
    if (!node) return NULL;
    if (robs_strcmp(node->spelling, (char *)spelling) == 0)
        return node;
    found = find_name(node->llink, spelling);
    if (found) return found;
    return find_name(node->rlink, spelling);
}
◇

```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

Defines: [find_name](#) [117b](#).

Uses: [Name](#) [160b](#), [robs_strcmp](#) [165a](#).

A recursive helper emits one fragment and all its transitive dependencies. A visited array (indexed by Name pointer cast to a simple mark) prevents infinite recursion on cyclic references.

```

"output.c" 116≡
static void extract_fragment(FILE *f, Name *name, Name *files, Name *macros)
{
    Scrap_Node *sn;

    if (!name || name->mark) return;
    name->mark = TRUE;

    for (sn = name->defs; sn; sn = sn->next) {
        fprintf(f, "\n## \"%s\" --- %s:%d\n",
            name->spelling,
            scrap_file_name(sn->scrap),
            scrap_file_line(sn->scrap));
        if (name->lang[0])
            fprintf(f, "' '%s\n", name->lang);
        else
            fputs("' '\n", f);
        dump_scrap_text(f, sn->scrap);
        fputs("\n' '\n", f);

        /* Show dependencies */
        {
            Uses *u = get_scrap_uses(sn->scrap);
            if (u) {
                fputs("\n### Uses\n", f);
                for (; u; u = u->next)
                    fprintf(f, "- %s\n", u->defn->spelling);
            }
        }

        /* Show used_by */
        if (name->uses) {
            Scrap_Node *usn;
            fputs("\n### Used by\n", f);
            for (usn = name->uses; usn; usn = usn->next) {
                fprintf(f, "- scrap %d (%s:%d)\n",
                    usn->scrap,
                    scrap_file_name(usn->scrap),
                    scrap_file_line(usn->scrap));
            }
        }

        /* Recurse into dependencies */
        for (sn = name->defs; sn; sn = sn->next) {
            Uses *u;
            for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
                extract_fragment(f, u->defn, files, macros);
            }
        }
    }
}

```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

Defines: [extract_fragment 117b](#).

Uses: [defs 131c](#), [dump_scrap_text 143](#), [FILE 22](#), [fprintf 22](#), [fputs 22](#), [get_scrap_uses 133c](#), [Name 160b](#), [next 131b](#), [scrap_file_line 134d](#), [scrap_file_name 134c](#), [Scrap_Node 160a](#), [TRUE 23a](#), [Uses 194c](#), [uses 131c](#).

Clear marks after extraction so the name trees remain usable:

```
"output.c" 117a≡
static void clear_marks(Name *node)
{
    if (!node) return;
    clear_marks(node->llink);
    node->mark = FALSE;
    clear_marks(node->rlink);
}
◇
```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

Defines: `clear_marks` [117b](#).

Uses: `FALSE` [23a](#), `Name` [160b](#).

```
"output.c" 117b≡
void write_extract(char *name, Name *files, Name *macros)
{
    Name *found = NULL;

    /* Search in macro_names first, then file_names */
    found = find_name(macros, name);
    if (!found)
        found = find_name(files, name);

    if (!found) {
        fprintf(stderr, "%s: fragment \"%s\" not found\n", command_name, name);
        exit(1);
    }

    fprintf(stdout, "# Extract: \"%s\"\n", found->spelling);
    extract_fragment(stdout, found, files, macros);

    /* Clean up marks */
    clear_marks(files);
    clear_marks(macros);
}
◇
```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

Defines: `write_extract` [41](#), [122b](#).

Uses: `clear_marks` [117a](#), `command_name` [30a](#), `exit` [22](#), `extract_fragment` [116](#), `file_names` [161a](#), `find_name` [115b](#), `first` [202a](#), `fprintf` [22](#), `macro_names` [161a](#), `Name` [160b](#), `stderr` [22](#).

2.6.3 Reverse Map Output

The reverse map feature provides programmatic navigation from tangled output back to the `.weft` source. Given a tangled output file (and optionally a line number), it parses the section markers embedded in that file and emits JSON mapping each region to its originating scrap, source file, and source line.

This operates entirely on the tangled output—no `.weft` files are needed and `pass1` is never invoked. The section markers have the form:

<code>COMMENT_BEGIN{N: file:line}</code>	<code>COMMENT_END</code>	(opening)
<code>COMMENT_BEGIN{:N}</code>	<code>COMMENT_END</code>	(closing)

where `COMMENT_BEGIN` and `COMMENT_END` vary by language (`/* ... */`, `// ...`, `# ...`, etc.).

A `MarkerFrame` records the data extracted from one opening marker. We maintain a stack of these to handle nested fragment expansions.

```
"output.c" 118≡
    typedef struct {
        int scrap;
        char file[FILENAME_MAX];
        int line;
        int start_line; /* line in the tangled file where this marker opened */
    } MarkerFrame;
    ◇
```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).
Defines: `MarkerFrame` [119](#), [121](#).

The `parse_open_marker` function scans a line for the pattern `{N: file:line}`. It returns 1 on success, filling in the supplied `MarkerFrame`. The scan is intentionally loose about what precedes the opening brace so that any comment style works.

"output.c" 119≡

```
static int parse_open_marker(const char *line, MarkerFrame *mf)
{
    const char *p = line;
    while (*p) {
        if (*p == '{') {
            const char *start = p + 1;
            /* scrap number */
            if (*start < '0' || *start > '9') { p++; continue; }
            int scrap = 0;
            while (*start >= '0' && *start <= '9')
                scrap = scrap * 10 + (*start++ - '0');
            if (*start != ':' || *(start+1) != ' ') { p++; continue; }
            start += 2; /* skip ": " */
            /* file:line | find the closing brace, then work backwards for :line */
            const char *end = strchr(start, '}');
            if (!end) { p++; continue; }
            /* find last ':' between start and end */
            const char *colon = NULL;
            for (const char *q = end - 1; q >= start; q--) {
                if (*q == ':') { colon = q; break; }
            }
            if (!colon) { p++; continue; }
            /* digits after colon? */
            int src_line = 0;
            const char *d = colon + 1;
            if (d == end) { p++; continue; }
            while (d < end) {
                if (*d < '0' || *d > '9') break;
                src_line = src_line * 10 + (*d++ - '0');
            }
            if (d != end) { p++; continue; }
            /* success */
            mf->scrap = scrap;
            mf->line = src_line;
            {
                size_t len = (size_t)(colon - start);
                if (len >= FILENAME_MAX) len = FILENAME_MAX - 1;
                memcpy(mf->file, start, len);
                mf->file[len] = '\0';
            }
            return 1;
        }
        p++;
    }
    return 0;
}
◇
```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

Defines: `parse_open_marker` [121](#).

Uses: `MarkerFrame` [118](#), `size_t` [22](#).

The `parse_close_marker` function scans for `{:N}` and returns the scrap number, or `-1` on failure.

```

"output.c" 120≡
static int parse_close_marker(const char *line)
{
    const char *p = line;
    while (*p) {
        if (*p == '{' && *(p+1) == ':') {
            const char *start = p + 2;
            if (*start < '0' || *start > '9') { p++; continue; }
            int scrap = 0;
            while (*start >= '0' && *start <= '9')
                scrap = scrap * 10 + (*start++ - '0');
            if (*start == '}')
                return scrap;
        }
        p++;
    }
    return -1;
}
◇

```

File defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).
Defines: `parse_close_marker` [121](#).

The main function `write_reverse_map` parses its argument to split an optional line number from the file-name, opens the tangled file, and reads it line by line. A stack of **MarkerFrame** entries tracks nested scrap regions.

In *line mode* (a specific line was requested), as soon as `current_line` reaches the target the innermost stack frame is emitted and the function returns.

In *full mode* (no line number), every time a close marker is encountered the region from the matching open to the current line is emitted.

"output.c" 121≡

```
void write_reverse_map(const char *arg)
{
    char filename[FILENAME_MAX];
    int target_line = -1;
    FILE *fp;
    char buf[8192];
    MarkerFrame stack[256];
    int depth = 0;
    int current_line = 0;
    int first_region = 1;

    /* Parse arg: find last ':' where suffix is all digits */
    {
        const char *colon = NULL;
        const char *p;
        for (p = arg + strlen(arg) - 1; p > arg; p--) {
            if (*p == ':') { colon = p; break; }
        }
        if (colon) {
            const char *d = colon + 1;
            int all_digits = (*d != '\0');
            while (*d) {
                if (*d < '0' || *d > '9') { all_digits = 0; break; }
                d++;
            }
            if (all_digits) {
                target_line = atoi(colon + 1);
                snprintf(filename, FILENAME_MAX, "%.s", (int)(colon - arg), arg);
            } else {
                snprintf(filename, FILENAME_MAX, "%s", arg);
            }
        } else {
            snprintf(filename, FILENAME_MAX, "%s", arg);
        }
    }

    fp = fopen(filename, "r");
    if (!fp) {
        fprintf(stderr, "weft: cannot open %s: ", filename);
        perror(NULL);
        exit(1);
    }

    if (target_line < 0) {
        /* Full mode header */
        fputs("{\"source\": ", stdout);
        json_puts(stdout, filename);
        fputs(", \"regions\": [\n", stdout);
    }

    while (fgets(buf, sizeof(buf), fp)) {
        MarkerFrame mf;
        int close_scrap;
        current_line++;

        if (parse_open_marker(buf, &mf)) {
            mf.start_line = current_line;
            if (depth < 256)
                stack[depth++] = mf;
            continue;
        }

        close_scrap = parse_close_marker(buf);
        if (close_scrap >= 0) {
            /* Find matching frame on stack */
            int i;
            for (i = depth - 1; i >= 0; i--) {
                if (stack[i].scrap == close_scrap) {
                    if (target_line < 0) {
                        /* Full mode: emit region */
                    }
                }
            }
        }
    }
}
```

2.7 Writing the Output Files

< Function prototypes 122a > $\equiv$
extern void write_files(Name *files);
 $\diamond$

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: Name 160b, write_files 122c.

< Function prototypes 122b > $\equiv$
extern void write_map(Name *file_names, Name *macro_names);
extern void write_extract(char *name, Name *file_names, Name *macro_names);
extern void write_reverse_map(const char *arg);
 $\diamond$

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: file_names 161a, macro_names 161a, Name 160b, write_extract 117b, write_map 115a, write_reverse_map 121.

"output.c" 122c $\equiv$
void write_files(Name *files)
{
 while (files) {
 write_files(files->llink);
 < Write out files->spelling 123a >
 files = files->rlink;
 }
}
 $\diamond$

File defined by 24d, 113, 114, 115ab, 116, 117ab, 118, 119, 120, 121, 122c.

Defines: write_files 41, 122a.

Uses: Name 160b.

MAX_INDENT defines the maximum number of characters tracked per line for indentation purposes. When a fragment is expanded mid-line, the characters preceding it are recorded so that subsequent lines of the fragment can reproduce the same indentation. Languages like HTML, XML, and SVG routinely contain lines longer than 500 characters, so we set this to 8192. If a line exceeds this limit, indentation beyond the recorded portion falls back to spaces instead of causing a fatal error.

< Type declarations 122d > $\equiv$

#define MAX_INDENT 8192
 $\diamond$

Fragment defined by 23a, 54c, 122d, 160ab, 177c, 179b, 182d, 194c, 201a.

Fragment referenced in 21.

Defines: MAX_INDENT 62b, 91b, 123a, 150a, 151b.

```

< Write out files->spelling 123a > ≡
{
    static char temp_name[FILENAME_MAX];
    static char real_name[FILENAME_MAX];
    static int temp_name_count = 0;
    char indent_chars[MAX_INDENT];
    FILE *temp_file;

    < Find a free temporary file 123b >

    sprintf(real_name, "%s%s%s", dirpath, path_sep, files->spelling);
    if (verbose_flag)
        fprintf(stderr, "writing %s [%s]\n", files->spelling, temp_name);
    write_scraps(temp_file, files->spelling, files->defs, 0, indent_chars,
                files->debug_flag, files->tab_flag, files->indent_flag,
                files->comment_flag, files->location_flag,
                NULL, NULL, 0, files->spelling);
    fclose(temp_file);

    < Move the temporary file to the target, if required 124a >
}◊

```

Fragment referenced in [122c](#).

Uses: [defs 131c](#), [fclose 22](#), [FILE 22](#), [fprintf 22](#), [MAX_INDENT 122d](#), [stderr 22](#), [verbose_flag 28](#), [write_scraps 147a](#).

```

< Find a free temporary file 123b > ≡
for( temp_name_count = 0; temp_name_count < 10000; temp_name_count++) {
    sprintf(temp_name, "%s%snw%06d", dirpath, path_sep, temp_name_count);
#ifdef O_EXCL
    if (-1 != (temp_file_fd = open(temp_name, O_CREAT|O_WRONLY|O_EXCL))) {
        temp_file = fdopen(temp_file_fd, "w");
        break;
    }
#else
    if (0 != (temp_file = fopen(temp_name, "a"))) {
        if ( 0L == ftell(temp_file)) {
            break;
        } else {
            fclose(temp_file);
            temp_file = 0;
        }
    }
#endif
}
if (!temp_file) {
    fprintf(stderr, "%s: can't create %s for a temporary file\n",
            command_name, temp_name);
    exit(-1);
}
◊

```

Fragment referenced in [123a](#).

Uses: [command_name 30a](#), [exit 22](#), [fclose 22](#), [fopen 22](#), [fprintf 22](#), [stderr 22](#).

Note the call to `remove` before `rename` – the ANSI/ISO C standard does *not* guarantee that renaming a file to an existing filename will overwrite the file.

```

⟨ Move the temporary file to the target, if required 124a ⟩ ≡
    if (compare_flag)
        ⟨ Compare the temp file and the old file 124b ⟩
    else {
        remove(real_name);
        ⟨ Rename the temporary file to the target 124c ⟩
    }
    ◇

```

Fragment referenced in 123a.
 Uses: compare_flag 28, remove 22.

Again, we use a call to `remove` before `rename`.

```

⟨ Compare the temp file and the old file 124b ⟩ ≡
{
    FILE *old_file = fopen(real_name, "r");
    if (old_file) {
        int x, y;
        temp_file = fopen(temp_name, "r");
        do {
            x = getc(old_file);
            y = getc(temp_file);
        } while (x == y && x != EOF);
        fclose(old_file);
        fclose(temp_file);
        if (x == y)
            remove(temp_name);
        else {
            remove(real_name);
            ⟨ Rename the temporary file to the target 124c ⟩
        }
    }
    else
        ⟨ Rename the temporary file to the target 124c ⟩
}
    ◇

```

Fragment referenced in 124a.
 Uses: fclose 22, FILE 22, fopen 22, getc 22, remove 22.

```

⟨ Rename the temporary file to the target 124c ⟩ ≡
    if (0 != rename(temp_name, real_name)) {
        fprintf(stderr, "%s: can't rename output file to %s\n",
            command_name, real_name);
    }
    ◇

```

Fragment referenced in 124ab.
 Uses: command_name 30a, fprintf 22, stderr 22.

Chapter 3

The Support Routines

3.1 Source Files

3.1.1 Global Declarations

We need two routines to handle reading the source files.

```
< Function prototypes 125a > ≡
extern void source_open(char *name); /* pass in the name of the source file */
extern int source_get(void); /* no args; returns the next char or EOF */
extern int source_last; /* what last source_get() returned. */
extern int source_peek; /* The next character to get */
◇
```

Fragment defined by [42a](#), [58b](#), [71a](#), [73b](#), [85a](#), [87](#), [122ab](#), [125a](#), [127b](#), [132b](#), [134b](#), [142b](#), [145b](#), [157b](#), [161c](#), [164b](#), [178d](#), [188c](#), [195a](#), [196c](#), [199b](#), [201c](#).

Fragment referenced in [21](#).

Uses: [next](#) [131b](#), [source_get](#) [127a](#), [source_last](#) [127a](#), [source_open](#) [131a](#), [source_peek](#) [127a](#).

There are also two global variables maintained for use in error messages and such.

```
< Global variable declarations 125b > ≡
extern char *source_name; /* name of the current file */
extern int source_line; /* current line in the source file */
◇
```

Fragment defined by [28](#), [29b](#), [30a](#), [46c](#), [51c](#), [67b](#), [125b](#), [135a](#), [161a](#), [201b](#).

Fragment referenced in [21](#).

Defines: [source_line](#) [45](#), [47c](#), [48a](#), [125c](#), [127ac](#), [128](#), [129](#), [130ab](#), [131a](#), [136c](#), [138](#), [140](#), [163b](#), [170](#), [171](#), [172](#), [173ab](#), [176](#), [178b](#), [180](#), [181](#), [source_name](#) [38](#), [39](#), [40](#), [41](#), [45](#), [47c](#), [48a](#), [125c](#), [128](#), [129](#), [130ab](#), [131a](#), [136c](#), [138](#), [140](#), [163b](#), [170](#), [171](#), [172](#), [173ab](#), [176](#), [177a](#), [178bc](#), [180](#), [181](#).

```
< Global variable definitions 125c > ≡
char *source_name = NULL;
int source_line = 0;
◇
```

Fragment defined by [29ac](#), [30b](#), [46d](#), [51d](#), [67c](#), [125c](#), [135b](#), [161b](#).

Fragment referenced in [25e](#).

Uses: [source_line](#) [125b](#), [source_name](#) [125b](#).

3.1.2 Local Declarations

```
"input.c" 126a≡
    static FILE *source_file; /* the current input file */
    static int double_at;
    static int include_depth;
    ◇
```

File defined by 25a, 126ab, 127ac, 131a.

Defines: `double_at` 128, 131a, `include_depth` 129, 130b, 131a, `source_file` 127ac, 128, 129, 130ab, 131a.

Uses: FILE 22.

```
"input.c" 126b≡
    static struct {
        FILE *file;
        char *name;
        int line;
    } stack[10];
    ◇
```

File defined by 25a, 126ab, 127ac, 131a.

Defines: `stack` 121, 129, 130b.

Uses: FILE 22.

3.1.3 Reading a File

The routine `source_get` returns the next character from the current source file. It notices newlines and keeps the line counter `source_line` up to date. It also catches EOF and watches for @ characters. All other characters are immediately returned. We define `source_last` to let us tell which type of scrap we are defining.

"input.c" 127a≡

```
int source_peek;
int source_last;
int source_get(void)
{
    int c;
    source_last = c = source_peek;
    switch (c) {
        case EOF:  ⟨ Handle EOF 130b ⟩
            return c;
        case '\n': source_line++;
            FALLTHROUGH;
        default:
            if (c==nw_char)
            {
                ⟨ Handle an "at" character 128 ⟩
                return c;
            }
            source_peek = getc(source_file);
            return c;
    }
}
```

File defined by 25a, 126ab, 127ac, 131a.

Defines: `source_get` 42c, 43a, 44, 45, 46ab, 47c, 48a, 49ab, 50a, 51ab, 56, 58a, 60c, 61a, 62ab, 67ad, 72, 74a, 75cde, 76a, 80a, 81b, 85c, 89ab, 90a, 91ab, 93c, 94a, 97a, 99a, 100b, 104a, 105a, 108a, 125a, 129, 130b, 137, 138, 139b, 140, 141c, 170, 171, 172, 173ab, 176, 177ab, 178c, 180, 181, 182a, `source_last` 58a, 72, 97a, 125a, `source_peek` 49a, 50a, 51a, 125a, 127c, 128, 129, 130b, 131a.

Uses: `FALLTHROUGH` 23a, `getc` 22, `nw_char` 29bc, `source_file` 126a, `source_line` 125b.

`source_ungetc` pushes a read character back to the `source_file`.

⟨ Function prototypes 127b ⟩ ≡
`extern void source_ungetc(int*);`
◇

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: `source_ungetc` 127c.

"input.c" 127c≡

```
void source_ungetc(int *c)
{
    ungetc(source_peek, source_file);
    if(*c == '\n')
        source_line--;
    source_peek=*c;
}
```

File defined by 25a, 126ab, 127ac, 131a.

Defines: `source_ungetc` 49b, 50a, 51b, 81b, 105a, 127b.

Uses: `source_file` 126a, `source_line` 125b, `source_peek` 127a.

This whole @ character handling mess is pretty annoying. I want to recognize @i so I can handle include files correctly. At the same time, it makes sense to recognize illegal @ sequences and complain; this avoids

ever having to check anywhere else. Unfortunately, I need to avoid tripping over the @@ sequence; hence this whole unsatisfactory `double_at` business.

```

⟨ Handle an "at" character 128 ⟩ ≡
{
  c = getc(source_file);
  if (double_at) {
    source_peek = c;
    double_at = FALSE;
    c = nw_char;
  }
  else
    switch (c) {
      case 'i': ⟨ Open an include file 129 ⟩
        break;
      case '#': case 'f': case 'm': case 'u': case 'v':
      case 'd': case 'o': case 'D': case 'O': case 's':
      case 'q': case 'Q': case 'S': case 'l': case 'L': case 't':
      case 'W':
      case '+':
      case '-':
      case '*':
      case '\\':
      case '{': case '}': case '<': case '>': case '|':
      case '(': case ')': case '[': case ']':
      case '%': case '_':
      case '.': case ',': case 'x': case 'c':
      case '1': case '2': case '3': case '4': case '5':
      case '6': case '7': case '8': case '9':
      case 'r':
        source_peek = c;
        c = nw_char;
        break;
      default:
        if (c==nw_char)
        {
          source_peek = c;
          double_at = TRUE;
          break;
        }
        fprintf(stderr, "%s: bad %c sequence %c[%d] (%s, line %d)\n",
                  command_name, nw_char, c, c, source_name, source_line);
        exit(-1);
    }
}
}◊

```

Fragment referenced in [127a](#).

Uses: `command_name` [30a](#), `double_at` [126a](#), `exit` [22](#), `FALSE` [23a](#), `fprintf` [22](#), `getc` [22](#), `nw_char` [29bc](#), `source_file` [126a](#), `source_line` [125b](#), `source_name` [125b](#), `source_peek` [127a](#), `stderr` [22](#), `TRUE` [23a](#).

< Open an include file 129 > ≡

```
{
    char name[FILENAME_MAX];
    char fullname[FILENAME_MAX];
    struct incl * p = include_list;

    if (include_depth >= 10) {
        fprintf(stderr, "%s: include nesting too deep (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
    < Collect include-file name 130a >
    stack[include_depth].file = source_file;
    fullname[0] = '\0';
    for (;;) {
        strcat(fullname, name);
        source_file = fopen(fullname, "r");
        if (source_file || !p)
            break;
        strcpy(fullname, p->name);
        strcat(fullname, "/");
        p = p->next;
    }
    if (!source_file) {
        fprintf(stderr, "%s: can't open include file %s\n",
            command_name, name);
        source_file = stack[include_depth].file;
    }
    else
    {
        stack[include_depth].name = source_name;
        stack[include_depth].line = source_line + 1;
        include_depth++;
        source_line = 1;
        source_name = save_string(fullname);
    }
    source_peek = getc(source_file);
    c = source_get();
}◇
```

Fragment referenced in [128](#).

Uses: [command_name 30a](#), [exit 22](#), [fopen 22](#), [fprintf 22](#), [getc 22](#), [incl 28](#), [include_depth 126a](#), [next 131b](#), [save_string 162b](#), [source_file 126a](#), [source_get 127a](#), [source_line 125b](#), [source_name 125b](#), [source_peek 127a](#), [stack 126b](#), [stderr 22](#).

⟨ *Collect include-file name 130a* ⟩ ≡

```
{
    char *p = name;
    do
        c = getc(source_file);
    while (c == ' ' || c == '\t');
    while (isgraph(c)) {
        *p++ = c;
        c = getc(source_file);
    }
    *p = '\0';
    if (c != '\n') {
        fprintf(stderr, "%s: unexpected characters after file name (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
}
}◊
```

Fragment referenced in [129](#).

Uses: [command_name 30a](#), [exit 22](#), [fprintf 22](#), [getc 22](#), [isgraph 22](#), [source_file 126a](#), [source_line 125b](#), [source_name 125b](#), [stderr 22](#).

If an EOF is discovered, the current file must be closed and input from the next stacked file must be resumed. If no more files are on the stack, the EOF is returned.

⟨ *Handle EOF 130b* ⟩ ≡

```
{
    fclose(source_file);
    if (include_depth) {
        include_depth--;
        source_file = stack[include_depth].file;
        source_line = stack[include_depth].line;
        source_name = stack[include_depth].name;
        source_peek = getc(source_file);
        c = source_get();
    }
}
}◊
```

Fragment referenced in [127a](#).

Uses: [fclose 22](#), [getc 22](#), [include_depth 126a](#), [source_file 126a](#), [source_get 127a](#), [source_line 125b](#), [source_name 125b](#), [source_peek 127a](#), [stack 126b](#).

3.1.4 Opening a File

The routine `source_open` takes a file name and tries to open the file. If unsuccessful, it complains and halts. Otherwise, it sets `source_name`, `source_line`, and `double_at`.

```
"input.c" 131a≡
void source_open(char *name)
{
    source_file = fopen(name, "r");
    if (!source_file) {
        fprintf(stderr, "%s: couldn't open %s\n", command_name, name);
        exit(-1);
    }
    nw_char = '@';
    source_name = name;
    source_line = 1;
    source_peek = getc(source_file);
    double_at = FALSE;
    include_depth = 0;
}
◇
```

File defined by 25a, 126ab, 127ac, 131a.

Defines: `source_open` 42b, 59b, 88b, 125a.

Uses: `command_name` 30a, `double_at` 126a, `exit` 22, `FALSE` 23a, `fopen` 22, `fprintf` 22, `getc` 22, `include_depth` 126a, `nw_char` 29bc, `source_file` 126a, `source_line` 125b, `source_name` 125b, `source_peek` 127a, `stderr` 22.

3.2 Scraps

```
"scraps.c" 131b≡
#define SLAB_SIZE 1024

typedef struct slab {
    struct slab *next;
    char chars[SLAB_SIZE];
} Slab;
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: `next` 28, 32, 34, 35b, 40, 47b, 54d, 68cd, 69ab, 77, 78, 79, 81a, 84ab, 86b, 94c, 95ab, 96, 101, 103, 104d, 107ab, 108c, 110bc, 111c, 112, 114, 116, 125a, 129, 135d, 141ab, 142ac, 144b, 145ac, 146a, 147a, 156a, 157a, 160a, 179bc, 182c, 183abd, 184abc, 185a, 187a, 190, 191, 192, 194abc, 195c, 196a, 197ab, 201d, 202c, 203a, 204, `Slab` 131c, 135cd, 136c, 142c, `SLAB_SIZE` 135d, 142c, 144a, 188b.

Uses: `slab` 131c.

"scraps.c" 131c≡

```
typedef struct {
    char *file_name;
    Slab *slab;
    struct uses *uses;
    struct uses *defs;
    int file_line;
    int page;
    char letter;
    unsigned char sector;
} ScrapEntry;
```

◇

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: **defs** 47b, 54d, 62b, 68cd, 77, 78, 79, 81a, 84a, 86b, 91b, 94c, 95a, 101, 103, 104d, 107a, 108c, 110c, 111a, 112, 114, 116, 123a, 132b, 134a, 136c, 141b, 145c, 147a, 149, 150bcd, 153, 154, 155ac, 156ac, 160b, 167, 182d, 183c, 192, 196abd, 197b, **file_line** 134d, 136c, 137, 149, **file_name** 42ab, 58b, 59b, 87, 88b, 134c, 136c, 137, 150bc, **letter** 133a, 136c, 159d, **page** 69b, 84a, 86b, 96, 107a, 108c, 110c, 111a, 112, 132b, 133ab, 136c, 158ab, 159bd, 196ab, 197b, 204, **ScrapEntry** 132ac, 136c, **sector** 58b, 59ab, 81b, 82b, 83b, 85bc, 86ab, 87, 88ab, 105a, 106ab, 108abc, 114, 136c, 140, 141abd, 143, 154, 157a, 160b, 161c, 163ab, 164a, 166, 167, 176, 178b, 180, 192, **slab** 131b, 136c, 143, 149, 192, **uses** 32, 69a, 84b, 86b, 95b, 107b, 108c, 112, 114, 116, 133c, 136c, 141a, 142a, 160b, 167, 168, 183c, 192, 194c, 195b, 197b.

Uses: **Slab** 131b.

There's some issue with reading the auxiliary file if it has more than 255 lines, so (as a workaround only) increase the size of the array of **ScrapEntry**s held in **SCRAP[n]**.

"scraps.c" 132a≡

```
#define SCRAP_BITS 10
#define SCRAP_SIZE (1<<SCRAP_BITS)
#define SCRAP_MASK (SCRAP_SIZE - 1)
#define SCRAP_SHIFT SCRAP_BITS
static ScrapEntry *SCRAP[SCRAP_SIZE];

#define scrap_array(i) SCRAP[(i) >> SCRAP_SHIFT][(i) & SCRAP_MASK]

static int scraps;
int num_scraps(void)
{
    return scraps;
}
< Forward declarations for scraps.c 147b, ... >
```

◇

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: **num_scraps** 82b, 106a, 132b, **SCRAP** 132c, 136c, **scraps** 24bc, 28, 58a, 59a, 63, 64a, 65b, 67a, 69b, 88a, 92ab, 93c, 94bd, 96, 114, 132c, 136c, 158ab, 159bc, 192, 204, **scrap_array** 133ac, 134acd, 136c, 137, 141ab, 143, 149, 150bc, 158ab, 159bd, 192, 195b, 196d, **SCRAP_BITS** Never used, **SCRAP_MASK** 136c, **SCRAP_SHIFT** 136c, **SCRAP_SIZE** 132c, 136c.

Uses: **ScrapEntry** 131c.

⟨ *Function prototypes 132b* ⟩ ≡

```
extern void init_scraps(void);
extern int collect_scrap(void);
extern int write_scraps(FILE *file, char *spelling, Scrap_Node *defs, int global_indent, char *indent_char);
extern void write_scrap_ref(FILE *file, int num, int first, int *page);
extern void write_single_scrap_ref(FILE *file, int num);
extern int num_scraps(void);
extern Uses * get_scrap_uses(int scrap);
extern Uses * get_scrap_defs(int scrap);
◇
```

Fragment defined by [42a](#), [58b](#), [71a](#), [73b](#), [85a](#), [87](#), [122ab](#), [125a](#), [127b](#), [132b](#), [134b](#), [142b](#), [145b](#), [157b](#), [161c](#), [164b](#), [178d](#), [188c](#), [195a](#), [196c](#), [199b](#), [201c](#).

Fragment referenced in [21](#).

Uses: Arglist [179b](#), collect_scrap [136b](#), defs [131c](#), FILE [22](#), first [202a](#), get_scrap_defs [134a](#), get_scrap_uses [133c](#), init_scraps [132c](#), num_scraps [132a](#), page [131c](#), Parameters [54c](#), Scrap_Node [160a](#), Uses [194c](#), write_scraps [147a](#), write_scrap_ref [133a](#), write_single_scrap_ref [133b](#).

"scraps.c" [132c](#)≡

```
void init_scraps(void)
{
    scraps = 1;
    SCRAP[0] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(ScrapEntry));
}
◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: init_scraps [42b](#), [132b](#).

Uses: arena_getmem [202b](#), SCRAP [132a](#), ScrapEntry [131c](#), scraps [132a](#), SCRAP_SIZE [132a](#).

"scraps.c" [133a](#)≡

```
void write_scrap_ref(FILE *file, int num, int first, int *page)
{
    if (scrap_array(num).page >= 0) {
        if (first!=0)
            fprintf(file, "%d", scrap_array(num).page);
        else if (scrap_array(num).page != *page)
            fprintf(file, ", %d", scrap_array(num).page);
        if (scrap_array(num).letter > 0)
            fputc(scrap_array(num).letter, file);
    }
    else {
        if (first!=0)
            putc('?', file);
        else
            fputs(" ?", file);
        ⟨ Warn (only once) about needing to rerun after Latex 134e ⟩
    }
    if (first>=0)
        *page = scrap_array(num).page;
}
◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: write_scrap_ref [69b](#), [84a](#), [86b](#), [96](#), [107a](#), [108c](#), [111a](#), [132b](#), [133b](#), [196b](#).

Uses: FILE [22](#), first [202a](#), fprintf [22](#), fputs [22](#), letter [131c](#), page [131c](#), putc [22](#), scrap_array [132a](#).

```
"scraps.c" 133b≡
void write_single_scrap_ref(FILE *file, int num)
{
    int page;
    write_scrap_ref(file, num, TRUE, &page);
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: write_single_scrap_ref 58a, 63, 64a, 69a, 78, 81a, 84b, 86b, 92ab, 95b, 101, 103, 104d, 107b, 108c, 132b, 155ac, 199c.

Uses: FILE 22, page 131c, TRUE 23a, write_scrap_ref 133a.

```
"scraps.c" 133c≡
Uses * get_scrap_uses(int scrap)
{
    return scrap_array(scrap).uses;
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: get_scrap_uses 110a, 114, 116, 132b.

Uses: scrap_array 132a, Uses 194c, uses 131c.

```
"scraps.c" 134a≡
Uses * get_scrap_defs(int scrap)
{
    return scrap_array(scrap).defs;
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: get_scrap_defs 111b, 132b.

Uses: defs 131c, scrap_array 132a, Uses 194c.

These accessors expose scrap metadata that is needed by the `--map` and `--extract` modes (Section 2.6) without leaking the internal `ScrapEntry` or `Slab` types.

```
< Function prototypes 134b > ≡
extern const char *scrap_file_name(int i);
extern int scrap_file_line(int i);
extern void dump_scrap_text(FILE *file, int scrap_idx);
◇
```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: dump_scrap_text 143, FILE 22, scrap_file_line 134d, scrap_file_name 134c.

```
"scraps.c" 134c≡
const char *scrap_file_name(int i) { return scrap_array(i).file_name; }
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: scrap_file_name 114, 116, 134b.

Uses: file_name 131c, scrap_array 132a.

```
"scraps.c" 134d≡
    int scrap_file_line(int i) { return scrap_array(i).file_line; }
    ◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: `scrap_file_line` [114](#), [116](#), [134b](#).

Uses: `file_line` [131c](#), `scrap_array` [132a](#).

```
< Warn (only once) about needing to rerun after Latex 134e > ≡
{
    if (!already_warned) {
        fprintf(stderr, "%s: you'll need to rerun weft after running latex\n",
            command_name);
        already_warned = TRUE;
    }
}◇
```

Fragment referenced in [133a](#), [158b](#).

Uses: `already_warned` [135a](#), `command_name` [30a](#), `fprintf` [22](#), `stderr` [22](#), `TRUE` [23a](#).

```
< Global variable declarations 135a > ≡
    extern int already_warned;
    ◇
```

Fragment defined by [28](#), [29b](#), [30a](#), [46c](#), [51c](#), [67b](#), [125b](#), [135a](#), [161a](#), [201b](#).

Fragment referenced in [21](#).

Defines: `already_warned` [134e](#), [135b](#), [158a](#).

```
< Global variable definitions 135b > ≡
    int already_warned = 0;
    ◇
```

Fragment defined by [29ac](#), [30b](#), [46d](#), [51d](#), [67c](#), [125c](#), [135b](#), [161b](#).

Fragment referenced in [25e](#).

Uses: `already_warned` [135a](#).

```
"scraps.c" 135c≡
    typedef struct {
        Slab *scrap;
        Slab *prev;
        int index;
    } Manager;
    ◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: `Manager` [135d](#), [136ab](#), [142c](#), [143](#), [144a](#), [146b](#), [149](#), [185c](#), [187c](#), [192](#).

Uses: `Slab` [131b](#).

```
"scraps.c" 135d≡
static void push(char c, Manager *manager)
{
    Slab *scrap = manager->scrap;
    int index = manager->index;
    scrap->chars[index++] = c;
    if (index == SLAB_SIZE) {
        Slab *new_slab = (Slab *) arena_getmem(sizeof(Slab));
        scrap->next = new_slab;
        manager->scrap = new_slab;
        index = 0;
    }
    manager->index = index;
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: push 52b, 56, 136a, 137, 138, 139a, 141cd.

Uses: arena_getmem 202b, Manager 135c, next 131b, Slab 131b, SLAB_SIZE 131b.

```
"scraps.c" 136a≡
static void pushes(char *s, Manager *manager)
{
    while (*s)
        push(*s++, manager);
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: pushes 56, 139a, 141d.

Uses: Manager 135c, push 135d.

```
"scraps.c" 136b≡
int collect_scrap(void)
{
    int current_scrap, lblseq = 0;
    int depth = 1;
    Manager writer;
    ⟨ Create new scrap, managed by writer 136c ⟩
    ⟨ Accumulate scrap and return scraps++ 137 ⟩
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: collect_scrap 46e, 47a, 56, 132b, 182c.

Uses: Manager 135c.

⟨ Create new scrap, managed by writer 136c ⟩ ≡

```
{
    Slab *scrap = (Slab *) arena_getmem(sizeof(Slab));
    if ((scraps & SCRAP_MASK) == 0)
        SCRAP[scraps >> SCRAP_SHIFT] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(ScrapEntry));
    scrap_array(scraps).slab = scrap;
    scrap_array(scraps).file_name = save_string(source_name);
    scrap_array(scraps).file_line = source_line;
    scrap_array(scraps).page = -1;
    scrap_array(scraps).letter = 0;
    scrap_array(scraps).uses = NULL;
    scrap_array(scraps).defs = NULL;
    scrap_array(scraps).sector = current_sector;
    writer.scrap = scrap;
    writer.index = 0;
    current_scrap = scraps++;
}◊
```

Fragment referenced in 136b.

Uses: arena_getmem 202b, current_sector 46c, defs 131c, file_line 131c, file_name 131c, letter 131c, page 131c, save_string 162b, SCRAP 132a, ScrapEntry 131c, scraps 132a, scrap_array 132a, SCRAP_MASK 132a, SCRAP_SHIFT 132a, SCRAP_SIZE 132a, sector 131c, Slab 131b, slab 131c, source_line 125b, source_name 125b, uses 131c.

⟨ Accumulate scrap and return scraps++ 137 ⟩ ≡

```
{
    int c = source_get();
    while (1) {
        switch (c) {
            case EOF: fprintf(stderr, "%s: unexpect EOF in (%s, %d)\n",
                                command_name, scrap_array(current_scrap).file_name,
                                scrap_array(current_scrap).file_line);
                        exit(-1);
            default:
                if (c==nw_char)
                {
                    ⟨ Handle at-sign during scrap accumulation 138 ⟩
                    break;
                }
                push(c, &writer);
                c = source_get();
                break;
        }
    }
}◊
```

Fragment referenced in 136b.

Uses: command_name 30a, exit 22, file_line 131c, file_name 131c, fprintf 22, nw_char 29bc, push 135d, scrap_array 132a, source_get 127a, stderr 22.

< Handle at-sign during scrap accumulation 138 > ≡

```
{
    c = source_get();
    switch (c) {
        case '(':
        case '[':
        case '{': depth++;
                break;

        case '+':
        case '-':
        case '*':
        case '|': < Collect user-specified index entries 140 >
                /* Fall through */

        case ')':
        case ']':
        case '}': if (--depth > 0)
                break;
                /* else fall through */

        case ',':
                push('\0', &writer);
                scrap_ended_with = c;
                return current_scrap;

        case '<': < Handle macro invocation in scrap 141c >
                break;

        case '%': < Skip commented-out code 75e >
                /* emit line break to the output file to keep #line in sync. */
                push('\n', &writer);
                c = source_get();
                break;

        case 'x': < Get label while collecting scrap 139a >
                break;

        case 'c': < Include block comment in a scrap 52b >
                break;

        case '1': case '2': case '3':
        case '4': case '5': case '6':
        case '7': case '8': case '9':
        case 'f': case '#': case 'v':
        case 't': case 's':
                push(nw_char, &writer);
                break;

        case '_': c = source_get();
                break;

        default :
                if (c==nw_char)
                {
                        push(nw_char, &writer);
                        push(nw_char, &writer);
                        c = source_get();
                        break;
                }
                fprintf(stderr, "%s: unexpected %c%c in scrap (%s, %d)\n",
                        command_name, nw_char, c, source_name, source_line);
                exit(-1);
    }
}
```

Fragment referenced in [137](#).

Uses: [command_name 30a](#), [exit 22](#), [fprintf 22](#), [nw_char 29bc](#), [push 135d](#), [source_get 127a](#), [source_line 125b](#), [source_name 125b](#), [stderr 22](#).

```

⟨ Get label while collecting scrap 139a ⟩ ≡
{
  ⟨ Get label from (139b source_get() ) 198c ⟩
  ⟨ Save label to label store 199e ⟩
  push(nw_char, &writer);
  push('x', &writer);
  pushs(label_name, &writer);
  push(nw_char, &writer);
}◊

```

Fragment referenced in [138](#).

Uses: `source_get` [127a](#).

< Collect user-specified index entries 140 > ≡

```

{
    do {
        int type = c;
        do {
            char new_name[MAX_NAME_LEN];
            char *p = new_name;
            unsigned int sector = 0;
            do
                c = source_get();
            while (isspace(c));
            if (c != nw_char) {
                Name *name;
                do {
                    *p++ = c;
                    c = source_get();
                } while (c != nw_char && !isspace(c));
                *p = '\0';
                switch (type) {
                    case '*':
                        sector = current_sector;
                        < Add user identifier use 141a >
                        break;
                    case '-':
                        < Add user identifier use 141a >
                        /* Fall through */
                    case '|':
                        sector = current_sector;
                        /* Fall through */
                    case '+':
                        < Add user identifier definition 141b >
                        break;
                }
            }
        } while (c != nw_char);
        c = source_get();
    } while (c == '|' || c == '*' || c == '-' || c == '+');
    if (c != '}' && c != ']' && c != ')') {
        fprintf(stderr, "%s: unexpected %c%c in index entry (%s, %d)\n",
            command_name, nw_char, c, source_name, source_line);
        exit(-1);
    }
}
}◊

```

Fragment referenced in [138](#).

Uses: [command_name 30a](#), [current_sector 46c](#), [exit 22](#), [fprintf 22](#), [isspace 22](#), [MAX_NAME_LEN 23b](#), [Name 160b](#), [nw_char 29bc](#), [sector 131c](#), [source_get 127a](#), [source_line 125b](#), [source_name 125b](#), [stderr 22](#).

⟨ Add user identifier use 141a ⟩ ≡

```
name = name_add(&user_names, new_name, sector);
if (!name->uses || name->uses->scrap != current_scrap) {
    Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    use->scrap = current_scrap;
    use->next = name->uses;
    name->uses = use;
    add_uses(&(scrap_array(current_scrap).uses), name);
}◊
```

Fragment referenced in 140.

Uses: add_uses 194b, arena_getmem 202b, name_add 166, next 131b, scrap_array 132a, Scrap_Node 160a, sector 131c, user_names 161a, uses 131c.

⟨ Add user identifier definition 141b ⟩ ≡

```
name = name_add(&user_names, new_name, sector);
if (!name->defs || name->defs->scrap != current_scrap) {
    Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    def->scrap = current_scrap;
    def->next = name->defs;
    name->defs = def;
    add_uses(&(scrap_array(current_scrap).defs), name);
}◊
```

Fragment referenced in 140.

Uses: add_uses 194b, arena_getmem 202b, defs 131c, name_add 166, next 131b, scrap_array 132a, Scrap_Node 160a, sector 131c, user_names 161a.

⟨ Handle macro invocation in scrap 141c ⟩ ≡

```
{
    Arglist *args = collect_scrap_name(current_scrap);
    Name *name = args->name;
    ⟨ Save macro name 141d ⟩
    add_to_use(name, current_scrap);
    if (scrap_name_has_parameters) {
        ⟨ Save macro parameters 56 ⟩
    }
    push(nw_char, &writer);
    push('>', &writer);
    c = source_get();
}◊
```

Fragment referenced in 138.

Uses: add_to_use 142a, Arglist 179b, collect_scrap_name 180, Name 160b, nw_char 29bc, push 135d, source_get 127a.

⟨ Save macro name 141d ⟩ ≡

```
{
    char buff[24];

    push(nw_char, &writer);
    push('<', &writer);
    push(name->sector, &writer);
    sprintf(buff, "%d", arglist_register(args));
    pushs(buff, &writer);
}◊
```

Fragment referenced in 141c.

Uses: arglist_register 148, nw_char 29bc, push 135d, pushs 136a, sector 131c.

"scraps.c" 142a≡

```
void
add_to_use(Name * name, int current_scrap)
{
    if (!name->uses || name->uses->scrap != current_scrap) {
        Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
        use->scrap = current_scrap;
        use->next = name->uses;
        name->uses = use;
    }
}
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: add_to_use 56, 141c, 142b, 183a.

Uses: arena_getmem 202b, Name 160b, next 131b, Scrap_Node 160a, uses 131c.

⟨ *Function prototypes* 142b ⟩ ≡

```
extern void add_to_use(Name * name, int current_scrap);
```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: add_to_use 142a, Name 160b.

"scraps.c" 142c≡

```
static char pop(Manager *manager)
{
    Slab *scrap = manager->scrap;
    int index = manager->index;
    char c = scrap->chars[index++];
    if (index == SLAB_SIZE) {
        manager->prev = scrap;
        manager->scrap = scrap->next;
        index = 0;
    }
    manager->index = index;
    return c;
}
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: pop 53abd, 57, 143, 146bc, 149, 151a, 152ac, 186b.

Uses: Manager 135c, next 131b, Slab 131b, SLAB_SIZE 131b.

The function `dump_scrap_text` walks the slab chain for a given scrap and writes every printable character (plus newlines and tabs) to the supplied file handle. The internal meta-character sequences that encode fragment references are decoded back to their `@<name@>` form; other control sequences are silently skipped.

```

"scraps.c" 143≡
void dump_scrap_text(FILE *file, int scrap_idx)
{
    Manager reader;
    char c;
    reader.scrap = scrap_array(scrap_idx).slab;
    reader.prev = NULL;
    reader.index = 0;
    c = pop(&reader);
    while (c) {
        if (c == nw_char) {
            /* handle internal control sequences */
            c = pop(&reader);
            if (c == '<') {
                /* fragment reference encoded as: sector_byte + decimal_index
                   followed by nw_char + '>' */
                char idx_buf[MAX_NAME_LEN];
                char *p = idx_buf;
                int idx;
                (void)pop(&reader); /* sector byte, discard */
                c = pop(&reader);
                while (c != nw_char && c != '\0') {
                    *p++ = c;
                    c = pop(&reader);
                }
                *p = '\0';
                if (sscanf(idx_buf, "%d", &idx) == 1) {
                    Arglist *a = arglist_lookup(idx);
                    if (a && a->name) {
                        fprintf(file, "%c<%s%c>", nw_char, a->name->spelling, nw_char);
                    }
                }
                /* skip past closing '>' marker */
                if (c == nw_char)
                    c = pop(&reader);
                /* skip optional parameter block */
                if (c == '(') {
                    while (c && c != nw_char)
                        c = pop(&reader);
                    if (c == nw_char)
                        c = pop(&reader);
                }
            } else {
                /* other control seq: skip */
            }
            c = pop(&reader);
            continue;
        }
        fputc(c, file);
        c = pop(&reader);
    }
}
◇

```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: [dump_scrap_text](#) [116](#), [134b](#).

Uses: [Arglist](#) [179b](#), [arglist_lookup](#) [148](#), [FILE](#) [22](#), [fprintf](#) [22](#), [Manager](#) [135c](#), [MAX_NAME_LEN](#) [23b](#), [nw_char](#) [29bc](#), [pop](#) [142c](#), [scrap_array](#) [132a](#), [sector](#) [131c](#), [slab](#) [131c](#).

```
"scraps.c" 144a≡
static void backup(int n, Manager *manager)
{
    int index = manager->index;
    if (n > index
        && manager->prev != NULL)
    {
        manager->scrap = manager->prev;
        manager->prev = NULL;
        index += SLAB_SIZE;
    }
    manager->index = (n <= index ? index - n : 0);
}
◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: [backup](#) [151a](#).

Uses: [Manager](#) [135c](#), [SLAB_SIZE](#) [131b](#).

```
"scraps.c" 144b≡
void
lookup(int n, Arglist * par, char * arg[9], Name **name, Arglist ** args)
{
    int i;
    Arglist * p = par;

    for (i = 0; i < n && p != NULL; i++)
        p = p->next;
    if (p == NULL) {
        char * a = arg[n];

        *name = NULL;
        *args = (Arglist *)a;
    }
    else {
        *name = p->name;
        *args = p->args;
    }
}
◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: [lookup](#) [54d](#), [145c](#).

Uses: [Arglist](#) [179b](#), [Name](#) [160b](#), [next](#) [131b](#).

```

"scraps.c" 145a≡
Arglist * instance(Arglist * a, Arglist * par, char * arg[9], int * ch)
{
    if (a != NULL) {
        int changed = 0;
        Arglist *args, *next;
        Name* name;
        ⟨ Set up name, args and next 145c ⟩
        if (changed){
            ⟨ Build a new arglist 146a ⟩
            *ch = 1;
        }
    }

    return a;
}
◇

```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: instance 62b, 91b, 145bc, 154.

Uses: Arglist 179b, Name 160b, next 131b.

```

⟨ Function prototypes 145b ⟩ ≡
Arglist *instance(Arglist *a, Arglist *par, char *arg[9], int *ch);
◇

```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: Arglist 179b, instance 145a.

```

⟨ Set up name, args and next 145c ⟩ ≡
next = instance(a->next, par, arg, &changed);
name = a->name;
if (name == (Name *)1) {
    Embed_Node * q = (Embed_Node *)arena_getmem(sizeof(Embed_Node));
    q->defs = (Scrap_Node *)a->args;
    q->args = par;
    args = (Arglist *)q;
    changed = 1;
} else if (name != NULL)
    args = instance(a->args, par, arg, &changed);
else {
    char * p = (char *)a->args;
    if (p[0] == ARG_CHR) {
        lookup(p[1] - '1', par, arg, &name, &args);
        changed = 1;
    }
    else {
        args = a->args;
    }
}
}◇

```

Fragment referenced in 145a.

Uses: arena_getmem 202b, Arglist 179b, ARG_CHR 177c, defs 131c, Embed_Node 182d, instance 145a, lookup 144b, Name 160b, next 131b, Scrap_Node 160a.

< Build a new arglist 146a > $\equiv$

```
a = (Arglist *)arena_getmem(sizeof(Arglist));
a->name = name;
a->args = args;
a->next = next;◇
```

Fragment referenced in [145a](#).

Uses: [arena_getmem 202b](#), [Arglist 179b](#), [next 131b](#).

"scraps.c" 146b $\equiv$

```
static Arglist *pop_scrap_name(Manager *manager, Parameters *parameters)
{
    char name[MAX_NAME_LEN];
    char *p = name;
    Arglist * args;
    int c;
    int idx;

    (void)pop(manager); /* not sure why we have to pop twice */
    c = pop(manager);

    while (c != nw_char) {
        *p++ = c;
        c = pop(manager);
    }
    *p = '\000';
    if (sscanf(name, "%d", &idx) != 1)
    {
        fprintf(stderr, "%s: found an internal problem (2)\n", command_name);
        exit(-1);
    }
    args = arglist_lookup(idx);
    < Check for end of scrap name 146c >
    return args;
}
◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: [pop_scrap_name 154](#).

Uses: [Arglist 179b](#), [arglist_lookup 148](#), [command_name 30a](#), [exit 22](#), [fprintf 22](#), [Manager 135c](#), [MAX_NAME_LEN 23b](#), [nw_char 29bc](#), [Parameters 54c](#), [pop 142c](#), [stderr 22](#).

< Check for end of scrap name 146c > $\equiv$

```
{
    c = pop(manager);
    < Check for macro parameters 57 >
}◇
```

Fragment referenced in [146b](#).

Uses: [pop 142c](#).

```

"scraps.c" 147a≡
int write_scraps(FILE *file, char *spelling, Scrap_Node *defs,
                int global_indent, char *indent_chars,
                char debug_flag, char tab_flag, char indent_flag,
                unsigned char comment_flag, char location_flag,
                Arglist *inArgs,
                char *inParams[9], Parameters parameters, char *title)
{
    /* This is in file file name */
    int indent = 0;
    int newline = 1;
    while (defs) {
        ⟨ Copy defs->scrap to file 149 ⟩
        defs = defs->next;
    }
    return indent + global_indent;
}
◇

```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: write_scraps 54d, 62b, 79, 91b, 103, 123a, 132b, 154, 156c.

Uses: Arglist 179b, defs 131c, FILE 22, next 131b, Parameters 54c, Scrap_Node 160a.

```

⟨ Forward declarations for scraps.c 147b ⟩ ≡
    int delayed_indent = 0;
◇

```

Fragment defined by 147b, 148, 156b, 175, 193c.

Fragment referenced in 132a.

Defines: delayed_indent 54d, 149, 151c, 154, 155ab.

⟨ Forward declarations for scraps.c 148 ⟩ ≡

```
#define MAX_ARGLISTS 4096
static Arglist *arglist_table[MAX_ARGLISTS];
static int arglist_count = 0;

void arglist_reset(void)
{
    arglist_count = 0;
}

static int arglist_register(Arglist *a)
{
    if (arglist_count >= MAX_ARGLISTS) {
        fprintf(stderr, "%s: too many macro argument lists (max %d)\n",
            command_name, MAX_ARGLISTS);
        exit(EXIT_FAILURE);
    }
    arglist_table[arglist_count] = a;
    return arglist_count++;
}

static Arglist *arglist_lookup(int idx)
{
    if (idx < 0 || idx >= arglist_count) {
        fprintf(stderr, "%s: invalid arglist index %d\n", command_name, idx);
        exit(EXIT_FAILURE);
    }
    return arglist_table[idx];
}
◇
```

Fragment defined by [147b](#), [148](#), [156b](#), [175](#), [193c](#).

Fragment referenced in [132a](#).

Defines: `arglist_count` Never used, `arglist_lookup` [143](#), [146b](#), [193b](#), `arglist_register` [141d](#), `arglist_reset` [201c](#), [203b](#), `arglist_table` Never used, `MAX_ARGLISTS` Never used.

Uses: `Arglist` [179b](#), `command_name` [30a](#), `exit` [22](#), `fprintf` [22](#), `stderr` [22](#).

```

< Copy defs->scrap to file 149> ≡
{
    char c;
    Manager reader;
    Parameters local_parameters = 0;
    int line_number = scrap_array(defs->scrap).file_line;
    reader.scrap = scrap_array(defs->scrap).slab;
    reader.prev = NULL;
    reader.index = 0;
    < Insert debugging information if required 150b>
    < Insert section opening marker 150c>
    if (delayed_indent)
    {
        < Insert appropriate indentation 151a>
    }
    c = pop(&reader);
    while (c) {
        switch (c) {
            case '\n':
                if (global_indent >= 0) {
                    putc(c, file);
                    line_number++;
                    newline = 1;
                    delayed_indent = 0;
                    < Insert appropriate indentation 151a>
                    break;
                } else {
                    /* Don't show newlines in embedded fragments */
                    fputs(". . .", file);
                    return 0;
                }
            case '\t': < Handle tab characters on output 151d>
                delayed_indent = 0;
                break;
            default:
                if (c==nw_char)
                {
                    < Check for macro invocation in scrap 152a>
                    break;
                }
                putc(c, file);
                if (global_indent >= 0) {
                    < Add more indentation ' ' 150a>
                }
                indent++;
                if (c > ' ') newline = 0;
                delayed_indent = 0;
                break;
            }
        c = pop(&reader);
    }
    < Insert section closing marker 150d>
}◊

```

Fragment referenced in 147a.

Uses: defs 131c, delayed_indent 147b, file_line 131c, fputs 22, Manager 135c, nw_char 29bc, Parameters 54c, pop 142c, putc 22, scrap_array 132a, slab 131c.

We need to make sure that we don't overflow `indent_chars[]`. If the line is longer than `MAX_INDENT`, we simply stop recording; the `Put out the indent` scrap will use spaces for any positions beyond the recorded range.

```
< Add more indentation char 150a > ≡
{
    if (global_indent + indent < MAX_INDENT)
        indent_chars[global_indent + indent] = char;
}◊
```

Fragment referenced in [149](#), [151d](#), [152a](#).

Uses: `MAX_INDENT` [122d](#).

```
< Insert debugging information if required 150b > ≡
if (debug_flag) {
    fprintf(file, "\n#line %d \"%s\"\n",
            line_number, scrap_array(defs->scrap).file_name);
    < Insert appropriate indentation 151a >
}◊
```

Fragment referenced in [149](#), [152a](#).

Uses: `defs` [131c](#), `file_name` [131c](#), `fprintf` [22](#), `scrap_array` [132a](#).

Section markers in the Knuth style delimit each scrap with opening and closing comments that include the scrap number and source location. For example, in C: `/* {5: weft.w:28} */` and `/* {:5} */`.

```
< Insert section opening marker 150c > ≡
if (location_flag && comment_flag) {
    fprintf(file, "%s{:d: %s:%d}%s\n",
            comment_begin[comment_flag],
            defs->scrap,
            scrap_array(defs->scrap).file_name,
            line_number,
            comment_end[comment_flag]);
}◊
```

Fragment referenced in [149](#).

Uses: `comment_begin` [175](#), `comment_end` [175](#), `defs` [131c](#), `file_name` [131c](#), `fprintf` [22](#), `scrap_array` [132a](#).

```
< Insert section closing marker 150d > ≡
if (location_flag && comment_flag) {
    fprintf(file, "%s{:d}%s\n",
            comment_begin[comment_flag],
            defs->scrap,
            comment_end[comment_flag]);
}◊
```

Fragment referenced in [149](#).

Uses: `comment_begin` [175](#), `comment_end` [175](#), `defs` [131c](#), `fprintf` [22](#).

⟨ Insert appropriate indentation 151a ⟩ ≡

```
{
    char c1 = pop(&reader);
    char c2 = pop(&reader);

    if (indent_flag && !(⟨ Indent suppressed 151c ⟩)) {
        ⟨ Put out the indent 151b ⟩
    }
    indent = 0;
    backup(2, &reader);
}◊
```

Fragment referenced in [149](#), [150b](#).

Uses: [backup 144a](#), [pop 142c](#).

⟨ Put out the indent 151b ⟩ ≡

```
if (tab_flag)
    for (indent=0; indent<global_indent; indent++)
        putc(' ', file);
else
    for (indent=0; indent<global_indent; indent++)
        putc(indent < MAX_INDENT ? indent_chars[indent] : ' ', file);
◊
```

Fragment referenced in [54d](#), [151a](#).

Uses: [MAX_INDENT 122d](#), [putc 22](#).

Indent will be suppressed if the next character is a newline or if the next two characters are `@#`. If the next two characters are `@<`, we suppress the indent for now but mark that it may be needed when the next fragment is started.

⟨ Indent suppressed 151c ⟩ ≡

```
c1 == '\n'
|| (c1 == nw_char && (c2 == '#' || (delayed_indent |= (c2 == '<'))))◊
```

Fragment referenced in [151a](#).

Uses: [delayed_indent 147b](#), [nw_char 29bc](#).

⟨ Handle tab characters on output 151d ⟩ ≡

```
{
    if (tab_flag)
        ⟨ Expand tab into spaces 73c ⟩
    else {
        putc('\t', file);
        if (global_indent >= 0) {
            ⟨ Add more indentation '\t' 150a ⟩
        }
        indent++;
    }
}◊
```

Fragment referenced in [149](#).

Uses: [putc 22](#).

```

⟨ Check for macro invocation in scrap 152a ⟩ ≡
{
    int oldin = indent;
    char oldcf = comment_flag;
    c = pop(&reader);
    switch (c) {
        case 't': ⟨ Copy fragment title into file 155c ⟩
            break;
        case 'c': ⟨ Copy block comment from scrap 53a ⟩
            break;
        case 'f': ⟨ Copy file name into file 153 ⟩
            break;
        case 'x': ⟨ Copy label from scrap into file 152b ⟩
        case '_': break;
        case 'v': ⟨ Copy version info into file 75a ⟩
            break;
        case 's': indent = -global_indent;
            comment_flag = 0;
            break;
        case '<': ⟨ Copy macro into file 154 ⟩
            ⟨ Insert debugging information if required 150b ⟩
            indent = oldin;
            comment_flag = oldcf;
            break;
        ⟨ Handle macro parameter substitution 54d ⟩
            indent = oldin;
            break;
        default:
            if(c==nw_char)
            {
                putc(c, file);
                if (global_indent >= 0) {
                    ⟨ Add more indentation ' ' 150a ⟩
                }
                indent++;
                break;
            }
            /* ignore, since we should already have a warning */
            break;
    }
}◊

```

Fragment referenced in 149.

Uses: `nw_char` 29bc, `pop` 142c, `putc` 22.

```

⟨ Copy label from scrap into file 152b ⟩ ≡
{
    ⟨ Get label from (152c pop(&reader) ) 198c ⟩
    write_label(label_name, file);
}◊

```

Fragment referenced in 152a.

Uses: `pop` 142c.

⟨ Copy file name into file 153 ⟩ $\equiv$

```
    if (defs->quoted)
        fprintf(file, "%cf", nw_char);
    else
        fputs(spelling, file);
    ◇
```

Fragment referenced in [152a](#).

Uses: `defs` [131c](#), `fprintf` [22](#), `fputs` [22](#), `nw_char` [29bc](#).

< Copy macro into file 154> ≡

```
{
    Arglist *a = pop_scrap_name(&reader, &local_parameters);
    Name *name = a->name;
    int changed;
    Arglist *args = instance(a->args, inArgs, inParams, &changed);
    int i, nargs;
    char *p = name->spelling;
    char **inParams = name->arg;
    Arglist *q = args;

    if (name->mark) {
        fprintf(stderr, "%s: recursive macro discovered involving <%s>\n",
            command_name, name->spelling);
        exit(-1);
    }
    if (name->defs && !defs->quoted) {
        unsigned char saved_comment = comment_flag;
        char saved_location = location_flag;
        char saved_debug = debug_flag;
        if (name->lang[0] != '\0') {
            comment_flag = name->comment_flag;
            location_flag = name->location_flag;
            debug_flag = name->debug_flag;
        }
        if (!newline) {
            comment_flag = 0;
            location_flag = FALSE;
            debug_flag = FALSE;
        }
        < Perhaps comment this macro 155a>
        name->mark = TRUE;
        indent = write_scraps(file, spelling, name->defs, global_indent + indent,
            indent_chars, debug_flag, tab_flag, indent_flag,
            comment_flag, location_flag, args, name->arg,
            local_parameters, name->spelling);
        comment_flag = saved_comment;
        location_flag = saved_location;
        debug_flag = saved_debug;
        indent -= global_indent;
        name->mark = FALSE;
    }
    else
    {
        if (delayed_indent)
        {
            for (i = indent + global_indent; --i >= 0; )
                putc(' ', file);
        }

        fprintf(file, "%c<", nw_char);
        if (name->sector == 0)
            fputc('+', file);
        < Comment this macro use 156a>
        fprintf(file, "%c>", nw_char);
        if (!defs->quoted && !tex_flag)
            fprintf(stderr, "%s: macro never defined <%s>\n",
                command_name, name->spelling);
    }
}
}◊
```

Fragment referenced in 152a.

Uses: Arglist 179b, command_name 30a, defs 131c, delayed_indent 147b, exit 22, FALSE 23a, fprintf 22, instance 145a, Name 160b, nw_char 29bc, pop_scrap_name 146b, putc 22, sector 131c, stderr 22, tex_flag 28, TRUE 23a, write_scraps 147a.

```

< Perhaps comment this macro 155a > ≡
    if (comment_flag && newline) {
        < Perhaps put a delayed indent 155b >
        fputs(comment_begin[comment_flag], file);
        < Comment this macro use 156a >
        if (xref_flag) {
            putc(' ', file);
            write_single_scrap_ref(file, name->defs->scrap);
        }
        fputs(comment_end[comment_flag], file);
        putc('\n', file);
        if (!delayed_indent)
            for (i = indent + global_indent; --i >= 0; )
                putc(' ', file);
    }
    ◇

```

Fragment referenced in [54d](#), [154](#).

Uses: [comment_begin 175](#), [comment_end 175](#), [defs 131c](#), [delayed_indent 147b](#), [fputs 22](#), [putc 22](#), [write_single_scrap_ref 133b](#), [xref_flag 28](#).

```

< Perhaps put a delayed indent 155b > ≡
    if (delayed_indent)
        for (i = indent + global_indent; --i >= 0; )
            putc(' ', file);
    ◇

```

Fragment referenced in [53a](#), [155a](#).

Uses: [delayed_indent 147b](#), [putc 22](#).

```

< Copy fragment title into file 155c > ≡
{
    char * p = title;
    Arglist *q = inArgs;
    int nargs;

    < Comment this macro use 156a >
    if (xref_flag) {
        putc(' ', file);
        write_single_scrap_ref(file, defs->scrap);
    }
}
    ◇

```

Fragment referenced in [152a](#).

Uses: [Arglist 179b](#), [defs 131c](#), [putc 22](#), [write_single_scrap_ref 133b](#), [xref_flag 28](#).

⟨ *Comment this macro use 156a* ⟩ ≡

```
narg = 0;
while (*p != '\000') {
    if (*p == ARG_CHR) {
        if (q == NULL) {
            if (defs->quoted)
                fprintf(file, "%c'%s%c'", nw_char, inParams[narg], nw_char);
            else
                fprintf(file, "'%s'", inParams[narg]);
        }
        else {
            comment_ArglistElement(file, q, defs->quoted);
            q = q->next;
        }
        p++;
        narg++;
    }
    else
        fputc(*p++, file);
}
}

```

Fragment referenced in [154](#), [155ac](#).

Uses: ARG_CHR [177c](#), comment_ArglistElement [156b](#), defs [131c](#), fprintf [22](#), next [131b](#), nw_char [29bc](#).

⟨ *Forward declarations for scraps.c 156b* ⟩ ≡

```
static void
comment_ArglistElement(FILE * file, Arglist * args, int quote)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        if (quote)
            fprintf(file, "%c'%s%c'", nw_char, (char *)q, nw_char);
        else
            fprintf(file, "'%s'", (char *)q);
    } else if (name == (Name *)1) {
        ⟨ Include an embedded scrap in comment 156c ⟩
    } else {
        ⟨ Include a fragment use in comment 157a ⟩
    }
}
}

```

Fragment defined by [147b](#), [148](#), [156b](#), [175](#), [193c](#).

Fragment referenced in [132a](#).

Defines: comment_ArglistElement [156a](#), [157a](#).

Uses: Arglist [179b](#), FILE [22](#), fprintf [22](#), Name [160b](#), nw_char [29bc](#).

⟨ *Include an embedded scrap in comment 156c* ⟩ ≡

```
Embed_Node * e = (Embed_Node *)q;
fputc('{', file);
write_scraps(file, "", e->defs, -1, "", 0, 0, 0, 0, 0, e->args, NULL, NULL, "");
fputc('}', file);

```

Fragment referenced in [156b](#).

Uses: defs [131c](#), Embed_Node [182d](#), write_scraps [147a](#).

⟨ *Include a fragment use in comment 157a* ⟩ ≡

```
char * p = name->spelling;
if (quote)
    fputc(nw_char, file);
fputc('<', file);
if (quote && name->sector == 0)
    fputc('+', file);
while (*p != '\000') {
    if (*p == ARG_CHR) {
        comment_ArglistElement(file, q, quote);
        q = q->next;
        p++;
    }
    else
        fputc(*p++, file);
}
if (quote)
    fputc(nw_char, file);
fputc('>', file);◇
```

Fragment referenced in [156b](#).

Uses: ARG_CHR [177c](#), comment_ArglistElement [156b](#), next [131b](#), nw_char [29bc](#), sector [131c](#).

3.2.1 Collecting Page Numbers

⟨ *Function prototypes 157b* ⟩ ≡

```
extern void collect_numbers(char *aux_name);
◇
```

Fragment defined by [42a](#), [58b](#), [71a](#), [73b](#), [85a](#), [87](#), [122ab](#), [125a](#), [127b](#), [132b](#), [134b](#), [142b](#), [145b](#), [157b](#), [161c](#), [164b](#), [178d](#), [188c](#), [195a](#), [196c](#), [199b](#), [201c](#).

Fragment referenced in [21](#).

Uses: collect_numbers [158a](#).

```

"scraps.c" 158a≡
void collect_numbers(char *aux_name)
{
    if (number_flag) {
        int i;
        for (i=1; i<scraps; i++)
            scrap_array(i).page = i;
    }
    else {
        FILE *aux_file = fopen(aux_name, "r");
        already_warned = FALSE;
        if (aux_file) {
            char aux_line[500];
            while (fgets(aux_line, 500, aux_file)) {
                ⟨ Read line in .aux file 158b ⟩
            }
            fclose(aux_file);
            ⟨ Add letters to scraps with duplicate page numbers 159a ⟩
        }
    }
}
◇

```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: collect_numbers 41, 157b.

Uses: already_warned 135a, FALSE 23a, fclose 22, FILE 22, fopen 22, number_flag 28, page 131c, scraps 132a, scrap_array 132a.

The *memoir* document class creates references in the .aux file with nested braces, so after each open brace we have to wait for the matching closing brace.

```

⟨ Read line in .aux file 158b ⟩ ≡
int scrap_number;
int page_number;
int i;
int dummy_idx;
int bracket_depth = 1;
if (1 == sscanf(aux_line,
                "\\newlabel{scrap%d}{%n",
                &scrap_number,
                &dummy_idx)) {
    for (i = dummy_idx; i < (int)strlen(aux_line) && bracket_depth > 0; i++) {
        if (aux_line[i] == '{') bracket_depth++;
        else if (aux_line[i] == '}') bracket_depth--;
    }
    if (i > dummy_idx
        && i < (int)strlen(aux_line)
        && 1 == sscanf(aux_line+i, "{%d}", &page_number)) {
        if (scrap_number < scraps)
            scrap_array(scrap_number).page = page_number;
        else
            ⟨ Warn (only once) about needing to rerun after Latex 134e ⟩
    }
}
}
◇

```

Fragment referenced in 158a.

Uses: page 131c, scraps 132a, scrap_array 132a, strlen 22.

```

⟨ Add letters to scraps with duplicate page numbers 159a ⟩ ≡
{
    int i = 0;

    ⟨ Step i to the next valid scrap 159b ⟩
    ⟨ For all remaining scraps 159c ⟩ {
        int j = i;
        ⟨ Step j to the next valid scrap 159b ⟩
        ⟨ Perhaps add letters to the page numbers 159d ⟩
        i = j;
    }
}
◇

```

Fragment referenced in [158a](#).

```

⟨ Step i to the next valid scrap 159b ⟩ ≡
do
    i++;
while (i < scraps && scrap_array(i).page == -1);
◇

```

Fragment referenced in [159a](#).

Uses: [page 131c](#), [scraps 132a](#), [scrap_array 132a](#).

```

⟨ For all remaining scraps 159c ⟩ ≡
while (i < scraps)◇

```

Fragment referenced in [159a](#).

Uses: [scraps 132a](#).

```

⟨ Perhaps add letters to the page numbers 159d ⟩ ≡
if (scrap_array(i).page == scrap_array(j).page) {
    if (scrap_array(i).letter == 0)
        scrap_array(i).letter = 'a';
    scrap_array(j).letter = scrap_array(i).letter + 1;
}
◇

```

Fragment referenced in [159a](#).

Uses: [letter 131c](#), [page 131c](#), [scrap_array 132a](#).

3.3 Names

Each scrap is linked through a simple node:

< Type declarations 160a > ≡

```
typedef struct scrap_node {
    struct scrap_node *next;
    int scrap;
    char quoted;
} Scrap_Node;
```

◇

Fragment defined by 23a, 54c, 122d, 160ab, 177c, 179b, 182d, 194c, 201a.

Fragment referenced in 21.

Defines: `Scrap_Node` 47b, 54d, 59a, 69b, 78, 79, 81a, 84ab, 86b, 88a, 96, 101, 103, 104d, 107ab, 108c, 110c, 112, 114, 116, 132b, 141ab, 142a, 145c, 147a, 160b, 182cd, 183cd, 192, 193c, 194a, 196a, 197b.

Uses: `next` 131b.

A **Name** represents a file or fragment definition. Beyond the spelling and the binary tree links, several flags control how the tangled output is formatted:

- `debug_flag` — if TRUE, emit `#line` directives (set by `-d` or auto-detected for C/C++).
- `comment_flag` — selects the comment style for section markers and block comments: 0=none, 1=C, 2=C++, 3=shell, 4=Lua/SQL, 5=HTML/XML (see Section 3.3.5).
- `location_flag` — if TRUE, emit Knuth-style opening and closing section markers around each scrap in the tangled output. Activated automatically by `@l` auto-detection or the `@L` command.
- `lang` — the language tag (e.g. "c", "python") used for syntax highlighting in the Markdown weave output. Set from a preceding `@l` command or inferred from the file extension of an `@o` name. Empty if unknown.

< Type declarations 160b > ≡

```
typedef struct name {
    char *spelling;
    struct name *llink;
    struct name *rlink;
    Scrap_Node *defs;
    Scrap_Node *uses;
    char * arg[9];
    int mark;
    char tab_flag;
    char indent_flag;
    char debug_flag;
    char location_flag;
    char suppress_markers;
    unsigned char comment_flag;
    char lang[64];
    unsigned char sector;
} Name;
```

◇

Fragment defined by 23a, 54c, 122d, 160ab, 177c, 179b, 182d, 194c, 201a.

Fragment referenced in 21.

Defines: `Name` 32, 46e, 47a, 54d, 59a, 62b, 63, 64a, 72, 77, 79, 80b, 82ab, 83a, 85ab, 86a, 88a, 91b, 92ab, 97a, 101, 103, 104b, 105b, 106a, 108b, 110c, 112, 114, 115ab, 116, 117ab, 122abc, 140, 141c, 142ab, 144b, 145ac, 154, 156b, 161abc, 163a, 164a, 166, 167, 172, 176, 178d, 179abc, 181, 182c, 183c, 184a, 187a, 189ab, 192, 193c, 194bc, 196a, 197b, 198b.

Uses: `defs` 131c, `Scrap_Node` 160a, `sector` 131c, `uses` 131c.

The variable `pending_lang` acts as a one-shot channel between the `@l` command in pass1 (which fills it with a language tag like "js" or "python") and `collect_file_name` or `collect_macro_name` (which consume it to set the flags and the `lang` field on the next `@o` or `@d` name).

Global variable declarations 161a ≡

```
extern Name *file_names;
extern Name *macro_names;
extern Name *user_names;
extern int scrap_name_has_parameters;
extern int scrap_ended_with;
extern char pending_lang[64];
◇
```

Fragment defined by 28, 29b, 30a, 46c, 51c, 67b, 125b, 135a, 161a, 201b.

Fragment referenced in 21.

Defines: `file_names` 41, 42b, 54b, 80a, 104a, 117b, 122b, 161b, 172, `macro_names` 41, 42b, 54b, 81b, 105a, 117b, 122b, 161b, 178b, `pending_lang` 43a, 161b, 174ab, `user_names` 42b, 54b, 85c, 108a, 141ab, 161b, 189a.

Uses: Name 160b.

Global variable definitions 161b ≡

```
Name *file_names = NULL;
Name *macro_names = NULL;
Name *user_names = NULL;
int scrap_name_has_parameters;
int scrap_ended_with;
char pending_lang[64] = "";
◇
```

Fragment defined by 29ac, 30b, 46d, 51d, 67c, 125c, 135b, 161b.

Fragment referenced in 25e.

Uses: `file_names` 161a, `macro_names` 161a, Name 160b, `pending_lang` 161a, `user_names` 161a.

Function prototypes 161c ≡

```
extern Name *collect_file_name(void);
extern Name *collect_macro_name(void);
extern Arglist *collect_scrap_name(int current_scrap);
extern Name *name_add(Name **rt, char *spelling, unsigned char sector);
extern Name *prefix_add(Name **rt, char *spelling, unsigned char sector);
extern char *save_string(char *s);
extern void reverse_lists(Name *names);
extern void collect_lang_def(void);
extern void collect_weave_format(void);
◇
```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: `Arglist` 179b, `collect_file_name` 172, `collect_lang_def` 170, `collect_macro_name` 176, `collect_scrap_name` 180, `collect_weave_format` 171, Name 160b, `name_add` 166, `prefix_add` 163a, `reverse_lists` 183c, `save_string` 162b, `sector` 131c.

```

"names.c" 162a≡
enum { LESS, GREATER, EQUAL, PREFIX, EXTENSION };

static int compare(char *x, char *y)
{
    int len, result;
    int xl = strlen(x);
    int yl = strlen(y);
    int xp = x[xl - 1] == ' ';
    int yp = y[yl - 1] == ' ';
    if (xp) xl--;
    if (yp) yl--;
    len = xl < yl ? xl : yl;
    result = strncmp(x, y, len);
    if (result < 0) return GREATER;
    else if (result > 0) return LESS;
    else if (xl < yl) {
        if (xp) return EXTENSION;
        else return LESS;
    }
    else if (xl > yl) {
        if (yp) return PREFIX;
        else return GREATER;
    }
    else return EQUAL;
}
◇

```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Defines: compare 163a, 164a, EQUAL 163a, 164a, EXTENSION 163a, 164a, GREATER 163a, 164a, LESS 163a, 164a, PREFIX 163a, 164a.

Uses: strlen 22.

```

"names.c" 162b≡
char *save_string(char *s)
{
    char *new_str = (char *) arena_getmem((strlen(s) + 1) * sizeof(char));
    strcpy(new_str, s);
    return new_str;
}
◇

```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Defines: save_string 35b, 40, 129, 136c, 161c, 163a, 167, 179a, 183b.

Uses: arena_getmem 202b, strlen 22.

```

"names.c" 163a≡
static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector);

static char * found_name = NULL;

Name *prefix_add(Name **rt, char *spelling, unsigned char sector)
{
    Name *node = *rt;
    int cmp;

    while (node) {
        switch ((cmp = compare(node->spelling, spelling))) {
            case GREATER:    rt = &node->rlink;
                            break;
            case LESS:      rt = &node->llink;
                            break;
            case EQUAL:
                found_name = node->spelling;
                FALLTHROUGH;
            case EXTENSION: if (node->sector > sector) {
                            rt = &node->rlink;
                            break;
                        }
                        else if (node->sector < sector) {
                            rt = &node->llink;
                            break;
                        }
                        if (cmp == EXTENSION)
                            node->spelling = save_string(spelling);
                        return node;
            case PREFIX:    < Check for ambiguous prefix 163b >
                            return node;
        }
        node = *rt;
    }
    < Create new name entry 167 >
}
◇

```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Defines: prefix_add 161c, 178b.

Uses: compare 162a, EQUAL 162a, EXTENSION 162a, FALLTHROUGH 23a, GREATER 162a, LESS 162a, Name 160b, PREFIX 162a, save_string 162b, sector 131c.

Since a very short prefix might match more than one fragment name, I need to check for other matches to avoid mistakes. Basically, I simply continue the search down *both* branches of the tree.

```

< Check for ambiguous prefix 163b > ≡
{
    if (ambiguous_prefix(node->llink, spelling, sector) ||
        ambiguous_prefix(node->rlink, spelling, sector))
        fprintf(stderr,
            "%s: ambiguous prefix %c<%s...%c> (%s, line %d)\n",
            command_name, nw_char, spelling, nw_char, source_name, source_line);
}
◇

```

Fragment referenced in 163a.

Uses: command_name 30a, fprintf 22, nw_char 29bc, sector 131c, source_line 125b, source_name 125b, stderr 22.

```

"names.c" 164a≡
static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector)
{
    while (node) {
        switch (compare(node->spelling, spelling)) {
            case GREATER:    node = node->rlink;
                            break;
            case LESS:       node = node->llink;
                            break;
            case EXTENSION:
            case PREFIX:
            case EQUAL:      if (node->sector > sector) {
                            node = node->rlink;
                            break;
                            }
                            else if (node->sector < sector) {
                            node = node->llink;
                            break;
                            }
                            return TRUE;
        }
    }
    return FALSE;
}
◇

```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Uses: compare 162a, EQUAL 162a, EXTENSION 162a, FALSE 23a, GREATER 162a, LESS 162a, Name 160b, PREFIX 162a, sector 131c, TRUE 23a.

Rob Shillingsburg suggested that I organize the index of user-specified identifiers more traditionally; that is, not relying on strict ASCII comparisons via `strcmp`. Ideally, we'd like to see the index ordered like this:

```

aardvark
Adam
atom
Atomic
atoms

```

The function `robs_strcmp` implements the desired predicate. It returns -2 for alphabetically less-than, -1 for less-than but only differing in case, zero for equal, 1 for greater-than but only in case and 2 for alphabetically greater-than.

```

⟨ Function prototypes 164b ⟩ ≡
extern int robs_strcmp(char*, char*);
◇

```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: `robs_strcmp` 165a.

```

"names.c" 165a≡
int robs_strcmp(char* x, char* y)
{
    int cmp = 0;

    for (; *x && *y; x++, y++)
    {
        < Skip invisibles on x 165b >
        < Skip invisibles on y 165b >
        if (*x == *y)
            continue;
        if (islower(*x) && toupper(*x) == *y)
        {
            if (!cmp) cmp = 1;
            continue;
        }
        if (islower(*y) && *x == toupper(*y))
        {
            if (!cmp) cmp = -1;
            continue;
        }
        return 2*(toupper(*x) - toupper(*y));
    }
    if (*x)
        return 2;
    if (*y)
        return -2;
    return cmp;
}
◇

```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.
 Defines: robs_strcmp 82c, 115b, 164b, 166, 167, 194b.
 Uses: islower 22, toupper 22.

Certain character sequences are invisible when printed. We don't want them to be considered for the alphabetical ordering.

```

< Skip invisibles on p 165b > ≡
    if (*p == '|')
        p++;
◇

```

Fragment referenced in 165a.

```

"names.c" 166≡
Name *name_add(Name **rt, char *spelling, unsigned char sector)
{
    Name *node = *rt;
    while (node) {
        int result = robs_strcmp(node->spelling, spelling);
        if (result > 0)
            rt = &node->llink;
        else if (result < 0)
            rt = &node->rlink;
        else
        {
            found_name = node->spelling;
            if (node->sector > sector)
                rt = &node->llink;
            else if (node->sector < sector)
                rt = &node->rlink;
            else
                return node;
        }
        node = *rt;
    }
    ( Create new name entry 167 )
}
◇

```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Defines: name_add 141ab, 161c, 172.

Uses: Name 160b, robs_strcmp 165a, sector 131c.

```

⟨ Create new name entry 167 ⟩ ≡
{
    node = (Name *) arena_getmem(sizeof(Name));
    if (found_name && robs_strcmp(found_name, spelling) == 0)
        node->spelling = found_name;
    else
        node->spelling = save_string(spelling);
    node->mark = FALSE;
    node->llink = NULL;
    node->rlink = NULL;
    node->uses = NULL;
    node->defs = NULL;
    node->arg[0] =
    node->arg[1] =
    node->arg[2] =
    node->arg[3] =
    node->arg[4] =
    node->arg[5] =
    node->arg[6] =
    node->arg[7] =
    node->arg[8] = NULL;
    node->tab_flag = TRUE;
    node->indent_flag = TRUE;
    node->debug_flag = FALSE;
    node->location_flag = FALSE;
    node->suppress_markers = FALSE;
    node->comment_flag = 0;
    node->lang[0] = '\0';
    node->sector = sector;
    *rt = node;
    return node;
}◊

```

Fragment referenced in [163a](#), [166](#).

Uses: [arena_getmem 202b](#), [defs 131c](#), [FALSE 23a](#), [Name 160b](#), [robs_strcmp 165a](#), [save_string 162b](#), [sector 131c](#), [TRUE 23a](#), [uses 131c](#).

3.3.1 Language Auto-Detection

When a Markdown fence (‘‘‘c, ‘‘‘python) or a L^AT_EX marker (%%c) precedes an @o directive, weft auto-detects the language’s comment style and, for C/C++, enables #line directives. This is driven by a data table that maps fence tags to one of the six comment styles defined in [Section 3.3.5](#).

Each entry records:

- **name** — the fence tag (lowercase, e.g. "js", "python", "lua").
- **comment** — index into the `comment_begin/comment_end` arrays (1–5).
- **line_directive** — TRUE only for languages whose toolchains honour #line (C family).

The built-in table covers over 250 entries (languages, file extensions, and aliases) organised by comment style. For any language not listed, the @L command ([Section 3.3.2](#)) lets users add entries at the document level. User-defined entries are searched first, so they can also override built-in defaults. If a fence tag matches neither table, weft defaults to C++ line comments (//) with no #line.

```

"names.c" 168≡
typedef struct {
    const char *name;
    unsigned char comment;
    char line_directive;
} LangEntry;

static const LangEntry lang_table[] = {
    /* ---- Style 1: C block comments WITH #line ---- */
    {"c",          1, TRUE},
    {"h",          1, TRUE},
    {"hpp",        2, TRUE},
    {"hxx",        2, TRUE},
    {"hh",         2, TRUE},
    {"cpp",        2, TRUE},
    {"c++",        2, TRUE},
    {"cxx",        2, TRUE},
    {"cc",         2, TRUE},
    {"objc",       1, TRUE},
    {"objcpp",     2, TRUE},
    {"m",          1, TRUE},
    {"mm",         2, TRUE},
    {"ino",        2, TRUE},    /* Arduino */
    {"pde",        2, TRUE},    /* Processing (C-like) */
    /* ---- Style 1: C block comments WITHOUT #line ---- */
    {"css",        1, FALSE},
    {"scss",       1, FALSE},
    {"sass",       1, FALSE},
    {"less",       1, FALSE},
    {"stylus",     1, FALSE},
    {"styl",       1, FALSE},
    /* ---- Style 2: C++ line comments (//) ---- */
    /* --- JavaScript / TypeScript --- */
    {"js",         2, FALSE},
    {"javascript", 2, FALSE},
    {"jsx",        2, FALSE},
    {"mjs",        2, FALSE},
    {"cjs",        2, FALSE},
    {"ts",         2, FALSE},
    {"typescript", 2, FALSE},
    {"tsx",        2, FALSE},
    {"mts",        2, FALSE},
    {"cts",        2, FALSE},
    {"svelte",     2, FALSE},
    {"vue",        2, FALSE},
    /* --- JVM languages --- */
    {"java",       2, FALSE},
    {"scala",      2, FALSE},
    {"sc",         2, FALSE},
    {"kotlin",     2, FALSE},
    {"kt",         2, FALSE},
    {"kts",        2, FALSE},
    {"groovy",     2, FALSE},
    {"gradle",     2, FALSE},
    {"clj",        2, FALSE},    /* Clojure (;; but // is close enough) */
    {"cljs",       2, FALSE},
    {"cljc",       2, FALSE},
    /* --- .NET --- */
    {"cs",         2, FALSE},
    {"csharp",     2, FALSE},
    {"fs",         2, FALSE},
    {"fsharp",     2, FALSE},
    {"fsx",        2, FALSE},
    {"vb",         2, FALSE},    /* VB.NET uses ' but // is a safe fallback */
    /* --- Systems languages --- */
    {"go",         2, FALSE},
    {"rust",       2, FALSE},
    {"rs",         2, FALSE},
    {"zig",        2, FALSE},
    {"d",          2, FALSE},
    {"--",         2, FALSE}

```

3.3.2 User-Defined Languages

The `@L` command lets users register new language mappings (or override built-in ones) directly in the document preamble. This is useful for languages that are not in the built-in table, or when the default comment style for a language is not what the user wants.

Syntax: `@L name style [+d]`

- *name* — the fence tag that will trigger this mapping (e.g. `nim`, `glsl`, `wgsl`).
- *style* — one of `//`, `#`, `--`, `/*`, or `<!--`, selecting the comment delimiters.
- `+d` — optional; if present, enables `#line` directive generation for this language.

Examples:

```
@L nim #           % Nim uses # comments
@L glsl // +d      % GLSL uses // and supports #line
@L c //           % override: use // instead of /* */ for C
```

Up to 32 user-defined languages may be registered. They are searched before the built-in table, so a definition like `@L c //` effectively overrides the built-in mapping for C.

The `@L` command is processed during `pass1` and silently skipped during weaving. It should appear in the document preamble (before any `@o` or `@d` directives that depend on it).

```

"names.c" 170≡
void collect_lang_def(void)
{
    char name[64];
    char *p = name;
    char style[16];
    char *sp;
    int c;
    unsigned char comment = 0;
    char line_dir = FALSE;

    /* Skip whitespace after @L */
    c = source_get();
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read language name */
    while (c != EOF && c != ' ' && c != '\t' && c != '\n'
           && (p - name) < 62) {
        *p++ = c;
        c = source_get();
    }
    *p = '\0';

    if (name[0] == '\0') {
        fprintf(stderr, "%s: expected language name after %cL (%s, %d)\n",
                command_name, nw_char, source_name, source_line);
        while (c != EOF && c != '\n') c = source_get();
        return;
    }

    /* Skip whitespace */
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read comment style token */
    sp = style;
    while (c != EOF && c != ' ' && c != '\t' && c != '\n'
           && (sp - style) < 14) {
        *sp++ = c;
        c = source_get();
    }
    *sp = '\0';

    /* Map style string to comment index */
    if (strcmp(style, "//") == 0)        comment = 2;
    else if (strcmp(style, "#") == 0)    comment = 3;
    else if (strcmp(style, "--") == 0)   comment = 4;
    else if (strcmp(style, "/*") == 0)   comment = 1;
    else if (strcmp(style, "<!--") == 0) comment = 5;
    else {
        fprintf(stderr,
                "%s: unknown comment style '%s' for %cL (%s, %d)\n",
                command_name, style, nw_char, source_name, source_line);
        fprintf(stderr, " Valid styles: // # -- /* <!--\n");
        while (c != EOF && c != '\n') c = source_get();
        return;
    }

    /* Check for optional +d flag (#line directives) */
    while (c == ' ' || c == '\t')
        c = source_get();
    if (c == '+') {
        c = source_get();
        if (c == 'd') line_dir = TRUE;
    }

    /* Consume rest of line */
    while (c != EOF && c != '\n') c = source_get();

    /* Register the user language */

```

3.3.3 Weave Format Declaration

The `@W` command declares the weave output format in the source file. Valid tokens are `md` (Markdown) and `tex` (L^AT_EX). If the user has already specified a format on the command line (`-w md` or `-w tex`), the declaration is silently ignored.

```
"names.c" 171≡
void collect_weave_format(void)
{
    char token[64];
    char *p = token;
    int c;

    /* Skip whitespace after @W */
    c = source_get();
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read format token */
    while (c != EOF && c != ' ' && c != '\t' && c != '\n'
           && (p - token) < 62) {
        *p++ = c;
        c = source_get();
    }
    *p = '\0';

    /* Consume rest of line */
    while (c != EOF && c != '\n') c = source_get();

    if (token[0] == '\0') {
        fprintf(stderr, "%s: expected format after %cW (md or tex) (%s, %d)\n",
                command_name, nw_char, source_name, source_line);
        return;
    }

    /* If CLI already set the format, skip silently */
    if (weave_format_from_cli)
        return;

    if (strcmp(token, "md") == 0)
        weave_format = 2;
    else if (strcmp(token, "tex") == 0)
        weave_format = 1;
    else {
        fprintf(stderr, "%s: unknown weave format '%s' for %cW (%s, %d)\n",
                command_name, token, nw_char, source_name, source_line);
        fprintf(stderr, "  Valid formats: md tex\n");
    }
}
◇
```

File defined by [25c](#), [162ab](#), [163a](#), [164a](#), [165a](#), [166](#), [168](#), [170](#), [171](#), [172](#), [176](#), [179ac](#), [180](#), [183cd](#).

Defines: [collect_weave_format](#) [43c](#), [161c](#).

Uses: [command_name](#) [30a](#), [fprintf](#) [22](#), [nw_char](#) [29bc](#), [source_get](#) [127a](#), [source_line](#) [125b](#), [source_name](#) [125b](#), [stderr](#) [22](#), [weave_format](#) [28](#), [weave_format_from_cli](#) [28](#).

3.3.4 Collecting File and Fragment Names

An output file name (@o) is terminated by whitespace. After reading the name we check for optional per-file flags (-d, -t, -i, -c) and then, if no explicit comment or debug flags were given, apply any pending language defaults from a preceding fence, %%% marker, or @L definition.

"names.c" 172≡

```
Name *collect_file_name(void)
{
    Name *new_name;
    char name[MAX_NAME_LEN];
    char *p = name;
    int start_line = source_line;
    int c = source_get(), c2;
    while (isspace(c))
        c = source_get();
    while (isgraph(c)) {
        *p++ = c;
        c = source_get();
    }
    if (p == name) {
        fprintf(stderr, "%s: expected file name (%s, %d)\n",
            command_name, source_name, start_line);
        exit(-1);
    }
    *p = '\0';
    /* File names are always global. */
    new_name = name_add(&file_names, name, 0);
    < Handle optional per-file flags 173a >
    < Apply pending language if no explicit flags 174a >
    c2 = source_get();
    if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
        fprintf(stderr, "%s: expected %c{, %c[, or %c( after file name (%s, %d)\n",
            command_name, nw_char, nw_char, nw_char, source_name, start_line);
        exit(-1);
    }
    return new_name;
}
◇
```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Defines: collect_file_name 46e, 63, 92a, 161c.

Uses: command_name 30a, exit 22, file_names 161a, fprintf 22, isgraph 22, isspace 22, MAX_NAME_LEN 23b, Name 160b, name_add 166, nw_char 29bc, source_get 127a, source_line 125b, source_name 125b, stderr 22.

< Handle optional per-file flags 173a > ≡

```
{
  while (1) {
    while (isspace(c))
      c = source_get();
    if (c == '-') {
      c = source_get();
      do {
        switch (c) {
          case 't': new_name->tab_flag = FALSE;
                    break;
          case 'd': new_name->debug_flag = TRUE;
                    break;
          case 'i': new_name->indent_flag = FALSE;
                    break;
          case 's': new_name->suppress_markers = TRUE;
                    break;
          case 'c': < Get comment delimiters 173b >
                    break;
          default : fprintf(stderr, "%s: unexpected per-file flag (%s, %d)\n",
                          command_name, source_name, source_line);
                    break;
        }
        c = source_get();
      } while (!isspace(c));
    }
    else break;
  }
}
```

Fragment referenced in [172](#).

Uses: [command_name 30a](#), [FALSE 23a](#), [fprintf 22](#), [isspace 22](#), [source_get 127a](#), [source_line 125b](#), [source_name 125b](#), [stderr 22](#), [TRUE 23a](#).

The explicit per-file comment flags (`-cc`, `-c+`, `-cp`) select among the first three comment styles. These flags predate the auto-detection system; they still work and take precedence over any fence-based or `@L`-based setting.

The `-s` flag suppresses `#line` directives and section markers in the tangled output for that file, while still allowing language auto-detection to set the comment style and the `lang` tag (used by the Markdown weaver for syntax highlighting classes). This is useful for bootstrap source files or any output where tangle markers are unwanted.

< Get comment delimiters 173b > ≡

```
c = source_get();
if (c == 'c')
  new_name->comment_flag = 1;
else if (c == '+')
  new_name->comment_flag = 2;
else if (c == 'p')
  new_name->comment_flag = 3;
else
  fprintf(stderr, "%s: Unrecognised comment flag (%s, %d)\n",
          command_name, source_name, source_line);
◇
```

Fragment referenced in [173a](#).

Uses: [command_name 30a](#), [fprintf 22](#), [source_get 127a](#), [source_line 125b](#), [source_name 125b](#), [stderr 22](#).

If there is a pending language tag from a fence or `%%` marker, and the user has not specified explicit comment or debug flags, we apply the language defaults automatically.

```

< Apply pending language if no explicit flags 174a > ≡
  if (pending_lang[0] != '\0') {
    strncpy(new_name->lang, pending_lang, 63);
    new_name->lang[63] = '\0';
    if (new_name->comment_flag == 0 && !new_name->debug_flag) {
      lang_to_flags(pending_lang,
                    &new_name->comment_flag,
                    &new_name->debug_flag,
                    &new_name->location_flag);
    }
    pending_lang[0] = '\0';
  } else if (new_name->lang[0] == '\0') {
    const char *dot = strrchr(new_name->spelling, '.');
    if (dot && dot[1]) {
      const LangEntry *e = find_lang(dot + 1);
      if (e) {
        strncpy(new_name->lang, dot + 1, 63);
        new_name->lang[63] = '\0';
        if (new_name->comment_flag == 0 && !new_name->debug_flag) {
          lang_to_flags(new_name->lang,
                        &new_name->comment_flag,
                        &new_name->debug_flag,
                        &new_name->location_flag);
        }
      }
    }
  }
  if (new_name->suppress_markers) {
    new_name->debug_flag = FALSE;
    new_name->location_flag = FALSE;
  }
}

```

Fragment referenced in [172](#).

Uses: `FALSE` [23a](#), `find_lang` [168](#), `LangEntry` [168](#), `lang_to_flags` [168](#), `pending_lang` [161a](#).

For `@d` (macro) definitions, the language tag is stored and flags are inferred from `@l`, just as for output files. At expansion time, if a fragment carries an explicit `@l` tag its own flags override the parent's—so `@l txt` before `@d` suppresses markers and newlines, enabling true inline expansion.

```

< Apply pending language to macro 174b > ≡
  if (pending_lang[0] != '\0') {
    strncpy(node->lang, pending_lang, 63);
    node->lang[63] = '\0';
    if (node->comment_flag == 0 && !node->debug_flag) {
      lang_to_flags(pending_lang,
                    &node->comment_flag,
                    &node->debug_flag,
                    &node->location_flag);
    }
    pending_lang[0] = '\0';
  }
}

```

Fragment referenced in [177a](#), [178c](#).

Uses: `lang_to_flags` [168](#), `pending_lang` [161a](#).

3.3.5 Comment Styles

weft supports six comment styles, indexed 0–5. The arrays below provide the opening, continuation, and closing delimiters for each. Style 0 means “no comments”; the remaining five cover the vast majority of programming languages:

Index	Style	Begin	Mid	End
0	none	(empty)	(empty)	(empty)
1	C	/*	*	*/
2	C++	//	//	(empty)
3	Shell	#	#	(empty)
4	Lua/SQL	--	--	(empty)
5	HTML	<!--	(spaces)	-->

These are used both for block comments (@c) and for the Knuth-style section markers generated by the language auto-detection system.

```
< Forward declarations for scraps.c 175 > ≡  
  char * comment_begin[6] = { "", "/* ", "// ", "# ", "-- ", "<!-- "};  
  char * comment_mid[6] = { "", " * ", "// ", "# ", "-- ", "    "};  
  char * comment_end[6] = { "", " */", "", "", "", " -->"};  
  ◇
```

Fragment defined by [147b](#), [148](#), [156b](#), [175](#), [193c](#).

Fragment referenced in [132a](#).

Defines: `comment_begin` [53a](#), [150cd](#), [155a](#), `comment_end` [53a](#), [150cd](#), [155a](#), `comment_mid` [53a](#).

Name terminated by \n or @{; but keep skipping until @{

```

"names.c" 176≡
Name *collect_macro_name(void)
{
    char name[MAX_NAME_LEN];
    char args[1000];
    char * arg[9];
    char * argp = args;
    int argc = 0;
    char *p = name;
    int start_line = source_line;
    int c = source_get(), c2;
    unsigned char sector = current_sector;

    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    while (isspace(c))
        c = source_get();
    while (c != EOF) {
        Name * node;
        switch (c) {
            case '\t':
            case ' ': *p++ = ' ';
                        do
                            c = source_get();
                        while (c == ' ' || c == '\t');
                        break;
            case '\n': < Skip until scrap begins, then return name 178c >
            default:
                if (c==nw_char)
                {
                    < Check for terminating at-sequence and return name 177a >
                    break;
                }
                *p++ = c;
                c = source_get();
                break;
        }
    }
    fprintf(stderr, "%s: expected fragment name (%s, %d)\n",
        command_name, source_name, start_line);
    exit(-1);
    return NULL; /* unreachable return to avoid warnings on some compilers */
}
◇

```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Defines: collect_macro_name 47a, 64a, 92b, 161c.

Uses: command_name 30a, current_sector 46c, exit 22, fprintf 22, isspace 22, MAX_NAME_LEN 23b, Name 160b, nw_char 29bc, sector 131c, source_get 127a, source_line 125b, source_name 125b, stderr 22.

⟨ *Check for terminating at-sequence and return name 177a* ⟩ ≡

```
{
    c = source_get();
    switch (c) {
        case '(':
        case '[':
        case '{': ⟨ Cleanup and install name 178b ⟩
                ⟨ Apply pending language to macro 174b ⟩
                return install_args(node, argc, arg);
        case '\\': ⟨ Enter the next argument 177b ⟩
                break;
        default:
            if (c==nw_char)
            {
                *p++ = c;
                break;
            }
            fprintf(stderr,
                "%s: unexpected %c%c in fragment definition name (%s, %d)\n",
                command_name, nw_char, c, source_name, start_line);
            exit(-1);
    }
}◊
```

Fragment referenced in [176](#).

Uses: [command_name 30a](#), [exit 22](#), [fprintf 22](#), [nw_char 29bc](#), [source_get 127a](#), [source_name 125b](#), [stderr 22](#).

⟨ *Enter the next argument 177b* ⟩ ≡

```
arg[argc] = argp;
while ((c = source_get()) != EOF) {
    if (c==nw_char) {
        c2 = source_get();
        if (c2=='\\') {
            ⟨ Make this argument 178a ⟩
            c = source_get();
            break;
        }
        else
            *argp++ = c2;
    }
    else
        *argp++ = c;
}
*p++ = ARG_CHR;
}◊
```

Fragment referenced in [177a](#).

Uses: [ARG_CHR 177c](#), [nw_char 29bc](#), [source_get 127a](#).

⟨ *Type declarations 177c* ⟩ ≡

```
#define ARG_CHR '\\001'
}◊
```

Fragment defined by [23a](#), [54c](#), [122d](#), [160ab](#), [177c](#), [179b](#), [182d](#), [194c](#), [201a](#).

Fragment referenced in [21](#).

Defines: [ARG_CHR 64b](#), [77](#), [79](#), [92c](#), [101](#), [103](#), [145c](#), [156a](#), [157a](#), [177b](#), [181](#), [182b](#).

⟨ *Make this argument 178a* ⟩ ≡

```
if (argc < 9) {
    *argp++ = '\000';
    argc += 1;
}
◇
```

Fragment referenced in [177b](#).

⟨ *Cleanup and install name 178b* ⟩ ≡

```
{
    if (p > name && p[-1] == ' ')
        p--;
    if (p - name > 3 && p[-1] == '.' && p[-2] == '.' && p[-3] == '.') {
        p[-3] = ' ';
        p -= 2;
    }
    if (p == name || name[0] == ' ') {
        fprintf(stderr, "%s: empty name (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
    *p = '\0';
    node = prefix_add(&macro_names, name, sector);
}◇
```

Fragment referenced in [177a](#), [178c](#), [181](#).

Uses: [command_name 30a](#), [exit 22](#), [fprintf 22](#), [macro_names 161a](#), [prefix_add 163a](#), [sector 131c](#), [source_line 125b](#), [source_name 125b](#), [stderr 22](#).

⟨ *Skip until scrap begins, then return name 178c* ⟩ ≡

```
{
    do
        c = source_get();
    while (isspace(c));
    c2 = source_get();
    if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
        fprintf(stderr, "%s: expected %c{ after fragment name (%s, %d)\n",
            command_name, nw_char, source_name, start_line);
        exit(-1);
    }
    ⟨ Cleanup and install name 178b ⟩
    ⟨ Apply pending language to macro 174b ⟩
    return install_args(node, argc, arg);
}◇
```

Fragment referenced in [176](#).

Uses: [command_name 30a](#), [exit 22](#), [fprintf 22](#), [isspace 22](#), [nw_char 29bc](#), [source_get 127a](#), [source_name 125b](#), [stderr 22](#).

⟨ *Function prototypes 178d* ⟩ ≡

```
extern Name *install_args(Name *name, int argc, char *arg[9]);
◇
```

Fragment defined by [42a](#), [58b](#), [71a](#), [73b](#), [85a](#), [87](#), [122ab](#), [125a](#), [127b](#), [132b](#), [134b](#), [142b](#), [145b](#), [157b](#), [161c](#), [164b](#), [178d](#), [188c](#), [195a](#), [196c](#), [199b](#), [201c](#).

Fragment referenced in [21](#).

Uses: [Name 160b](#).

```
"names.c" 179a≡
Name *install_args(Name * name, int argc, char *arg[9])
{
    int i;

    for (i = 0; i < argc; i++) {
        if (name->arg[i] == NULL)
            name->arg[i] = save_string(arg[i]);
    }
    return name;
}
◇
```

File defined by [25c](#), [162ab](#), [163a](#), [164a](#), [165a](#), [166](#), [168](#), [170](#), [171](#), [172](#), [176](#), [179ac](#), [180](#), [183cd](#).
 Uses: Name [160b](#), save_string [162b](#).

```
< Type declarations 179b > ≡
typedef struct arglist
{Name * name;
struct arglist * args;
struct arglist * next;
} Arglist;
◇
```

Fragment defined by [23a](#), [54c](#), [122d](#), [160ab](#), [177c](#), [179b](#), [182d](#), [194c](#), [201a](#).

Fragment referenced in [21](#).

Defines: Arglist [54d](#), [59a](#), [62b](#), [77](#), [79](#), [88a](#), [91b](#), [101](#), [103](#), [132b](#), [141c](#), [143](#), [144b](#), [145abc](#), [146ab](#), [147a](#), [148](#), [154](#), [155c](#), [156b](#), [161c](#), [179c](#), [180](#), [182cd](#), [183b](#), [185b](#), [186a](#), [187a](#), [193b](#).

Uses: Name [160b](#), next [131b](#).

```
"names.c" 179c≡
Arglist * buildArglist(Name * name, Arglist * a)
{
    Arglist * args = (Arglist *)arena_getmem(sizeof(Arglist));

    args->args = a;
    args->next = NULL;
    args->name = name;
    return args;
}
◇
```

File defined by [25c](#), [162ab](#), [163a](#), [164a](#), [165a](#), [166](#), [168](#), [170](#), [171](#), [172](#), [176](#), [179ac](#), [180](#), [183cd](#).

Defines: buildArglist [181](#), [182c](#), [183b](#).

Uses: arena_getmem [202b](#), Arglist [179b](#), Name [160b](#), next [131b](#).

Terminated by @>

```

"names.c" 180≡
Arglist * collect_scrap_name(int current_scrap)
{
    char name[MAX_NAME_LEN];
    char *p = name;
    int c = source_get();
    unsigned char sector = current_sector;
    Arglist * head = NULL;
    Arglist ** tail = &head;

    if (c == '+')
    {
        sector = 0;
        c = source_get();
    }
    while (c == ' ' || c == '\t')
        c = source_get();
    while (c != EOF) {
        switch (c) {
            case '\t':
            case ' ': *p++ = ' ';
                        do
                            c = source_get();
                        while (c == ' ' || c == '\t');
                        break;

            default:
                if (c==nw_char)
                {
                    ⟨ Look for end of scrap name and return 181 ⟩
                    break;
                }
                if (!isgraph(c)) {
                    fprintf(stderr,
                        "%s: unexpected character in fragment name (%s, %d)\n",
                        command_name, source_name, source_line);
                    exit(-1);
                }
                *p++ = c;
                c = source_get();
                break;
        }
    }
    fprintf(stderr, "%s: unexpected end of file (%s, %d)\n",
        command_name, source_name, source_line);
    exit(-1);
    return NULL; /* unreachable return to avoid warnings on some compilers */
}
◇

```

File defined by 25c, 162ab, 163a, 164a, 165a, 166, 168, 170, 171, 172, 176, 179ac, 180, 183cd.

Defines: collect_scrap_name 62b, 77, 91b, 101, 141c, 161c, 183a.

Uses: Arglist 179b, command_name 30a, current_sector 46c, exit 22, fprintf 22, isgraph 22, MAX_NAME_LEN 23b, nw_char 29bc, sector 131c, source_get 127a, source_line 125b, source_name 125b, stderr 22.

⟨ Look for end of scrap name and return 181 ⟩ ≡

```

{
    Name * node;

    c = source_get();
    switch (c) {

        case '\\': {
            ⟨ Add plain string argument 182a ⟩
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
        case '1': case '2': case '3':
        case '4': case '5': case '6':
        case '7': case '8': case '9': {
            ⟨ Add a propagated argument 182b ⟩
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
        case '{': {
            ⟨ Add an inline scrap argument 182c ⟩
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
        case '<':
            ⟨ Add macro call argument 183a ⟩
            *p++ = ARG_CHR;
            c = source_get();
            break;
        case '(':
            scrap_name_has_parameters = 1;
            ⟨ Cleanup and install name 178b ⟩
            return buildArglist(node, head);
        case '>':
            scrap_name_has_parameters = 0;
            ⟨ Cleanup and install name 178b ⟩
            return buildArglist(node, head);

        default:
            if (c==nw_char)
            {
                *p++ = c;
                c = source_get();
                break;
            }
            fprintf(stderr,
                "%s: unexpected %c%c in fragment invocation name (%s, %d)\n",
                command_name, nw_char, c, source_name, source_line);
            exit(-1);
    }
}
}◊

```

Fragment referenced in 180.

Uses: ARG_CHR 177c, buildArglist 179c, command_name 30a, exit 22, fprintf 22, Name 160b, nw_char 29bc, source_get 127a, source_line 125b, source_name 125b, stderr 22.

A plain string argument has no name and a string as a value.

```

< Add plain string argument 182a > ≡
char buff[MAX_NAME_LEN];
char * s = buff;
int c, c2;

while ((c = source_get()) != EOF) {
    if (c==nw_char) {
        c2 = source_get();
        if (c2=='\''')
            break;
        *s++ = c2;
    }
    else
        *s++ = c;
}
*s = '\000';
< Add buff to current arg list 183b >◇

```

Fragment referenced in [181](#).

Uses: MAX_NAME_LEN [23b](#), nw_char [29bc](#), source_get [127a](#).

A parameter argument propagated into an interior fragment has no name and and a string of ARG_CHAR followed by a digit as value.

```

< Add a propagated argument 182b > ≡
char buff[3];
buff[0] = ARG_CHR;
buff[1] = c;
buff[2] = '\000';
< Add buff to current arg list 183b >◇

```

Fragment referenced in [181](#).

Uses: ARG_CHR [177c](#).

```

< Add an inline scrap argument 182c > ≡
int s = collect_scrap();
Scrap_Node * d = (Scrap_Node *)arena_getmem(sizeof(Scrap_Node));
d->scrap = s;
d->quoted = 0;
d->next = NULL;
*tail = buildArglist((Name *)1, (Arglist *)d);
tail = &(*tail)->next;◇

```

Fragment referenced in [181](#).

Uses: arena_getmem [202b](#), Arglist [179b](#), buildArglist [179c](#), collect_scrap [136b](#), Name [160b](#), next [131b](#), Scrap_Node [160a](#).

```

< Type declarations 182d > ≡
typedef struct embed {
    Scrap_Node * defs;
    Arglist * args;
} Embed_Node;
◇

```

Fragment defined by [23a](#), [54c](#), [122d](#), [160ab](#), [177c](#), [179b](#), [182d](#), [194c](#), [201a](#).

Fragment referenced in [21](#).

Defines: Embed_Node [54d](#), [145c](#), [156c](#).

Uses: Arglist [179b](#), defs [131c](#), Scrap_Node [160a](#).

```

⟨ Add macro call argument 183a ⟩ ≡
    *tail = collect_scrap_name(current_scrap);
    if (current_scrap >= 0)
        add_to_use((*tail)->name, current_scrap);
    tail = &(*tail)->next;
    ◇

```

Fragment referenced in [181](#).

Uses: [add_to_use 142a](#), [collect_scrap_name 180](#), next [131b](#).

```

⟨ Add buff to current arg list 183b ⟩ ≡
    *tail = buildArglist(NULL, (Arglist *)save_string(buff));
    tail = &(*tail)->next;
    ◇

```

Fragment referenced in [182ab](#).

Uses: [Arglist 179b](#), [buildArglist 179c](#), next [131b](#), [save_string 162b](#).

```

"names.c" 183c≡
    static Scrap_Node *reverse(Scrap_Node *a); /* a forward declaration */

    void reverse_lists(Name *names)
    {
        while (names) {
            reverse_lists(names->llink);
            names->defs = reverse(names->defs);
            names->uses = reverse(names->uses);
            names = names->rlink;
        }
    }
    ◇

```

File defined by [25c](#), [162ab](#), [163a](#), [164a](#), [165a](#), [166](#), [168](#), [170](#), [171](#), [172](#), [176](#), [179ac](#), [180](#), [183cd](#).

Defines: [reverse_lists 54b](#), [161c](#).

Uses: [defs 131c](#), [Name 160b](#), [reverse 183d](#), [Scrap_Node 160a](#), [uses 131c](#).

Just for fun, here's a non-recursive version of the traditional list reversal code. Note that it reverses the list in place; that is, it does no new allocations.

```

"names.c" 183d≡
    static Scrap_Node *reverse(Scrap_Node *a)
    {
        if (a) {
            Scrap_Node *b = a->next;
            a->next = NULL;
            while (b) {
                Scrap_Node *c = b->next;
                b->next = a;
                a = b;
                b = c;
            }
        }
        return a;
    }
    ◇

```

File defined by [25c](#), [162ab](#), [163a](#), [164a](#), [165a](#), [166](#), [168](#), [170](#), [171](#), [172](#), [176](#), [179ac](#), [180](#), [183cd](#).

Defines: [reverse 28](#), [183c](#).

Uses: next [131b](#), [Scrap_Node 160a](#).

3.4 Searching for Index Entries

Given the array of scraps and a set of index entries, we need to search all the scraps for occurrences of each entry. The obvious approach to this problem would be quite expensive for large documents; however, there is an interesting paper describing an efficient solution [1].

"scraps.c" 184a≡

```
typedef struct name_node {
    struct name_node *next;
    Name *name;
} Name_Node;
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: Name_Node 184b, 190, 191, 192.

Uses: Name 160b, next 131b.

"scraps.c" 184b≡

```
typedef struct goto_node {
    Name_Node *output;           /* list of words ending in this state */
    struct move_node *moves;     /* list of possible moves */
    struct goto_node *fail;      /* and where to go when no move fits */
    struct goto_node *next;      /* next goto node with same depth */
} Goto_Node;
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: Goto_Node 184cd, 185a, 189a, 190, 191, 192.

Uses: Name_Node 184a, next 131b.

"scraps.c" 184c≡

```
typedef struct move_node {
    struct move_node *next;
    Goto_Node *state;
    char c;
} Move_Node;
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: Move_Node 185a, 190, 191.

Uses: Goto_Node 184b, next 131b.

"scraps.c" 184d≡

```
static Goto_Node *root[256];
static int max_depth;
static Goto_Node **depths;
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: depths 189a, 190, 191, max_depth 189a, 190, 191, root 189a, 190, 191, 192, 193c, 194b.

Uses: Goto_Node 184b.

```
"scraps.c" 185a≡
static Goto_Node *goto_lookup(char c, Goto_Node *g)
{
    Move_Node *m = g->moves;
    while (m && m->c != c)
        m = m->next;
    if (m)
        return m->state;
    else
        return NULL;
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: goto_lookup 190, 191, 192.

Uses: Goto_Node 184b, Move_Node 184c, next 131b.

3.4.1 Retrieving scrap uses

```
"scraps.c" 185b≡
typedef struct ArgMgr_s
{
    char * pv;
    char * bgn;
    Arglist * arg;
    struct ArgMgr_s * old;
} ArgMgr;
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: ArgMgr 185c, 186ab, 187c.

Uses: Arglist 179b.

```
"scraps.c" 185c≡
typedef struct ArgManager_s
{
    Manager * m;
    ArgMgr * a;
} ArgManager;
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: ArgManager 186ab, 187c, 189a, 192, 198b.

Uses: ArgMgr 185b, Manager 135c.

```

"scraps.c" 186a≡
static void
pushArglist(ArgManager * mgr, Arglist * a)
{
    ArgMgr * b = malloc(sizeof(ArgMgr));

    if (b == NULL)
    {
        fprintf(stderr, "Can't allocate space for an argument manager\n");
        exit(EXIT_FAILURE);
    }
    b->pv = b->bgn = NULL;
    b->arg = a;
    b->old = mgr->a;
    mgr->a = b;
}
◇

```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: pushArglist 187a, 193b.

Uses: Arglist 179b, ArgManager 185c, ArgMgr 185b, exit 22, fprintf 22, malloc 22, stderr 22.

```

"scraps.c" 186b≡
static char argpop(ArgManager * mgr)
{
    while (mgr->a != NULL)
    {
        ArgMgr * a = mgr->a;

        ⟨ Perhaps —return— a character from the current arg 186c ⟩
        ⟨ Perhaps start a new arg 187a ⟩
        ⟨ Otherwise pop the current arg 187b ⟩
    }

    return (pop(mgr->m));
}
◇

```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: argpop 192, 193ab.

Uses: ArgManager 185c, ArgMgr 185b, pop 142c.

We separate individual arguments using spaces.

```

⟨ Perhaps —return— a character from the current arg 186c ⟩ ≡
    if (a->pv != NULL)
    {
        char c = *a->pv++;

        if (c != '\0')
            return c;
        a->pv = NULL;
        return ' ';
    }
◇

```

Fragment referenced in 186b.

I'm pretty sure that only the first case is ever used. I don't think the others can occur in this context, which makes the whole —ArgManager— thing doubtful.

```

⟨ Perhaps start a new arg 187a ⟩ ≡
    if (a->arg) {
        Arglist * b = a->arg;

        a->arg = b->next;
        if (b->name == NULL) {
            a->bgn = a->pv = (char *)b->args;
        } else if (b->name == (Name *)1) {
            a->bgn = a->pv = "{Embedded Scrap}";
        } else {
            pushArglist(mgr, b->args);
        }
    }

```

Fragment referenced in [186b](#).

Uses: Arglist [179b](#), Name [160b](#), next [131b](#), pushArglist [186a](#).

```

⟨ Otherwise pop the current arg 187b ⟩ ≡
    } else {
        mgr->a = a->old;
        free(a);
    }

```

Fragment referenced in [186b](#).

```

"scraps.c" 187c ≡
    static char
    prev_char(ArgManager * mgr, int n)
    {
        char c = '\0';
        ArgMgr * a = mgr->a;
        Manager * m = mgr->m;

        if (a != NULL) {
            ⟨ Get the nth previous character from an argument 188a ⟩
        } else {
            ⟨ Get the nth previous character from a scrap 188b ⟩
        }

        return c;
    }

```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: prev_char [198b](#).

Uses: ArgManager [185c](#), ArgMgr [185b](#), Manager [135c](#).

⟨ *Get the nth previous character from an argument* 188a ⟩ ≡

```

if (a->pv && a->pv - n >= a->bgn)
    c = *a->pv;
else if (a->bgn) {
    int j = strlen(a->bgn) + 1;

    if (n >= j)
        c = a->bgn[j - n];
    else
        c = ' ';
}
◇

```

Fragment referenced in [187c](#).

Uses: `strlen` [22](#).

When the offset `k` falls within the current slab we index it directly. When `k` is negative we must reach into the previous slab: since `k` is already negative, `SLAB_SIZE + k` yields the correct index (e.g. $k = -1$ gives `chars[1023]`, the last element).

⟨ *Get the nth previous character from a scrap* 188b ⟩ ≡

```

int k = m->index - n - 2;

if (k >= 0)
    c = m->scrap->chars[k];
else if (m->prev)
    c = m->prev->chars[SLAB_SIZE + k];
◇

```

Fragment referenced in [187c](#).

Uses: `SLAB_SIZE` [131b](#).

3.4.2 Building the Automata

⟨ *Function prototypes* 188c ⟩ ≡

```

extern void search(void);
◇

```

Fragment defined by [42a](#), [58b](#), [71a](#), [73b](#), [85a](#), [87](#), [122ab](#), [125a](#), [127b](#), [132b](#), [134b](#), [142b](#), [145b](#), [157b](#), [161c](#), [164b](#), [178d](#), [188c](#), [195a](#), [196c](#), [199b](#), [201c](#).

Fragment referenced in [21](#).

Uses: `search` [189a](#).

```

"scraps.c" 189a≡
static void build_gotos(Name *tree);
static int reject_match(Name *name, char post, ArgManager *reader);

void search(void)
{
    int i;
    for (i=0; i<256; i++)
        root[i] = NULL;
    max_depth = 10;
    depths = (Goto_Node **) arena_getmem(max_depth * sizeof(Goto_Node *));
    for (i=0; i<max_depth; i++)
        depths[i] = NULL;
    build_gotos(user_names);
    ⟨ Build failure functions 191 ⟩
    ⟨ Search scraps 192 ⟩
}
◇

```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: search 32, 42b, 188c, 204.

Uses: arena_getmem 202b, ArgManager 185c, build_gotos 189b, depths 184d, Goto_Node 184b, max_depth 184d, Name 160b, reject_match 198b, root 184d, user_names 161a.

```

"scraps.c" 189b≡
static void build_gotos(Name *tree)
{
    while (tree) {
        ⟨ Extend goto graph with tree->spelling 190 ⟩
        build_gotos(tree->rlink);
        tree = tree->llink;
    }
}
◇

```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: build_gotos 189a.

Uses: Name 160b.

⟨ *Extend goto graph with tree->spelling 190* ⟩ ≡

```
{
    int depth = 2;
    char *p = tree->spelling;
    char c = *p++;
    Goto_Node *q = root[(unsigned char)c];
    Name_Node *last;
    if (!q) {
        q = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
        root[(unsigned char)c] = q;
        q->moves = NULL;
        q->fail = NULL;
        q->output = NULL;
        q->next = depths[1];
        depths[1] = q;
    }
    while ((c = *p++)) {
        Goto_Node *new_node = goto_lookup(c, q);
        if (!new_node) {
            Move_Node *new_move = (Move_Node *) arena_getmem(sizeof(Move_Node));
            new_node = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
            new_node->moves = NULL;
            new_node->fail = NULL;
            new_node->output = NULL;
            new_move->state = new_node;
            new_move->c = c;
            new_move->next = q->moves;
            q->moves = new_move;
            if (depth == max_depth) {
                int i;
                Goto_Node **new_depths =
                    (Goto_Node **) arena_getmem(2*depth*sizeof(Goto_Node *));
                max_depth = 2 * depth;
                for (i=0; i<depth; i++)
                    new_depths[i] = depths[i];
                depths = new_depths;
                for (i=depth; i<max_depth; i++)
                    depths[i] = NULL;
            }
            new_node->next = depths[depth];
            depths[depth] = new_node;
        }
        q = new_node;
        depth++;
    }
    last = q->output;
    q->output = (Name_Node *) arena_getmem(sizeof(Name_Node));
    q->output->next = last;
    q->output->name = tree;
}◊
```

Fragment referenced in [189b](#).

Uses: [arena_getmem 202b](#), [depths 184d](#), [goto_lookup 185a](#), [Goto_Node 184b](#), [max_depth 184d](#), [Move_Node 184c](#), [Name_Node 184a](#), [next 131b](#), [root 184d](#).

$\langle \textit{Build failure functions 191} \rangle \equiv$

```

{
    int depth;
    for (depth=1; depth<max_depth; depth++) {
        Goto_Node *r = depths[depth];
        while (r) {
            Move_Node *m = r->moves;
            while (m) {
                char a = m->c;
                Goto_Node *s = m->state;
                Goto_Node *state = r->fail;
                while (state && !goto_lookup(a, state))
                    state = state->fail;
                if (state)
                    s->fail = goto_lookup(a, state);
                else
                    s->fail = root[(unsigned char)a];
                if (s->fail) {
                    Name_Node *p = s->fail->output;
                    while (p) {
                        Name_Node *q = (Name_Node *) arena_getmem(sizeof(Name_Node));
                        q->name = p->name;
                        q->next = s->output;
                        s->output = q;
                        p = p->next;
                    }
                }
                m = m->next;
            }
            r = r->next;
        }
    }
}
}

```

Fragment referenced in 189a.

Uses: arena_getmem 202b, depths 184d, goto_lookup 185a, Goto_Node 184b, max_depth 184d, Move_Node 184c, Name_Node 184a, next 131b, root 184d.

$$\langle Search\ scraps\ 192 \rangle \equiv$$

```
{
for (i=1; i<scrap; i++) {
    char c, last = '\0';
    Manager rd;
    ArgManager reader;
    Goto_Node *state = NULL;
    rd.prev = NULL;
    rd.scrap = scrap_array(i).slab;
    rd.index = 0;
    reader.m = &rd;
    reader.a = NULL;
    c = argpop(&reader);
    while (c) {
        while (state && !goto_lookup(c, state))
            state = state->fail;
        if (state)
            state = goto_lookup(c, state);
        else
            state = root[(unsigned char)c];
        < Skip over at at 193a>
        < Skip over a scrap use 193b>
        < Skip over a block comment 53d>
        last = c;
        c = argpop(&reader);
        if (state && state->output) {
            Name_Node *p = state->output;
            do {
                Name *name = p->name;
                if (!reject_match(name, c, &reader) &&
                    scrap_array(i).sector == name->sector &&
                    (!name->uses || name->uses->scrap != i)) {
                    Scrap_Node *new_use =
                        (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
                    new_use->scrap = i;
                    new_use->next = name->uses;
                    name->uses = new_use;
                    if (!scrap_is_in(name->defs, i))
                        add_uses(&(scrap_array(i).uses), name);
                }
                p = p->next;
            } while (p);
        }
    }
}
}
```

Uses: [add Uses 194b](#), [arena_getmem 202b](#), [ArgManager 185c](#), [argpop 186b](#), [defs 131c](#), [goto_lookup 185a](#), [Goto_Node 184b](#), [Manager 135c](#), [Name 160b](#), [Name_Node 184a](#), [next 131b](#), [reject_match 198b](#), [root 184d](#), [scraps 132a](#), [scrap_array 132a](#), [scrap_is_in 194a](#), [Scrap_Node 160a](#), [sector 131c](#), [slab 131c](#), [uses 131c](#).

```

⟨ Skip over at at 193a ⟩ ≡
    if (last == nw_char && c == nw_char)
    {
        last = '\0';
        c = argpop(&reader);
    }
    ◇

```

Fragment referenced in 192.

Uses: [argpop 186b](#), [nw_char 29bc](#).

```

⟨ Skip over a scrap use 193b ⟩ ≡
    if (last == nw_char && c == '<')
    {
        char buf[MAX_NAME_LEN];
        char * p = buf;
        Arglist * args;
        int idx;

        c = argpop(&reader);
        while ((c = argpop(&reader)) != nw_char)
            *p++ = c;
        c = argpop(&reader);
        *p = '\0';
        if (sscanf(buf, "%d", &idx) != 1) {
            fprintf(stderr, "%s: found an internal problem (3)\n", command_name);
            exit(-1);
        }
        args = arglist_lookup(idx);
        pushArglist(&reader, args);
    }◇

```

Fragment referenced in 192.

Uses: [Arglist 179b](#), [arglist_lookup 148](#), [argpop 186b](#), [command_name 30a](#), [exit 22](#), [fprintf 22](#), [MAX_NAME_LEN 23b](#), [nw_char 29bc](#), [pushArglist 186a](#), [stderr 22](#).

```

⟨ Forward declarations for scraps.c 193c ⟩ ≡

    static void add_uses(Uses **root, Name *name);
    static int scrap_is_in(Scrap_Node *list, int i);
    ◇

```

Fragment defined by [147b](#), [148](#), [156b](#), [175](#), [193c](#).

Fragment referenced in [132a](#).

Uses: [add_uses 194b](#), [Name 160b](#), [root 184d](#), [scrap_is_in 194a](#), [Scrap_Node 160a](#), [Uses 194c](#).

"scraps.c" 194a≡

```
static int scrap_is_in(Scrap_Node * list, int i)
{
    while (list != NULL) {
        if (list->scrap == i)
            return TRUE;
        list = list->next;
    }
    return FALSE;
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: scrap_is_in 192, 193c.

Uses: FALSE 23a, next 131b, Scrap_Node 160a, TRUE 23a.

"scraps.c" 194b≡

```
static void add_uses(Uses * * root, Name *name)
{
    int cmp;
    Uses *p, **q = root;

    while ((p = *q, p != NULL)
        && (cmp = robs_strcmp(p->defn->spelling, name->spelling)) < 0)
        q = &(p->next);
    if (p == NULL || cmp > 0)
    {
        Uses *new_use = arena_getmem(sizeof(Uses));
        new_use->next = p;
        new_use->defn = name;
        *q = new_use;
    }
}
◇
```

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: add_uses 141ab, 192, 193c.

Uses: arena_getmem 202b, Name 160b, next 131b, robs_strcmp 165a, root 184d, Uses 194c.

⟨ *Type declarations* 194c ⟩ ≡

```
typedef struct uses {
    struct uses *next;
    Name *defn;
} Uses;
◇
```

Fragment defined by 23a, 54c, 122d, 160ab, 177c, 179b, 182d, 194c, 201a.

Fragment referenced in 21.

Defines: Uses 60a, 110ab, 111b, 114, 116, 132b, 133c, 134a, 193c, 194b, 195b, 196d.

Uses: Name 160b, next 131b, uses 131c.

⟨ *Function prototypes 195a* ⟩ ≡

```
extern void format_uses_refs(FILE *, int);
```

◇

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: FILE 22, format_uses_refs 195b.

"scraps.c" 195b≡

```
void
format_uses_refs(FILE * tex_file, int scrap)
{
    Uses * p = scrap_array(scrap).uses;
    if (p != NULL)
        ⟨ Write uses references 195c ⟩
}
```

◇

File defined by 25b, 131bc, 132ac, 133abc, 134acd, 135cd, 136ab, 142ac, 143, 144ab, 145a, 146b, 147a, 158a, 184abcd, 185abc, 186ab, 187c, 189ab, 194ab, 195b, 196d, 198ab, 199a.

Defines: format_uses_refs 63, 64a, 195a.

Uses: FILE 22, scrap_array 132a, Uses 194c, uses 131c.

⟨ *Write uses references 195c* ⟩ ≡

```
{
    char join = ' ';
    fputs("\\item \\NWtxtIdentsUsed\\nobreak\\", tex_file);
    do {
        ⟨ Write one use reference 196a ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".", tex_file);
}◇
```

Fragment referenced in 195b.

Uses: fputs 22, next 131b.

```

⟨ Write one use reference 196a ⟩ ≡
    Name * name = p->defn;
    Scrap_Node *defs = name->defs;
    int first = TRUE, page = -1;
    fprintf(tex_file,
        "%c \\verb%c%s%c\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
    if (defs)
    {
        do {
            ⟨ Write one referenced scrap 196b ⟩
            first = FALSE;
            defs = defs->next;
        }while (defs!= NULL);
    }
    else
    {
        fputs("\\NWnotglobal", tex_file);
    }
}
◇

```

Fragment referenced in 195c.

Uses: defs 131c, FALSE 23a, first 202a, fprintf 22, fputs 22, Name 160b, next 131b, nw_char 29bc, page 131c, Scrap_Node 160a, TRUE 23a.

```

⟨ Write one referenced scrap 196b ⟩ ≡
    fputs("\\NWlink{weft", tex_file);
    write_scrap_ref(tex_file, defs->scrap, -1, &page);
    fputs("}{", tex_file);
    write_scrap_ref(tex_file, defs->scrap, first, &page);
    fputs("}", tex_file);◇

```

Fragment referenced in 196a, 197b.

Uses: defs 131c, first 202a, fputs 22, page 131c, write_scrap_ref 133a.

```

⟨ Function prototypes 196c ⟩ ≡
    extern void format_defs_refs(FILE *, int);
}
◇

```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: FILE 22, format_defs_refs 196d.

"scraps.c" 196d≡

```
void
format_defs_refs(FILE * tex_file, int scrap)
{
    Uses * p = scrap_array(scrap).defs;
    if (p != NULL)
        ⟨ Write defs references 197a ⟩
}
◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: `format_defs_refs` [63](#), [64a](#), [196c](#).

Uses: `defs` [131c](#), `FILE` [22](#), `scrap_array` [132a](#), `Uses` [194c](#).

⟨ Write defs references 197a ⟩ ≡

```
{
    char join = ' ';
    fputs("\\\\item \\NWtxtIdentsDefed\\nobreak\\", tex_file);
    do {
        ⟨ Write one def reference 197b ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".", tex_file);
}◇
```

Fragment referenced in [196d](#).

Uses: `fputs` [22](#), `next` [131b](#).

⟨ Write one def reference 197b ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c \\verb%c%s%c\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
    fputs("\\NWtxtIdentsNotUsed", tex_file);
}
else {
    do {
        if (defs->scrap != scrap) {
            ⟨ Write one referenced scrap 196b ⟩
            first = FALSE;
        }
        defs = defs->next;
    }while (defs!= NULL);
}
◇
```

Fragment referenced in [197a](#).

Uses: `defs` [131c](#), `FALSE` [23a](#), `first` [202a](#), `fprintf` [22](#), `fputs` [22](#), `Name` [160b](#), `next` [131b](#), `nw_char` [29bc](#), `page` [131c](#), `Scrap_Node` [160a](#), `TRUE` [23a](#), `uses` [131c](#).

Rejecting Matches

A problem with simple substring matching is that the string “he” would match longer strings like “she” and “her.” Norman Ramsey suggested examining the characters occurring immediately before and after a match and rejecting the match if it appears to be part of a longer token. Of course, the concept of *token* is language-dependent, so we may be occasionally mistaken. For the present, we’ll consider the mechanism an experiment.

```
"scraps.c" 198a≡
    #define sym_char(c) (isalnum(c) || (c) == '_')

    static int op_char(char c)
    {
        switch (c) {
            case '!':          case '#': case '%': case '$': case '^':
            case '&': case '*': case '-': case '+': case '=': case '/':
            case '|': case '~': case '<': case '>':
                return TRUE;
            default:
                return c==nw_char ? TRUE : FALSE;
        }
    }
◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: `op_char` [198b](#), `sym_char` [198b](#).

Uses: `FALSE` [23a](#), `nw_char` [29bc](#), `TRUE` [23a](#).

```
"scraps.c" 198b≡
    static int reject_match(Name *name, char post, ArgManager *reader)
    {
        int len = strlen(name->spelling);
        char first = name->spelling[0];
        char last = name->spelling[len - 1];
        char prev = prev_char(reader, len);
        if (sym_char(last) && sym_char(post)) return TRUE;
        if (sym_char(first) && sym_char(prev)) return TRUE;
        if (op_char(last) && op_char(post)) return TRUE;
        if (op_char(first) && op_char(prev)) return TRUE;
        return FALSE; /* Here is 198b-01 */
    }
◇
```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

Defines: `reject_match` [189a](#), [192](#).

Uses: `ArgManager` [185c](#), `FALSE` [23a](#), `first` [202a](#), `Name` [160b](#), `op_char` [198a](#), `prev_char` [187c](#), `strlen` [22](#), `sym_char` [198a](#), `TRUE` [23a](#).

3.5 Labels

Refer to [198c-01](#). And another one [198b-01](#).

```

⟨ Get label from 198c ⟩ ≡
    char  label_name[MAX_NAME_LEN];
    char * p = label_name;
    while (c = @1, c != nw_char) /* Here is 198c-01 */
        *p++ = c;
    *p = '\0';
    c = @1;
    ◇

```

Fragment referenced in [75b](#), [139a](#), [152b](#).
 Uses: [MAX_NAME_LEN](#) [23b](#), [nw_char](#) [29bc](#).

```

"scraps.c" 199a≡
    void
    write_label(char label_name[], FILE * file)
    ⟨ Search for label(⟨ Write the label to file 199c ⟩,⟨ Complain about missing label 199d ⟩) 200c ⟩
    ◇

```

File defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#),
[186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).
 Defines: [write_label](#) [75b](#), [152b](#), [199b](#).
 Uses: [FILE](#) [22](#).

```

⟨ Function prototypes 199b ⟩ ≡
    void write_label(char label_name[], FILE * file);
    ◇

```

Fragment defined by [42a](#), [58b](#), [71a](#), [73b](#), [85a](#), [87](#), [122ab](#), [125a](#), [127b](#), [132b](#), [134b](#), [142b](#), [145b](#), [157b](#), [161c](#), [164b](#), [178d](#), [188c](#),
[195a](#), [196c](#), [199b](#), [201c](#).
 Fragment referenced in [21](#).
 Uses: [FILE](#) [22](#), [write_label](#) [199a](#).

```

⟨ Write the label to file 199c ⟩ ≡
    write_single_scrap_ref(file, lbl->scrap);
    fprintf(file, "-%02d", lbl->seq);◇

```

Fragment referenced in [199a](#).
 Uses: [fprintf](#) [22](#), [write_single_scrap_ref](#) [133b](#).

```

⟨ Complain about missing label 199d ⟩ ≡
    fprintf(stderr, "Can't find label %s.\n", label_name);◇

```

Fragment referenced in [199a](#).
 Uses: [fprintf](#) [22](#), [stderr](#) [22](#).

```

⟨ Save label to label store 199e ⟩ ≡
    if (label_name[0])
        ⟨ Search for label(⟨ Complain about duplicate labels 200a ⟩,⟨ Create a new label entry 199f ⟩) 200c ⟩
    else
    {
        ⟨ Complain about empty label 200b ⟩
    }◇

```

Fragment referenced in [139a](#).

```

⟨ Create a new label entry 199f ⟩ ≡
    lbl = (label_node *)arena_getmem(sizeof(label_node) + (p - label_name));
    lbl->left = lbl->right = NULL;
    strcpy(lbl->name, label_name);
    lbl->scrap = current_scrap;
    lbl->seq = ++lblseq;
    *plbl = lbl;◇

```

Fragment referenced in [199e](#).

Uses: [arena_getmem 202b](#), [label_node 201a](#).

```

⟨ Complain about duplicate labels 200a ⟩ ≡
    fprintf(stderr, "Duplicate label %s.\n", label_name);◇

```

Fragment referenced in [199e](#).

Uses: [fprintf 22](#), [stderr 22](#).

```

⟨ Complain about empty label 200b ⟩ ≡
    fprintf(stderr, "Empty label.\n");◇

```

Fragment referenced in [199e](#).

Uses: [fprintf 22](#), [stderr 22](#).

```

⟨ Search for label(Found,Notfound) 200c ⟩ ≡
{
    label_node * * plbl = &label_tab;
    for (;;)
    {
        label_node * lbl = *plbl;

        if (lbl)
        {
            int cmp = label_name[0] - lbl->name[0];

            if (cmp == 0)
                cmp = strcmp(label_name + 1, lbl->name + 1);
            if (cmp < 0)
                plbl = &lbl->left;
            else if (cmp > 0)
                plbl = &lbl->right;
            else
            {
                Found
                break;
            }
        }
        else
        {
            Notfound
            break;
        }
    }
}
◇

```

Fragment referenced in [199ae](#).

Uses: [label_node 201a](#), [label_tab 200d](#).

```
"global.c" 200d≡
    label_node * label_tab = NULL;
    ◇
```

File defined by 25e, 200d.

Defines: `label_tab` 200c, 201b.

Uses: `label_node` 201a.

```
< Type declarations 201a > ≡
    typedef struct l_node
    {
        struct l_node * left, * right;
        int scrap, seq;
        char name[1];
    } label_node;
    ◇
```

Fragment defined by 23a, 54c, 122d, 160ab, 177c, 179b, 182d, 194c, 201a.

Fragment referenced in 21.

Defines: `label_node` 199f, 200cd, 201b.

```
< Global variable declarations 201b > ≡
    extern label_node * label_tab;
    ◇
```

Fragment defined by 28, 29b, 30a, 46c, 51c, 67b, 125b, 135a, 161a, 201b.

Fragment referenced in 21.

Uses: `label_node` 201a, `label_tab` 200d.

3.6 Memory Management

I manage memory using a simple scheme inspired by Hanson's idea of *arenas* [2]. Basically, I allocate all the storage required when processing a source file (primarily for names and scraps) using calls to `arena_getmem(n)`, where `n` specifies the number of bytes to be allocated. When the storage is no longer required, the entire arena is freed with a single call to `arena_free()`. Both operations are quite fast.

```
< Function prototypes 201c > ≡
    extern void *arena_getmem(size_t n);
    extern void arena_free(void);
    extern void arglist_reset(void);
    ◇
```

Fragment defined by 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c.

Fragment referenced in 21.

Uses: `arena_free` 203b, `arena_getmem` 202b, `arglist_reset` 148, `size_t` 22.

```
"arena.c" 201d≡
    typedef struct chunk {
        struct chunk *next;
        char *limit;
        char *avail;
    } Chunk;
    ◇
```

File defined by [25d](#), [201d](#), [202ab](#), [203b](#).

Defines: `Chunk` [202ac](#), [203a](#).

Uses: `next` [131b](#).

We define an empty chunk called `first`. The variable `arena` points at the current chunk of memory; it's initially pointed at `first`. As soon as some storage is required, a “real” chunk of memory will be allocated and attached to `first->next`; storage will be allocated from the new chunk (and later chunks if necessary).

```
"arena.c" 202a≡
    static Chunk first = { NULL, NULL, NULL };
    static Chunk *arena = &first;
    ◇
```

File defined by [25d](#), [201d](#), [202ab](#), [203b](#).

Defines: `arena` [202bc](#), [203ab](#), `first` [32](#), [110c](#), [111a](#), [112](#), [114](#), [115a](#), [117b](#), [132b](#), [133a](#), [196ab](#), [197b](#), [198b](#), [203b](#), [204](#).

Uses: `Chunk` [201d](#).

3.6.1 Allocating Memory

The routine `arena_getmem(n)` returns a pointer to (at least) `n` bytes of memory. Note that `n` is rounded up to ensure that returned pointers are always aligned. We align to the nearest 8-byte segment, since that'll satisfy the more common 2-byte and 4-byte alignment restrictions too.

```
"arena.c" 202b≡
    void *arena_getmem(size_t n)
    {
        char *q;
        char *p = arena->avail;
        n = (n + 7) & ~7;          /* ensuring alignment to 8 bytes */
        q = p + n;
        if (q <= arena->limit) {
            arena->avail = q;
            return p;
        }
        ⟨ Find a new chunk of memory 202c ⟩
    }
    ◇
```

File defined by [25d](#), [201d](#), [202ab](#), [203b](#).

Defines: `arena_getmem` [35b](#), [40](#), [47b](#), [57](#), [132c](#), [135d](#), [136c](#), [141ab](#), [142a](#), [145c](#), [146a](#), [162b](#), [167](#), [179c](#), [182c](#), [189a](#), [190](#), [191](#), [192](#), [194b](#), [199f](#), [201c](#).

Uses: `arena` [202a](#), `size_t` [22](#).

If the current chunk doesn't have adequate space (at least `n` bytes) we examine the rest of the list of chunks (starting at `arena->next`) looking for a chunk with adequate space. If `n` is very large, we may not find it right away or we may not find a suitable chunk at all.

```

⟨ Find a new chunk of memory 202c ⟩ ≡
{
    Chunk *ap = arena;
    Chunk *np = ap->next;
    while (np) {
        char *v = sizeof(Chunk) + (char *) np;
        if (v + n <= np->limit) {
            np->avail = v + n;
            arena = np;
            return v;
        }
        ap = np;
        np = ap->next;
    }
    ⟨ Allocate a new chunk of memory 203a ⟩
}◊

```

Fragment referenced in 202b.

Uses: arena 202a, Chunk 201d, next 131b.

If there isn't a suitable chunk of memory on the free list, then we need to allocate a new one.

```

⟨ Allocate a new chunk of memory 203a ⟩ ≡
{
    size_t m = n + 10000;
    np = (Chunk *) malloc(m);
    np->limit = m + (char *) np;
    np->avail = n + sizeof(Chunk) + (char *) np;
    np->next = NULL;
    ap->next = np;
    arena = np;
    return sizeof(Chunk) + (char *) np;
}◊

```

Fragment referenced in 202c.

Uses: arena 202a, Chunk 201d, malloc 22, next 131b, size_t 22.

3.6.2 Freeing Memory

To free all the memory in the arena, we need only point `arena` back to the first empty chunk.

```

"arena.c" 203b ≡
void arena_free(void)
{
    arena = &first;
    arglist_reset();
}
◊

```

File defined by 25d, 201d, 202ab, 203b.

Defines: arena_free 41, 201c.

Uses: arena 202a, arglist_reset 148, first 202a.

Chapter 4

Man page

Here is the UNIX man page for weft:

```
"weft.1" 204≡
.TH WEFT 1 "March 2026"
.SH NAME
weft, a literate programming tool
.SH SYNOPSIS
.B weft
.br
\fBweft\fP [options] [file] ...
.SH DESCRIPTION
.I weft
is a literate programming tool like Knuth's
.I WEB,
only simpler.
A
.I weft
file contains program source code interleaved with documentation.
When
.I weft
is given a
.I weft
file, it writes the program file(s).
By default only tangling is performed. Weaving (documentation
generation) is requested with \fB-w\fP.
.SH COMMAND LINE OPTIONS
.br
\fB-t\fP Suppresses weave output (even if \fB-w\fP is given).
.br
\fB-o\fP Suppresses generation of the output files.
.br
\fB-d\fP List dangling identifier references in indexes.
.br
\fB-c\fP Forces output files to overwrite old files of the same
name without comparing for equality first.
.br
\fB-v\fP The verbose flag. Forces output of progress reports.
.br
\fB-n\fP Forces sequential numbering of scraps (instead of page
numbers).
.br
\fB-s\fP Doesn't print list of scraps making up file at end of
```

each scrap.

`\fB-p path\fP` Prepend path to the filenames for all the output files.

`\fB-V string\fP` Provide the string for replacement of the @v operation. This is intended as a means for including version information in generated output.

`\fB-x\fP` Include cross-references in comments in output files.

`\fB-h options\fP` Turn on hyperlinks using the hyperref package of LaTeX and provide the options to the package.

`\fB-r\fP` Turn on hyperlinks using the hyperref package of LaTeX, with the package options being in the text.

`\fB-I path\fP` Provide a directory to search for included files. This may appear several times.

.br

`\fB-m\fP` Emit a JSON map of all output files, fragments, their definitions, and dependency relationships to stdout. No weave or tangle is performed. Useful for AI tooling and project navigation.

.br

`\fB-e name\fP` Extract the named fragment and all its transitive dependencies as Markdown to stdout. Fragment names support prefix matching. No weave or tangle is performed.

.br

`\fB-R file[:line]\fP` Read section markers from a tangled output file and emit JSON mapping to stdout. If a line number is given, emit the single innermost scrap covering that line. Without a line number, emit all scrap regions. No .weft source files are needed.

.br

`\fB-w [format]\fP` Request weave output. Without a format token, the format is taken from the `\fB@W\fP` directive in the source file. With `\fBmd\fP`, a `\fB.md\fP` file is generated; with `\fBtex\fP`, a `\fB.tex\fP` file is generated. A CLI format overrides `\fB@W\fP`. If `\fB-w\fP` is given without a format and no `\fB@W\fP` exists, weft exits with an error.

.br

`\fB--help\fP` Print a comprehensive help message including all options, usage examples, and an AI-assisted workflow guide, then exit.

.SH FORMAT OF WEFT FILES

A

.I weft

file contains mostly ordinary

.I LaTeX.

The file is read and copied to output (.tex file) unless a

.I weft

command is encountered. All

.I weft

commands start with an ‘‘at-sign’’ (@).

Files and fragments are defined with the following commands:

.PP

@o \fIfile-name flags scrap\fP where scrap is smaller than one page.

.br

@O \fIfile-name flags scrap\fP where scrap is bigger than one page.

.br

@d \fIfragment-name scrap\fP. Where scrap is smaller than one page.

.br

@D \fIfragment-name scrap\fP. Where scrap is bigger than one page.

.PP

@q \fIfragment-name scrap\fP. Where scrap is smaller than one page.

The scrap is not expanded in the output, allowing you to construct

output files which can, perhaps after further processing, be input to

```
.I weft.
```

```
.br
```

@Q \fIfragment-name scrap\fP. Where scrap is bigger than one page.
Likewise.

```
.PP
```

Scraps have specific begin and end markers;
which begin and end marker you use determines how the scrap will be typeset in the .tex file:

```
.br
```

\fB@{\fP...\fB@}\fP for verbatim "terminal style" formatting or, with the -l flag, LaTeX listing package.

```
.br
```

\fB@[\fP...\fB@]\fP for LaTeX paragraph mode formatting, and

```
.br
```

\fB@(\fP...\fB@)\fP for LaTeX math mode formatting.

```
.br
```

Any amount of whitespace (including carriage returns) may appear between a name and the beginning of a scrap.

```
.PP
```

Several code/file scraps may have the same name;

```
.I weft
```

concatenates their definitions to produce a single scrap.
Code scrap definitions are like macro definitions;

```
.I weft
```

extracts a program by expanding one scrap.
The definition of that scrap contains references to other scraps, which are themselves expanded, and so on.
\fIleft\fP's output is readable; it preserves the indentation of expanded scraps with respect to the scraps in which they appear.

```
.PP
```

.SH PER FILE OPTIONS

When defining an output file, the programmer has the option of using flags to control the output.

```
.PP
```

\fB-d\fR option,

```
.I weft
```

will emit line number indications at scrap boundaries.

```
.br
```

\fB-i\fR option,

```
.I weft
```

suppresses the indentation of fragments (useful for \fBFortran\fR).

```
.br
```

\fB-t\fP option makes \fIleft\fP copy tabs untouched from input to output.

```
.br
```

\fB-c\fIx\fP Include comments in the output file.
\fIx\fP may be \fBc\fP for C-style comments, \fB+\fP for C++ and \fBp\fP for perl and similar.

```
.br
```

\fB-s\fP Suppress section markers and \fB#line\fP directives in the tangled output for this file. Language auto-detection still sets the comment style and the \fBlang\fP tag (used for Markdown syntax highlighting), but no markers appear in the output.

```
.PP
```

.SH MINOR COMMANDS

```
.br
@@ Causes a single ‘‘at-sign’’ to be copied into the output.
.br
@\_ Causes the text between it and the next {\tt @\_} to be made bold
      (for keywords, etc.) in the formatted document
.br
@% Comments out a line so that it doesn’t appear in the output.
.br
@i \fBfilename\fR causes the file named to be included.
.br
@f Creates an index of output files.
.br
@m Creates an index of fragments.
.br
@u Creates an index of user-specified identifiers.
.br
@W \fIformat\fP Declares the weave output format (\fBmd\fP or \fBtex\fP).
Used by \fB-w\fP when no explicit format is given on the command line.
.br
@l \fItag\fP Sets the language for the next @o or @d directive.
The tag is looked up in the language table to determine the comment
style for section markers and the \fBclass\fP attribute for Markdown
syntax highlighting. Place it on the line before the directive it
applies to.
.br
@L \fIname style\fP [+d] Defines a custom language mapping.
\fIname\fP is the tag (e.g. \fBnim\fP, \fBgls\fP),
\fIstyle\fP is one of \fB//\fP, \fB#\fP, \fB--\fP, \fB*\fP, or \fB<!--\fP.
The optional \fB+d\fP flag enables \fB#line\fP directive generation.
User-defined languages override built-in entries. Must appear in the
document preamble before any @o or @d that depends on it.
.PP
.SH SECTION MARKERS
By default, when a language is detected (via @l, @L, or file extension
of @o), weft wraps each scrap in the tangled output with opening and
closing section markers using the appropriate comment syntax. For example,
C output includes:
.nf

/* {1: source.w:10} */
int main() { return 0; }
/* {:1} */

.fi
The opening marker shows the scrap number and source location; the closing
marker shows only the scrap number. Nested fragment expansions produce
nested markers.
.PP
Languages with \fB#line\fP support (C, C++, Objective-C, etc.) also
emit \fB#line\fP directives at scrap boundaries.
.PP
The \fB-s\fP per-file flag suppresses both section markers and \fB#line\fP
directives.
.PP
.SH LANGUAGE INFERENCE
When no @l tag precedes an @o directive, weft infers the language
from the output file’s extension. For example, \fB@o app.js\fP
automatically detects JavaScript (// comments, no #line). The inferred
```

```

language is used for section markers in the tangle and for the
\fbclass="language-X"\fP attribute in the Markdown weave.
.PP
Over 250 file extensions are recognised, covering C, C++, Python,
JavaScript, Go, Rust, Ruby, Lua, SQL, HTML, and many more.
Extensions not in the table produce no markers.
.PP
To mark an identifier for inclusion in the index, it must be mentioned
at the end of the scrap it was defined in. The line starts
with @| and ends with the \fbend of scrap\fP mark \fb@}\fP.
.PP
.SH ERROR MESSAGES
.PP
.SH BUGS
.PP
.SH AUTHOR
Preston Briggs.
Internet address \fbpreston@cs.rice.edu\fP.
.SH MAINTAINER
Simon Wright
Internet address \fbsimon@pushface.org\fP
.br
Keith Harwood
Internet address \fbKeith.Harwood@vitalmis.com\fP
◇

```

Uses: [exit 22](#), [FILE 22](#), first [202a](#), main [26a](#), next [131b](#), page [131c](#), scraps [132a](#), search [189a](#).

Chapter 5

Indices

Three sets of indices can be created automatically: an index of file names, an index of fragment names, and an index of user-specified identifiers. An index entry includes the name of the entry, where it was defined, and where it was referenced.

5.1 Files

"arena.c" Defined by [25d](#), [201d](#), [202ab](#), [203b](#).

"global.c" Defined by [25e](#), [200d](#).

"global.h" Defined by [21](#).

"input.c" Defined by [25a](#), [126ab](#), [127ac](#), [131a](#).

"latex.c" Defined by [24b](#), [59ab](#), [65a](#), [69b](#), [70ab](#), [71bc](#), [72](#), [73a](#), [79](#), [80b](#), [82ab](#), [85b](#), [86a](#).

"main.c" Defined by [23c](#), [26a](#).

"markdown.c" Defined by [24c](#), [88ab](#), [93a](#), [96](#), [97a](#), [102](#), [103](#), [104b](#), [105b](#), [106a](#), [108b](#), [110a](#), [111b](#).

"names.c" Defined by [25c](#), [162ab](#), [163a](#), [164a](#), [165a](#), [166](#), [168](#), [170](#), [171](#), [172](#), [176](#), [179ac](#), [180](#), [183cd](#).

"output.c" Defined by [24d](#), [113](#), [114](#), [115ab](#), [116](#), [117ab](#), [118](#), [119](#), [120](#), [121](#), [122c](#).

"pass1.c" Defined by [24a](#), [42b](#).

"scraps.c" Defined by [25b](#), [131bc](#), [132ac](#), [133abc](#), [134acd](#), [135cd](#), [136ab](#), [142ac](#), [143](#), [144ab](#), [145a](#), [146b](#), [147a](#), [158a](#), [184abcd](#), [185abc](#), [186ab](#), [187c](#), [189ab](#), [194ab](#), [195b](#), [196d](#), [198ab](#), [199a](#).

"weft.1" Defined by [204](#).

5.2 Fragments

⟨ Accumulate scrap and return `scraps++` [137](#) ⟩ Referenced in [136b](#).

⟨ Add a propagated argument [182b](#) ⟩ Referenced in [181](#).

⟨ Add an at character to the block or break [50a](#) ⟩ Referenced in [49b](#).

⟨ Add an inline scrap argument [182c](#) ⟩ Referenced in [181](#).

⟨ Add any other character to the block [50b](#) ⟩ Referenced in [49b](#).

⟨ Add buff to current arg list [183b](#) ⟩ Referenced in [182ab](#).

⟨ Add letters to scraps with duplicate page numbers [159a](#) ⟩ Referenced in [158a](#).

⟨ Add macro call argument [183a](#) ⟩ Referenced in [181](#).

⟨ Add more indentation *char* [150a](#) ⟩ Referenced in [149](#), [151d](#), [152a](#).

⟨ Add one char to the block buffer [49b](#) ⟩ Referenced in [48b](#).

⟨ Add plain string argument [182a](#) ⟩ Referenced in [181](#).

⟨ Add the source path to the include path list [40](#) ⟩ Referenced in [39](#).

⟨ Add user identifier definition [141b](#) ⟩ Referenced in [140](#).

⟨ Add user identifier use [141a](#) ⟩ Referenced in [140](#).

⟨ Add `scrap` to `name`'s definition list [47b](#) ⟩ Referenced in [46e](#), [47a](#).

⟨ Allocate a new chunk of memory [203a](#) ⟩ Referenced in [202c](#).

⟨ Apply pending language if no explicit flags [174a](#) ⟩ Referenced in [172](#).

⟨ Apply pending language to macro [174b](#) ⟩ Referenced in [177a](#), [178c](#).

<Begin or end a block comment 54a> Referenced in 61a.
 <Begin the cross-reference environment 68a> Referenced in 63, 64a.
 <Begin the scrap environment 65b> Referenced in 63, 64a.
 <Bold Keyword 76a> Referenced in 74a.
 <Bold md Keyword 100b> Referenced in 99a.
 <Build a new arglist 146a> Referenced in 145a.
 <Build failure functions 191> Referenced in 189a.
 <Build fragment definition 47a> Referenced in 45.
 <Build output file definition 46e> Referenced in 45.
 <Build `source_name` and `tex_name` 39> Referenced in 38.
 <Check at-sequence for end-of-scrap 74a> Referenced in 72.
 <Check for ambiguous prefix 163b> Referenced in 163a.
 <Check for cosmetic fence line 89b> Referenced in 89a.
 <Check for end of scrap name 146c> Referenced in 146b.
 <Check for long options 31> Referenced in 30d.
 <Check for macro invocation in scrap 152a> Referenced in 149.
 <Check for macro parameters 57> Referenced in 146c.
 <Check for terminating at-sequence and return name 177a> Referenced in 176.
 <Check md at-sequence for end-of-scrap 99a> Referenced in 97a.
 <Cleanup and install name 178b> Referenced in 177a, 178c, 181.
 <Close the current sector 46b> Referenced in 45, 61a, 90a.
 <Collect a block comment 48b> Referenced in 45.
 <Collect include-file name 130a> Referenced in 129.
 <Collect language apply in pass1 43a> Referenced in 45.
 <Collect user language definition in pass1 43b> Referenced in 45.
 <Collect user-specified index entries 140> Referenced in 138.
 <Collect weave format declaration in pass1 43c> Referenced in 45.
 <Comment this macro use 156a> Referenced in 154, 155ac.
 <Compare the temp file and the old file 124b> Referenced in 124a.
 <Complain about duplicate labels 200a> Referenced in 199e.
 <Complain about empty label 200b> Referenced in 199e.
 <Complain about missing label 199d> Referenced in 199a.
 <Copy block comment from scrap 53a> Referenced in 152a.
 <Copy file name into file 153> Referenced in 152a.
 <Copy fragment title into file 155c> Referenced in 152a.
 <Copy label from scrap into file 152b> Referenced in 152a.
 <Copy label from source into 75b> Referenced in 61a, 74a, 90a, 99a.
 <Copy macro into file 154> Referenced in 152a.
 <Copy version info into file 75a> Referenced in 74a, 99a, 152a.
 <Copy version info into md file 91a> Referenced in 90a.
 <Copy version info into tex file 62a> Referenced in 61a.
 <Copy `defs->scrap` to file 149> Referenced in 147a.
 <Copy `source_file` into `md_file` 89a> Referenced in 88b.
 <Copy `source_file` into `tex_file` 60c> Referenced in 59b.
 <Create a new label entry 199f> Referenced in 199e.
 <Create new name entry 167> Referenced in 163a, 166.
 <Create new scrap, managed by `writer` 136c> Referenced in 136b.
 <End block 66c> Referenced in 54a, 65b, 67a.
 <Enter the next argument 177b> Referenced in 177a.
 <Expand macro into file 62b> Referenced in 61a.
 <Expand md macro into file 91b> Referenced in 90a.
 <Expand md tab into spaces 98> Referenced in 97a.
 <Expand tab into spaces 73c> Referenced in 72, 151d.
 <Extend goto graph with `tree->spelling` 190> Referenced in 189b.
 <Fill in the md scrap 93b> Referenced in 92ab.
 <Fill in the middle of the scrap environment 66d> Referenced in 63, 64a.
 <Find a free temporary file 123b> Referenced in 123a.
 <Find a new chunk of memory 202c> Referenced in 202b.

< Finish md scrap 93c > Referenced in 92ab.
 < Finish the cross-reference environment 68b > Referenced in 63, 64a.
 < Finish the scrap environment 67a > Referenced in 63, 64a.
 < For all remaining scraps 159c > Referenced in 159a.
 < Format a file index entry 80c > Referenced in 80b.
 < Format a md file index entry 104c > Referenced in 104b.
 < Format a md index entry 106b > Referenced in 106a.
 < Format a md user index entry 108c > Referenced in 108b.
 < Format a user index entry 86b > Referenced in 86a.
 < Format an index entry 83b > Referenced in 82b.
 < Format macro name 77 > Referenced in 74a.
 < Format macro parameters 58a > Referenced in 77, 79.
 < Format md macro name 101 > Referenced in 99a.
 < Forward declarations for scraps.c 147b, 148, 156b, 175, 193c > Referenced in 132a.
 < Function prototypes 42a, 58b, 71a, 73b, 85a, 87, 122ab, 125a, 127b, 132b, 134b, 142b, 145b, 157b, 161c, 164b, 178d, 188c, 195a, 196c, 199b, 201c > Referenced in 21.
 < Get comment delimiters 173b > Referenced in 173a.
 < Get label from 198c > Referenced in 75b, 139a, 152b.
 < Get label while collecting scrap 139a > Referenced in 138.
 < Get the nth previous character from a scrap 188b > Referenced in 187c.
 < Get the nth previous character from an argument 188a > Referenced in 187c.
 < Global variable declarations 28, 29b, 30a, 46c, 51c, 67b, 125b, 135a, 161a, 201b > Referenced in 21.
 < Global variable definitions 29ac, 30b, 46d, 51d, 67c, 125c, 135b, 161b > Referenced in 25e.
 < Handle an “at” character 128 > Referenced in 127a.
 < Handle at-sign during scrap accumulation 138 > Referenced in 137.
 < Handle macro invocation in scrap 141c > Referenced in 138.
 < Handle macro parameter substitution 54d > Referenced in 152a.
 < Handle optional per-file flags 173a > Referenced in 172.
 < Handle reverse map mode 36d > Referenced in 26a.
 < Handle tab characters on output 151d > Referenced in 149.
 < Handle the file name in `argv[arg]` 38 > Referenced in 37b.
 < Handle EOF 130b > Referenced in 127a.
 < HTML-escape character c 97b > Referenced in 97a.
 < If we break the line at this word 53c > Referenced in 53a.
 < Include a fragment use in comment 157a > Referenced in 156b.
 < Include an embedded scrap in comment 156c > Referenced in 156b.
 < Include block comment in a scrap 52b > Referenced in 138.
 < Include files 22 > Referenced in 21.
 < Indent suppressed 151c > Referenced in 151a.
 < Insert appropriate indentation 151a > Referenced in 149, 150b.
 < Insert debugging information if required 150b > Referenced in 149, 152a.
 < Insert section closing marker 150d > Referenced in 149.
 < Insert section opening marker 150c > Referenced in 149.
 < Interpret at-sequence 61a > Referenced in 60c.
 < Interpret command-line arguments 30cd > Referenced in 26a.
 < Interpret md at-sequence 90a > Referenced in 89a.
 < Interpret the argument string `s` 34 > Referenced in 30d.
 < Italic “*whatever*” 76b > Referenced in 74a.
 < Italic md “*whatever*” 100c > Referenced in 99a.
 < Limits 23b > Referenced in 21.
 < Look for end of scrap name and return 181 > Referenced in 180.
 < Make this argument 178a > Referenced in 177b.
 < Move a word to the file 53b > Referenced in 53a.
 < Move the temporary file to the target, if required 124a > Referenced in 123a.
 < Open an include file 129 > Referenced in 128.
 < Operating System Dependencies 26b > Referenced in 21, 25e.
 < Otherwise pop the current arg 187b > Referenced in 186b.
 < Perhaps add an include path 35b > Referenced in 30d.

<Perhaps add letters to the page numbers 159d> Referenced in 159a.
 <Perhaps comment this macro 155a> Referenced in 54d, 154.
 <Perhaps get the extract name 36b> Referenced in 30d.
 <Perhaps get the hyperref options 36a> Referenced in 30d.
 <Perhaps get the prepend path 35a> Referenced in 30d.
 <Perhaps get the reverse map argument 36c> Referenced in 30d.
 <Perhaps get the version info string 35c> Referenced in 30d.
 <Perhaps put a delayed indent 155b> Referenced in 53a, 155a.
 <Perhaps —return— a character from the current arg 186c> Referenced in 186b.
 <Perhaps skip white-space 51a> Referenced in 50b.
 <Perhaps start a new arg 187a> Referenced in 186b.
 <Print help message 32> Referenced in 31.
 <Process a file 41> Referenced in 38.
 <Process the remaining arguments (file names) 37b> Referenced in 26a.
 <Put out the indent 151b> Referenced in 54d, 151a.
 <Read line in .aux file 158b> Referenced in 158a.
 <Rename the temporary file to the target 124c> Referenced in 124ab.
 <Reverse cross-reference lists 54b> Referenced in 42b.
 <Rob’s ordering 82c> Referenced in 82b, 106a.
 <Save label to label store 199e> Referenced in 139a.
 <Save macro name 141d> Referenced in 141c.
 <Save macro parameters 56> Referenced in 141c.
 <Scan at-sequence 45> Referenced in 42c.
 <Scan the source file, looking for at-sequences 42c> Referenced in 42b.
 <Search for label(*Found,Notfound*) 200c> Referenced in 199ae.
 <Search scraps 192> Referenced in 189a.
 <Set locale information 37a> Referenced in 26a.
 <Set up name, args and next 145c> Referenced in 145a.
 <Show md presence of a block comment 100a> Referenced in 99a.
 <Show presence of a block comment 52a> Referenced in 74a.
 <Skip at-L language definition line 44> Referenced in 61a, 90a.
 <Skip commented-out code 75e> Referenced in 74a, 99a, 138.
 <Skip invisibles on *p* 165b> Referenced in 165a.
 <Skip over a block comment 53d> Referenced in 192.
 <Skip over a scrap use 193b> Referenced in 192.
 <Skip over an in-text scrap 47c> Referenced in 45.
 <Skip over at at 193a> Referenced in 192.
 <Skip over at-sign or go to skipped 48a> Referenced in 47c.
 <Skip over index entries 75d> Referenced in 74a, 99a.
 <Skip to the next nw-char 51b> Referenced in 48b.
 <Skip until scrap begins, then return name 178c> Referenced in 176.
 <Skip whitespace 49a> Referenced in 48b.
 <Sort *key* of size *n* for *ordering* 83a> Referenced in 82b, 106a.
 <Start block 66a> Referenced in 54a, 65b.
 <Step *i* to the next valid scrap 159b> Referenced in 159a.
 <Step to next sector 46a> Referenced in 45, 61a, 90a.
 <Switch block 66b> Referenced in 65b.
 <Type declarations 23a, 54c, 122d, 160ab, 177c, 179b, 182d, 194c, 201a> Referenced in 21.
 <Warn (only once) about needing to rerun after Latex 134e> Referenced in 133a, 158b.
 <Write abbreviated definition list 78> Referenced in 77, 79.
 <Write defining scrap numbers 84a> Referenced in 83b.
 <Write defs references 197a> Referenced in 196d.
 <Write file defs 68c> Referenced in 63.
 <Write file’s defining scrap numbers 81a> Referenced in 80c.
 <Write in-text scrap 67d> Referenced in 61a.
 <Write index of file names 80a> Referenced in 61a.
 <Write index of macro names 81b> Referenced in 61a.
 <Write index of user-specified names 85c> Referenced in 61a.

< Write LaTeX limbo definitions [60a](#) > Referenced in [59b](#).
 < Write macro definition [64a](#) > Referenced in [61a](#).
 < Write macro defs [68d](#) > Referenced in [64a](#).
 < Write macro refs [69a](#) > Referenced in [64a](#).
 < Write md defining scrap numbers [107a](#) > Referenced in [106b](#).
 < Write md defs references [111c](#) > Referenced in [111b](#).
 < Write md file cross-refs [94b](#) > Referenced in [92a](#).
 < Write md file defs [94c](#) > Referenced in [94b](#).
 < Write md file's defining scrap numbers [104d](#) > Referenced in [104c](#).
 < Write md in-text scrap [94a](#) > Referenced in [90a](#).
 < Write md index of file names [104a](#) > Referenced in [90a](#).
 < Write md index of macro names [105a](#) > Referenced in [90a](#).
 < Write md index of user-specified names [108a](#) > Referenced in [90a](#).
 < Write md macro cross-refs [94d](#) > Referenced in [92b](#).
 < Write md macro definition [92b](#) > Referenced in [90a](#).
 < Write md macro defs [95a](#) > Referenced in [94d](#).
 < Write md macro refs [95b](#) > Referenced in [94d](#).
 < Write md output file definition [92a](#) > Referenced in [90a](#).
 < Write md referencing scrap numbers [107b](#) > Referenced in [106b](#).
 < Write md uses references [110b](#) > Referenced in [110a](#).
 < Write one def reference [197b](#) > Referenced in [197a](#).
 < Write one md def reference [112](#) > Referenced in [111c](#).
 < Write one md referenced scrap [111a](#) > Referenced in [110c](#), [112](#).
 < Write one md use reference [110c](#) > Referenced in [110b](#).
 < Write one referenced scrap [196b](#) > Referenced in [196a](#), [197b](#).
 < Write one use reference [196a](#) > Referenced in [195c](#).
 < Write out files->spelling [123a](#) > Referenced in [122c](#).
 < Write output file definition [63](#) > Referenced in [61a](#).
 < Write referencing scrap numbers [84b](#) > Referenced in [83b](#).
 < Write the hyperlink usage macro [60b](#) > Referenced in [60a](#).
 < Write the label to file [199c](#) > Referenced in [199a](#).
 < Write the macro's name [64b](#) > Referenced in [64a](#), [83b](#).
 < Write the md macro's name [92c](#) > Referenced in [92b](#), [106b](#).
 < Write uses references [195c](#) > Referenced in [195b](#).

5.3 Identifiers

Knuth prints his index of identifiers in a two-column format. I could force this automatically by emitting the `\twocolumn` command; but this has the side effect of forcing a new page. Therefore, it seems better to leave it this up to the user.

add_to_use: [56](#), [141c](#), [142a](#), [142b](#), [183a](#).
 add_uses: [141ab](#), [192](#), [193c](#), [194b](#).
 already_warned: [134e](#), [135a](#), [135b](#), [158a](#).
 arena: [202a](#), [202bc](#), [203ab](#).
 arena_free: [41](#), [201c](#), [203b](#).
 arena_getmem: [35b](#), [40](#), [47b](#), [57](#), [132c](#), [135d](#), [136c](#), [141ab](#), [142a](#), [145c](#), [146a](#), [162b](#), [167](#), [179c](#), [182c](#), [189a](#), [190](#), [191](#),
[192](#), [194b](#), [199f](#), [201c](#), [202b](#).
 Arglist: [54d](#), [59a](#), [62b](#), [77](#), [79](#), [88a](#), [91b](#), [101](#), [103](#), [132b](#), [141c](#), [143](#), [144b](#), [145abc](#), [146ab](#), [147a](#), [148](#), [154](#), [155c](#), [156b](#),
[161c](#), [179b](#), [179c](#), [180](#), [182cd](#), [183b](#), [185b](#), [186a](#), [187a](#), [193b](#).
 arglist_count: [148](#).
 arglist_lookup: [143](#), [146b](#), [148](#), [193b](#).
 arglist_register: [141d](#), [148](#).
 arglist_reset: [148](#), [201c](#), [203b](#).
 arglist_table: [148](#).
 ArgManager: [185c](#), [186ab](#), [187c](#), [189a](#), [192](#), [198b](#).
 ArgMgr: [185b](#), [185c](#), [186ab](#), [187c](#).
 argpop: [186b](#), [192](#), [193ab](#).

ARG_CHR: [64b](#), [77](#), [79](#), [92c](#), [101](#), [103](#), [145c](#), [156a](#), [157a](#), [177b](#), [177c](#), [181](#), [182b](#).
 backup: [144a](#), [151a](#).
 blockBuff: [48b](#), [51c](#), [51d](#), [52b](#).
 buildArglist: [179c](#), [181](#), [182c](#), [183b](#).
 build_gotos: [189a](#), [189b](#).
 Chunk: [201d](#), [202ac](#), [203a](#).
 clear_marks: [117a](#), [117b](#).
 collect_file_name: [46e](#), [63](#), [92a](#), [161c](#), [172](#).
 collect_lang_def: [43b](#), [161c](#), [170](#).
 collect_macro_name: [47a](#), [64a](#), [92b](#), [161c](#), [176](#).
 collect_numbers: [41](#), [157b](#), [158a](#).
 collect_scrap: [46e](#), [47a](#), [56](#), [132b](#), [136b](#), [182c](#).
 collect_scrap_name: [62b](#), [77](#), [91b](#), [101](#), [141c](#), [161c](#), [180](#), [183a](#).
 collect_weave_format: [43c](#), [161c](#), [171](#).
 command_name: [30a](#), [30bc](#), [32](#), [34](#), [37b](#), [41](#), [45](#), [47c](#), [48a](#), [59b](#), [69a](#), [77](#), [79](#), [88b](#), [95b](#), [101](#), [103](#), [117b](#), [123b](#), [124c](#), [128](#), [129](#), [130a](#), [131a](#), [134e](#), [137](#), [138](#), [140](#), [146b](#), [148](#), [154](#), [163b](#), [170](#), [171](#), [172](#), [173ab](#), [176](#), [177a](#), [178bc](#), [180](#), [181](#), [193b](#).
 comment_ArglistElement: [156a](#), [156b](#), [157a](#).
 comment_begin: [53a](#), [150cd](#), [155a](#), [175](#).
 comment_end: [53a](#), [150cd](#), [155a](#), [175](#).
 comment_mid: [53a](#), [175](#).
 compare: [162a](#), [163a](#), [164a](#).
 compare_flag: [28](#), [29a](#), [34](#), [124a](#).
 copy_scrap: [58a](#), [59a](#), [66d](#), [67d](#), [72](#).
 current_sector: [41](#), [46ab](#), [46c](#), [46d](#), [81b](#), [85c](#), [105a](#), [108a](#), [136c](#), [140](#), [176](#), [180](#).
 dangling_flag: [28](#), [29a](#), [34](#), [86b](#), [108c](#).
 defs: [47b](#), [54d](#), [62b](#), [68cd](#), [77](#), [78](#), [79](#), [81a](#), [84a](#), [86b](#), [91b](#), [94c](#), [95a](#), [101](#), [103](#), [104d](#), [107a](#), [108c](#), [110c](#), [111a](#), [112](#), [114](#), [116](#), [123a](#), [131c](#), [132b](#), [134a](#), [136c](#), [141b](#), [145c](#), [147a](#), [149](#), [150bcd](#), [153](#), [154](#), [155ac](#), [156ac](#), [160b](#), [167](#), [182d](#), [183c](#), [192](#), [196abd](#), [197b](#).
 delayed_indent: [54d](#), [147b](#), [149](#), [151c](#), [154](#), [155ab](#).
 depths: [184d](#), [189a](#), [190](#), [191](#).
 double_at: [126a](#), [128](#), [131a](#).
 dump_scrap_text: [116](#), [134b](#), [143](#).
 Embed_Node: [54d](#), [145c](#), [156c](#), [182d](#).
 emit_name_json: [114](#), [115a](#).
 EQUAL: [162a](#), [163a](#), [164a](#).
 exit: [22](#), [26a](#), [31](#), [36d](#), [37b](#), [41](#), [47c](#), [48a](#), [70b](#), [117b](#), [121](#), [123b](#), [128](#), [129](#), [130a](#), [131a](#), [137](#), [138](#), [140](#), [146b](#), [148](#), [154](#), [172](#), [176](#), [177a](#), [178bc](#), [180](#), [181](#), [186a](#), [193b](#), [204](#).
 EXTENSION: [162a](#), [163a](#), [164a](#).
 extract_flag: [28](#), [29a](#), [34](#), [36b](#).
 extract_fragment: [116](#), [117b](#).
 extract_name: [28](#), [29a](#), [36b](#), [41](#).
 extra_scrap: [66d](#), [67a](#), [67b](#), [67c](#), [79](#), [93bc](#), [103](#).
 FALLTHROUGH: [23a](#), [45](#), [61a](#), [74a](#), [90a](#), [127a](#), [163a](#).
 FALSE: [23a](#), [28](#), [29a](#), [34](#), [35abc](#), [36abc](#), [41](#), [60c](#), [61a](#), [62b](#), [66c](#), [67d](#), [69b](#), [79](#), [84a](#), [85b](#), [86b](#), [89ab](#), [90a](#), [91b](#), [94a](#), [96](#), [103](#), [107a](#), [108c](#), [110c](#), [112](#), [117a](#), [128](#), [131a](#), [154](#), [158a](#), [164a](#), [167](#), [168](#), [170](#), [173a](#), [174a](#), [194a](#), [196a](#), [197b](#), [198ab](#).
 fclose: [22](#), [59b](#), [88b](#), [121](#), [123ab](#), [124b](#), [130b](#), [158a](#).
 FILE: [22](#), [59ab](#), [65a](#), [69b](#), [71c](#), [72](#), [79](#), [80b](#), [82b](#), [86a](#), [88ab](#), [93a](#), [96](#), [97a](#), [102](#), [103](#), [104b](#), [106a](#), [108b](#), [110a](#), [111b](#), [113](#), [114](#), [116](#), [121](#), [123a](#), [124b](#), [126ab](#), [132b](#), [133ab](#), [134b](#), [143](#), [147a](#), [156b](#), [158a](#), [195ab](#), [196cd](#), [199ab](#), [204](#).
 file_line: [131c](#), [134d](#), [136c](#), [137](#), [149](#).
 file_name: [42ab](#), [58b](#), [59b](#), [87](#), [88b](#), [131c](#), [134c](#), [136c](#), [137](#), [150bc](#).
 file_names: [41](#), [42b](#), [54b](#), [80a](#), [104a](#), [117b](#), [122b](#), [161a](#), [161b](#), [172](#).
 find_lang: [168](#), [174a](#).
 find_name: [115b](#), [117b](#).
 first: [32](#), [110c](#), [111a](#), [112](#), [114](#), [115a](#), [117b](#), [132b](#), [133a](#), [196ab](#), [197b](#), [198b](#), [202a](#), [203b](#), [204](#).
 fopen: [22](#), [59b](#), [88b](#), [121](#), [123b](#), [124b](#), [129](#), [131a](#), [158a](#).
 format_defs_refs: [63](#), [64a](#), [196c](#), [196d](#).
 format_entry: [59a](#), [81b](#), [82b](#).

format_file_entry: 59a, 80a, [80b](#).
 format_user_entry: 59a, 85c, [86a](#).
 format_uses_refs: 63, 64a, 195a, [195b](#).
 fprintf: [22](#), 34, 37ab, 41, 42b, 45, 47c, 48a, 52a, 58a, 59b, 60b, 63, 65b, 68c, 69a, 70b, 76ab, 77, 79, 80c, 86b, 88b, 92a, 93b, 94c, 95b, 100bc, 101, 103, 104c, 108c, 110c, 112, 113, 114, 116, 117b, 121, 123ab, 124c, 128, 129, 130a, 131a, 133a, 134e, 137, 138, 140, 143, 146b, 148, 150bcd, 153, 154, 156ab, 163b, 170, 171, 172, 173ab, 176, 177a, 178bc, 180, 181, 186a, 193b, 196a, 197b, 199cd, 200ab.
 fputs: [22](#), 52a, 53a, 54d, 58a, 60a, 62a, 63, 64a, 65ab, 66abcd, 67a, 68abcd, 69ab, 71c, 72, 74a, 75a, 76ab, 77, 78, 79, 80ac, 81ab, 83b, 84ab, 85c, 86b, 91a, 92ab, 93ab, 94c, 95ab, 96, 97ab, 99a, 100ab, 101, 102, 103, 104acd, 105a, 106b, 107ab, 108ac, 110bc, 111ac, 112, 113, 114, 115a, 116, 121, 133a, 149, 153, 155a, 195c, 196ab, 197ab.
 getc: [22](#), 124b, 127a, 128, 129, 130ab, 131a.
 getenv: [22](#), 37a.
 get_scrap_defs: 111b, 132b, [134a](#).
 get_scrap_uses: 110a, 114, 116, 132b, [133c](#).
 goto_lookup: [185a](#), 190, 191, 192.
 Goto_Node: [184b](#), 184cd, 185a, 189a, 190, 191, 192.
 GREATER: [162a](#), 163a, 164a.
 has_sector: 81b, 85a, [85b](#), 85c, 105a, 108a.
 hyperoptions: [28](#), 29a, 36a, 60ab.
 hyperopt_flag: [28](#), 29a, 34, 36a.
 hyperref_flag: [28](#), 29a, 34, 36a, 60a.
 incl: [28](#), 29a, 35b, 40, 129.
 includepath_flag: [28](#), 29a, 34, 35b.
 include_depth: [126a](#), 129, 130b, 131a.
 init_scrap: 42b, 132b, [132c](#).
 instance: 62b, 91b, [145a](#), 145bc, 154.
 isgraph: [22](#), 130a, 172, 180.
 islower: [22](#), 165a.
 isspace: [22](#), 67a, 93c, 140, 172, 173a, 176, 178c.
 json_puts: [113](#), 114, 121.
 label_node: 199f, 200cd, [201a](#), 201b.
 label_tab: 200c, [200d](#), 201b.
 LangEntry: [168](#), 174a.
 lang_table: [168](#).
 lang_to_flags: [168](#), 174ab.
 LESS: [162a](#), 163a, 164a.
 letter: [131c](#), 133a, 136c, 159d.
 load_entry: [82a](#), 82b.
 lookup: 54d, [144b](#), 145c.
 macro_names: 41, 42b, 54b, 81b, 105a, 117b, 122b, [161a](#), 161b, 178b.
 main: [26a](#), 204.
 malloc: [22](#), 82b, 106a, 186a, 203a.
 Manager: [135c](#), 135d, 136ab, 142c, 143, 144a, 146b, 149, 185c, 187c, 192.
 map_flag: [28](#), 29a, 34, 41.
 markdown_flag: [28](#), 29a, 34, 41.
 MarkerFrame: [118](#), 119, 121.
 MAX_ARGLISTS: [148](#).
 max_depth: [184d](#), 189a, 190, 191.
 MAX_INDENT: 62b, 91b, [122d](#), 123a, 150a, 151b.
 MAX_NAME_LEN: [23b](#), 140, 143, 146b, 172, 176, 180, 182a, 193b, 198c.
 md_copy_scrap: 88a, 93b, 94a, [97a](#).
 md_format_defs_refs: 88a, 94bd, [111b](#).
 md_format_entry: 88a, 105a, [106a](#).
 md_format_file_entry: 88a, 104a, [104b](#).
 md_format_user_entry: 88a, 108a, [108b](#).
 md_format_uses_refs: 88a, 94bd, [110a](#).
 md_load_entry: [105b](#), 106a.
 md_print_scrap_numbers: 88a, 94c, 95ab, [96](#), 104d, 107b.

md_scrap_type: [97a](#), [99a](#).
 md_write_arg: [88a](#), [92c](#), [93a](#), [99a](#), [103](#).
 md_write_ArglistElement: [88a](#), [101](#), [103](#).
 md_write_literal: [88a](#), [101](#), [102](#), [103](#).
 Move_Node: [184c](#), [185a](#), [190](#), [191](#).
 Name: [32](#), [46e](#), [47a](#), [54d](#), [59a](#), [62b](#), [63](#), [64a](#), [72](#), [77](#), [79](#), [80b](#), [82ab](#), [83a](#), [85ab](#), [86a](#), [88a](#), [91b](#), [92ab](#), [97a](#), [101](#), [103](#), [104b](#), [105b](#), [106a](#), [108b](#), [110c](#), [112](#), [114](#), [115ab](#), [116](#), [117ab](#), [122abc](#), [140](#), [141c](#), [142ab](#), [144b](#), [145ac](#), [154](#), [156b](#), [160b](#), [161abc](#), [163a](#), [164a](#), [166](#), [167](#), [172](#), [176](#), [178d](#), [179abc](#), [181](#), [182c](#), [183c](#), [184a](#), [187a](#), [189ab](#), [192](#), [193c](#), [194bc](#), [196a](#), [197b](#), [198b](#).
 name_add: [141ab](#), [161c](#), [166](#), [172](#).
 Name_Node: [184a](#), [184b](#), [190](#), [191](#), [192](#).
 next: [28](#), [32](#), [34](#), [35b](#), [40](#), [47b](#), [54d](#), [68cd](#), [69ab](#), [77](#), [78](#), [79](#), [81a](#), [84ab](#), [86b](#), [94c](#), [95ab](#), [96](#), [101](#), [103](#), [104d](#), [107ab](#), [108c](#), [110bc](#), [111c](#), [112](#), [114](#), [116](#), [125a](#), [129](#), [131b](#), [135d](#), [141ab](#), [142ac](#), [144b](#), [145ac](#), [146a](#), [147a](#), [156a](#), [157a](#), [160a](#), [179bc](#), [182c](#), [183abd](#), [184abc](#), [185a](#), [187a](#), [190](#), [191](#), [192](#), [194abc](#), [195c](#), [196a](#), [197ab](#), [201d](#), [202c](#), [203a](#), [204](#).
 number_flag: [28](#), [29a](#), [34](#), [41](#), [158a](#).
 num_scraps: [82b](#), [106a](#), [132a](#), [132b](#).
 nw_char: [29b](#), [29c](#), [41](#), [42c](#), [45](#), [47c](#), [48a](#), [49b](#), [51b](#), [52b](#), [53d](#), [56](#), [57](#), [58a](#), [60c](#), [61a](#), [63](#), [70a](#), [71c](#), [72](#), [73a](#), [74a](#), [75d](#), [76a](#), [80c](#), [86b](#), [89a](#), [90a](#), [97a](#), [99a](#), [100b](#), [127a](#), [128](#), [131a](#), [137](#), [138](#), [139a](#), [140](#), [141cd](#), [143](#), [146b](#), [149](#), [151c](#), [152a](#), [153](#), [154](#), [156ab](#), [157a](#), [163b](#), [170](#), [171](#), [172](#), [176](#), [177ab](#), [178c](#), [180](#), [181](#), [182a](#), [193ab](#), [196a](#), [197b](#), [198ac](#).
 op_char: [198a](#), [198b](#).
 output_flag: [28](#), [29a](#), [34](#), [41](#).
 page: [69b](#), [84a](#), [86b](#), [96](#), [107a](#), [108c](#), [110c](#), [111a](#), [112](#), [131c](#), [132b](#), [133ab](#), [136c](#), [158ab](#), [159bd](#), [196ab](#), [197b](#), [204](#).
 Parameters: [54c](#), [57](#), [62b](#), [91b](#), [132b](#), [146b](#), [147a](#), [149](#).
 parse_close_marker: [120](#), [121](#).
 parse_open_marker: [119](#), [121](#).
 pass1: [41](#), [42a](#), [42b](#), [74a](#), [99a](#).
 pending_lang: [43a](#), [161a](#), [161b](#), [174ab](#).
 pop: [53abd](#), [57](#), [142c](#), [143](#), [146bc](#), [149](#), [151a](#), [152ac](#), [186b](#).
 pop_scrap_name: [146b](#), [154](#).
 PREFIX: [162a](#), [163a](#), [164a](#).
 prefix_add: [161c](#), [163a](#), [178b](#).
 prev_char: [187c](#), [198b](#).
 prev_sector: [41](#), [46a](#), [46c](#), [46d](#).
 print_scrap_numbers: [59a](#), [68cd](#), [69a](#), [69b](#), [81a](#), [84b](#).
 push: [52b](#), [56](#), [135d](#), [136a](#), [137](#), [138](#), [139a](#), [141cd](#).
 pushArglist: [186a](#), [187a](#), [193b](#).
 pushes: [56](#), [136a](#), [139a](#), [141d](#).
 putc: [22](#), [53ab](#), [60c](#), [61a](#), [72](#), [73c](#), [77](#), [79](#), [80c](#), [81a](#), [83b](#), [84ab](#), [86b](#), [89ab](#), [90a](#), [97ab](#), [98](#), [99a](#), [101](#), [103](#), [104c](#), [106b](#), [107a](#), [133a](#), [149](#), [151bd](#), [152a](#), [154](#), [155abc](#).
 reject_match: [189a](#), [192](#), [198b](#).
 remove: [22](#), [124ab](#).
 reverse: [28](#), [183c](#), [183d](#).
 reverse_lists: [54b](#), [161c](#), [183c](#).
 reverse_map_arg: [28](#), [29a](#), [36cd](#).
 reverse_map_flag: [28](#), [29a](#), [34](#), [36c](#).
 robs_strcmp: [82c](#), [115b](#), [164b](#), [165a](#), [166](#), [167](#), [194b](#).
 root: [184d](#), [189a](#), [190](#), [191](#), [192](#), [193c](#), [194b](#).
 save_string: [35b](#), [40](#), [129](#), [136c](#), [161c](#), [162b](#), [163a](#), [167](#), [179a](#), [183b](#).
 SCRAP: [132a](#), [132c](#), [136c](#).
 ScrapEntry: [131c](#), [132ac](#), [136c](#).
 scraps: [24bc](#), [28](#), [58a](#), [59a](#), [63](#), [64a](#), [65b](#), [67a](#), [69b](#), [88a](#), [92ab](#), [93c](#), [94bd](#), [96](#), [114](#), [132a](#), [132c](#), [136c](#), [158ab](#), [159bc](#), [192](#), [204](#).
 scrap_array: [132a](#), [133ac](#), [134acd](#), [136c](#), [137](#), [141ab](#), [143](#), [149](#), [150bc](#), [158ab](#), [159bd](#), [192](#), [195b](#), [196d](#).
 SCRAP_BITS: [132a](#).
 scrap_file_line: [114](#), [116](#), [134b](#), [134d](#).
 scrap_file_name: [114](#), [116](#), [134b](#), [134c](#).
 scrap_flag: [28](#), [29a](#), [34](#), [63](#), [94b](#).
 scrap_is_in: [192](#), [193c](#), [194a](#).

SCRAP_MASK: [132a](#), [136c](#).
 Scrap_Node: [47b](#), [54d](#), [59a](#), [69b](#), [78](#), [79](#), [81a](#), [84ab](#), [86b](#), [88a](#), [96](#), [101](#), [103](#), [104d](#), [107ab](#), [108c](#), [110c](#), [112](#), [114](#), [116](#), [132b](#), [141ab](#), [142a](#), [145c](#), [147a](#), [160a](#), [160b](#), [182cd](#), [183cd](#), [192](#), [193c](#), [194a](#), [196a](#), [197b](#).
 SCRAP_SHIFT: [132a](#), [136c](#).
 SCRAP_SIZE: [132a](#), [132c](#), [136c](#).
 scrap_type: [52a](#), [71b](#), [72](#), [74a](#), [76ab](#), [77](#), [79](#).
 search: [32](#), [42b](#), [188c](#), [189a](#), [204](#).
 sector: [58b](#), [59ab](#), [81b](#), [82b](#), [83b](#), [85bc](#), [86ab](#), [87](#), [88ab](#), [105a](#), [106ab](#), [108abc](#), [114](#), [131c](#), [136c](#), [140](#), [141abd](#), [143](#), [154](#), [157a](#), [160b](#), [161c](#), [163ab](#), [164a](#), [166](#), [167](#), [176](#), [178b](#), [180](#), [192](#).
 setlocale: [22](#), [37a](#).
 size_t: [22](#), [119](#), [201c](#), [202b](#), [203a](#).
 Slab: [131b](#), [131c](#), [135cd](#), [136c](#), [142c](#).
 slab: [131b](#), [131c](#), [136c](#), [143](#), [149](#), [192](#).
 SLAB_SIZE: [131b](#), [135d](#), [142c](#), [144a](#), [188b](#).
 source_file: [126a](#), [127ac](#), [128](#), [129](#), [130ab](#), [131a](#).
 source_get: [42c](#), [43a](#), [44](#), [45](#), [46ab](#), [47c](#), [48a](#), [49ab](#), [50a](#), [51ab](#), [56](#), [58a](#), [60c](#), [61a](#), [62ab](#), [67ad](#), [72](#), [74a](#), [75cde](#), [76a](#), [80a](#), [81b](#), [85c](#), [89ab](#), [90a](#), [91ab](#), [93c](#), [94a](#), [97a](#), [99a](#), [100b](#), [104a](#), [105a](#), [108a](#), [125a](#), [127a](#), [129](#), [130b](#), [137](#), [138](#), [139b](#), [140](#), [141c](#), [170](#), [171](#), [172](#), [173ab](#), [176](#), [177ab](#), [178c](#), [180](#), [181](#), [182a](#).
 source_last: [58a](#), [72](#), [97a](#), [125a](#), [127a](#).
 source_line: [45](#), [47c](#), [48a](#), [125b](#), [125c](#), [127ac](#), [128](#), [129](#), [130ab](#), [131a](#), [136c](#), [138](#), [140](#), [163b](#), [170](#), [171](#), [172](#), [173ab](#), [176](#), [178b](#), [180](#), [181](#).
 source_name: [38](#), [39](#), [40](#), [41](#), [45](#), [47c](#), [48a](#), [125b](#), [125c](#), [128](#), [129](#), [130ab](#), [131a](#), [136c](#), [138](#), [140](#), [163b](#), [170](#), [171](#), [172](#), [173ab](#), [176](#), [177a](#), [178bc](#), [180](#), [181](#).
 source_open: [42b](#), [59b](#), [88b](#), [125a](#), [131a](#).
 source_peek: [49a](#), [50a](#), [51a](#), [125a](#), [127a](#), [127c](#), [128](#), [129](#), [130b](#), [131a](#).
 source_ungetc: [49b](#), [50a](#), [51b](#), [81b](#), [105a](#), [127b](#), [127c](#).
 stack: [121](#), [126b](#), [129](#), [130b](#).
 stderr: [22](#), [34](#), [37ab](#), [41](#), [42b](#), [45](#), [47c](#), [48a](#), [59b](#), [68c](#), [69a](#), [70b](#), [77](#), [79](#), [88b](#), [94c](#), [95b](#), [101](#), [103](#), [117b](#), [121](#), [123ab](#), [124c](#), [128](#), [129](#), [130a](#), [131a](#), [134e](#), [137](#), [138](#), [140](#), [146b](#), [148](#), [154](#), [163b](#), [170](#), [171](#), [172](#), [173ab](#), [176](#), [177a](#), [178bc](#), [180](#), [181](#), [186a](#), [193b](#), [199d](#), [200ab](#).
 strlen: [22](#), [53a](#), [121](#), [158b](#), [162ab](#), [188a](#), [198b](#).
 sym_char: [198a](#), [198b](#).
 tex_flag: [28](#), [29a](#), [34](#), [41](#), [42b](#), [154](#).
 toupper: [22](#), [165a](#).
 TRUE: [23a](#), [28](#), [29a](#), [34](#), [36a](#), [41](#), [58a](#), [61a](#), [62b](#), [66ad](#), [69b](#), [79](#), [84a](#), [85b](#), [86b](#), [89ab](#), [90a](#), [91b](#), [93bc](#), [96](#), [103](#), [107a](#), [108c](#), [110c](#), [112](#), [116](#), [128](#), [133b](#), [134e](#), [154](#), [164a](#), [167](#), [168](#), [170](#), [173a](#), [194a](#), [196a](#), [197b](#), [198ab](#).
 update_delimit_scrap: [45](#), [61a](#), [73a](#), [73b](#), [90a](#).
 user_langs: [168](#), [170](#).
 user_lang_count: [168](#), [170](#).
 user_lang_names: [168](#), [170](#).
 user_names: [42b](#), [54b](#), [85c](#), [108a](#), [141ab](#), [161a](#), [161b](#), [189a](#).
 Uses: [60a](#), [110ab](#), [111b](#), [114](#), [116](#), [132b](#), [133c](#), [134a](#), [193c](#), [194b](#), [194c](#), [195b](#), [196d](#).
 uses: [32](#), [69a](#), [84b](#), [86b](#), [95b](#), [107b](#), [108c](#), [112](#), [114](#), [116](#), [131c](#), [133c](#), [136c](#), [141a](#), [142a](#), [160b](#), [167](#), [168](#), [183c](#), [192](#), [194c](#), [195b](#), [197b](#).
 verbose_flag: [28](#), [29a](#), [34](#), [42b](#), [59b](#), [88b](#), [123a](#).
 version_info_flag: [28](#), [29a](#), [34](#), [35c](#).
 version_string: [28](#), [29a](#), [35c](#), [62a](#), [75a](#), [91a](#).
 weave_flag: [28](#), [29a](#), [34](#), [41](#).
 weave_format: [28](#), [29a](#), [34](#), [41](#), [171](#).
 weave_format_from_cli: [28](#), [29a](#), [34](#), [41](#), [171](#).
 write_arg: [59a](#), [64b](#), [65a](#), [74a](#), [79](#).
 write_ArglistElement: [59a](#), [77](#), [79](#).
 write_extract: [41](#), [117b](#), [122b](#).
 write_files: [41](#), [122a](#), [122c](#).
 write_label: [75b](#), [152b](#), [199a](#), [199b](#).
 write_literal: [59a](#), [71c](#), [77](#), [79](#).
 write_map: [41](#), [115a](#), [122b](#).
 write_md: [41](#), [87](#), [88b](#).

write_reverse_map: 36d, [121](#), 122b.
write_scraps: 54d, 62b, 79, 91b, 103, 123a, 132b, [147a](#), 154, 156c.
write_scrap_ref: 69b, 84a, 86b, 96, 107a, 108c, 111a, 132b, [133a](#), 133b, 196b.
write_single_scrap_ref: 58a, 63, 64a, 69a, 78, 81a, 84b, 86b, 92ab, 95b, 101, 103, 104d, 107b, 108c, 132b, [133b](#),
155ac, 199c.
write_tex: [41](#), 58b, [59b](#).
xref_flag: [28](#), 29a, 34, 155ac.

Bibliography

- [1] A[lfred] V. Aho and M[argaret] J. Corasick, *Efficient string matching: An aid to bibliographic search.*, CACM **18** (1975), no. 6, 333–340. [185](#)
- [2] C[hris] Hanson, *Efficient stack allocation for tail-recursive languages.*, (1990). [202](#)
- [3] Donald E. Knuth, *Literate programming*, The Computer Journal **27** (1984), no. 2, 97–111. [1](#)
- [4] ———, *The T_EXbook*, Computers and Typesetting, vol. 1986aA, Addison-Wesley, Reading, MA, USA, 1986. [1](#)
- [5] ———, *T_EX: The program*, Computers and Typesetting, vol. B, Addison-Wesley, Reading, MA, USA, 1986b1986. [1](#)
- [6] ———, *METAFONT: THE PROGRAM*, Computers and Typesetting, vol. D, Addison-Wesley, Reading, MA, USA, 1986d1986. [1](#)
- [7] Leslie Lamport, *L^AT_EXa document preparation system user's guide and reference manual*, Addison-Wesley, Reading, MA, USA, 1985. [1](#)
- [8] Silvio Levy, *WEB adapted to C, another approach*, TUB **8** (1987), no. 1, 12–13. [1](#)
- [9] Norman Ramsey, *Literate-programming tools need not be complex*, Report at `ftp.cs.princeton.edu` in `/reports/1991/351.ps.Z`. Software at `ftp.cs.princeton.edu` in `/pub/noweb.shar.Z` and at `bellcore.com` in `/pub/norman/noweb.shar.Z`. CS-TR-351-91, Department of Computer Science, Princeton University, August 1992, Submitted to *IEEE Software*. [1](#)
- [10] Ross Williams, *Funnelweb user's manual*, `ftp.adelaide.edu.au` in `/pub/compression` and `/pub/funnelweb`, University of Adelaide, Adelaide, South Australia, Australia, 1992. [1](#)